

Moving a Step of ChaCha in Syncopated Rhythm

Shichang Wang^{1,2}, Meicheng Liu^{1,2} (✉), Shiqi Hou^{1,2}, Chengan Hou^{1,2}, Dongdai Lin^{1,2}

Abstract

The stream cipher ChaCha is one of the most widely used ciphers in the real world, such as in TLS, SSH and so on. In this paper, we study the security of ChaCha via differential cryptanalysis based on probabilistic neutral bits (PNBs). We introduce the *syncopation* technique for the PNB-based approximation in the backward direction, which significantly amplifies its correlation by utilizing the property of ARX structure. In virtue of this technique, we present a new and efficient method for finding a good set of PNBs, and then a refined framework of key-recovery attack is formalized for round-reduced ChaCha. Further, we generalize the PNB-based approximation by a concept called *probabilistic neutral expressions* (PNEs). In the PNE-based framework, a new key guessing strategy is presented along with the *carry-preserving* technique. The new techniques allow us to break 7.5 rounds of 256-bit ChaCha, as well as to bring faster attacks on 7 rounds of 256-bit ChaCha. In addition, to the best of our knowledge, we present the first related-key attack on 256-bit ChaCha8 which is one out of three original ciphers in the ChaCha family. Regarding 128-bit ChaCha, our techniques permit us to defeat 7 rounds when excluding the last rotation.

Index Terms

Stream Ciphers, ChaCha, Differential Cryptanalysis, PNB, Syncopation, PNE, Carry-preserving, New key guessing strategy.

I. INTRODUCTION

SYMMETRIC key cryptosystems play an indispensable role in cryptography. Owing to the significant performance advantage of symmetric cryptographic primitives, there is massive deployment of symmetric primitives, including stream ciphers, block ciphers, hash functions, and cryptographic permutations, in virtually all real-world applications and scenarios related to cryptography. Among them, ARX-based design is one important and attractive branch with simplicity and efficiency in both software and hardware implementations, especially in consideration of the persistent focus on lightweight cryptography. ARX is short for modular Addition, Rotation and bit-wise XOR. ARX-based designs not only have very high efficiency, but also provide good security properties. The algebraic degree of ARX ciphers is usually high after a very few rounds since the function of carry bit within one modular addition already reaches almost maximal degree. With regard to differential [BS90] and linear [Mat93] attacks, the probabilities of differentials and absolute correlations of linear approximations decrease very quickly when the numbers of rounds increase.

One of the most important sets of ARX ciphers is the family consisting of Salsa20, ChaCha, and their variants. In 2005, Bernstein proposed the stream cipher Salsa20 [Ber08b] as a candidate for the eSTREAM competition [eST], and its 12-round variant was accepted into the final software portfolio. Later, as a variant of Salsa20, ChaCha [Ber08a] was proposed by Bernstein in 2008 to provide better diffusion and cryptanalytic resistance without slowing down encryption. The stream cipher ChaCha has a total of 20 rounds. In addition, the ChaCha family also includes reduced-round ciphers—the 8-round variant ChaCha8 and the 12-round variant ChaCha12. For ChaCha, there are two versions with a 256-bit key and a 128-bit one, which we will refer to as 256-bit ChaCha and 128-bit ChaCha respectively. In the following, we mainly concentrate on 256-bit ChaCha due to its wide deployment.

ChaCha, along with the message authentication code Poly1305 [Ber05], is adopted as one of the cipher suites of Transport Layer Security (TLS) [LCM⁺16], which has been supported by Google on both Chrome and Android. Also, the RFC 7634 [Nir15] proposes the use of ChaCha along with Poly1305 for IKEv2 and IPsec. In addition, it has been implemented in many other protocols such as SSH, Noise, QUIC, WireGuard, S/MIME 4.0 and so on. Apart from the usage of encryption, ChaCha has also been used as a pseudo-random number generator in the operating system with Linux kernel. Up to now, there are plenty of protocols and software which have implemented the stream cipher ChaCha, and please refer to [IAN] for details. In a nutshell, ChaCha is one of the most widely-used ciphers in practice.

1) *Related works*: Because of the wide range of usage and deployment of ChaCha, it is crucial to systematically and deeply analyze its security. And indeed, cryptanalysts have advanced lots of important and profound works on the security evaluation of round reduced ChaCha.

At FSE 2008, Aumasson et al. [AFK⁺08] proposed a significant improvement on the differential cryptanalysis of both Salsa20 and ChaCha with introducing a new concept called *probabilistic neutral bits* (shortcut PNBs). In most cases, the attacks are launched by a meet-in-the-middle approach, in which for forward direction one applies some input difference to the initial state to observe the output difference after certain rounds, and for backward direction once a set of key bits with

This paper is an extended version of the previous work [WLHL23], including new techniques and results, as well as all the proofs of theorems.

¹Key Laboratory of Cyberspace Security Defense, Institute of Information Engineering, CAS, Beijing, China

²School of Cyber Security, University of Chinese Academy of Sciences, Beijing, China

Email: {wangshichang, liumeicheng, houshiqi, houchengan, ddlin}@iie.ac.cn

less influence on the output difference is identified using the PNB method, attackers can obtain the output difference at the middle from the final state. Since most of the attacks on Salsa and ChaCha to date have been carried out based on PNBs, this idea will be reviewed in detail in Section III-A.

Several further enhancements have been proposed following this line of research. In 2012, Shi et al. [SZFW12] achieved some incremental advancements for both Salsa20 and ChaCha by introducing the concept of *column chaining distinguisher* (CCD). Maitra [Mai16] provided the idea of chosen IV cryptanalysis to obtain certain improvements in the key-recovery attacks of both ciphers. In 2017, Choudhuri and Maitra [CM16] improved the attacks with the idea of extending single-bit differentials to multi-bit differentials using linear approximations derived theoretically, which is essentially a differential-linear cryptanalysis [LH94]. Dey and Sarkar [DS17] improved the attacks by giving an algorithm for constructing a good PNB set, and then in [DS20] they provided a theoretical justification of the distinguishers of both ciphers. At CRYPTO 2020, Beierle, Leander and Todo [BLT20] provided the first 3.5-round single-bit distinguisher, and improved the attacks for ChaCha7 with time complexity of $2^{230.86}$. Later, Coutinho and Souza Neto [CN21b] presented a few more distinguishers for 3.5 rounds of ChaCha, and provided a further improvement on the differential-linear attacks including the distinguishers and key-recovery attacks for 7 rounds of ChaCha. However, in a recent work [DDSM22], Dey et al. showed that the correlation of differential at 3.5-th round in [CN21b] is much smaller than their claim which results to inaccurate complexities of the attacks. The authors of [CN21b] immediately corrected this mistake and reevaluated the complexities in [CN21a]. In [DDSM22], Dey et al. also provided a theoretical explanation of the issue that not all keys have available IV's to satisfy the given differential, called *strong keys* at [BLT20], mainly for two special cases, and provided experimental results for the other six cases without theoretical explanations. Subsequently, at EUROCRYPT 2022, Dey et al. [DGSS22] further improved the attack on ChaCha7 with time complexity of $2^{221.95}$ by introducing a concept of *exploitable keys* and constructing a list to store combinations of exploitable keys with a favorable IV for each key. At ASIACRYPT 2022, Coutinho et al. [CPV⁺22] presented a differential-linear distinguisher against 7 rounds of ChaCha by improving the linear part, with both time and data complexities of 2^{214} . Dey et al. [DGM23] presented a full key recovery attack on ChaCha6 with a complexity of $2^{99.48}$.

Since the first published analysis of ChaCha by Aumasson et al. in 2008 [AFK⁺08], its valid attacks documented in the literature have stayed within 7 rounds [SZFW12], [CM16], [BLT20], [DGSS22]. In [MIM22], the authors presented an attack on 7.25 rounds of ChaCha with time complexity of $2^{255.62}$ and success probability of 0.5. But as declared in [MIM22], this attack is less efficient than a brute force attack.

Recently, Dey et al. [DGSS23] demonstrated that the assignment of PNBs affects the correlation of PNB-based approximation, and improved their previous results using the special assignment $10 \cdots 0$ of PNBs. At FSE 2024, Bellini et al. [BGG⁺23] presented a differential-linear distinguisher of five-round ChaCha with MILP, and presented vast improved differential-linear cryptanalysis of ChaCha. However, since the data complexity of attacks exceeds 2^{96} the limitation for using the technique of Sect 3.1 of [BLT20], the key-recovery attacks on ChaCha7 and ChaCha7.25 [BGG⁺23] can no longer use the technique in [BLT20] to achieve the speedup by a factor of 2^7 . Very recently, in [Dey24], Dey presented a way to improve the key-recovery attack of ChaCha in which they pre-compute and store the values for split target parts in several huge tables, and online combine the corresponding entries of these huge tables by XOR operations. However, the attack on ChaCha7.5 [Dey24] with time complexity $2^{255.24}$ is worse than a brute-force attack since its success probability is around 0.5 and the complexity should be multiplied by a factor of two (the input pairs are used in the differential attack). Besides, in the complexity calculation of [Dey24], the cost of accessing huge tables is neglected. We refer to Appendix A for detailed discussions of the attack complexity of [BGG⁺23], [Dey24]. In parallel with our work, Xu et al. [XXTQ24] presented a more accurate correlation of differential-linear distinguisher for five-round ChaCha by differential-linear hull, and improved key-recovery attacks of ChaCha.

2) *Contributions:* The paper significantly improves the key-recovery attacks on 256-bit ChaCha and presents an 7.5-round attack in the single-key setting, and shows an attack on 256-bit ChaCha8 in the related-key setting in which one pair of keys with a specific difference is used. Besides, the paper also provides the security evaluation of 128-bit ChaCha. In the following, firstly the technical contributions are introduced, and then the application results on ChaCha are explained.

a) *Technical contributions:* So far, the best results of key-recovery attacks against ChaCha are obtained by the PNB-based differential cryptanalysis. First, we proposed several new techniques to enhance the PNB-based differential cryptanalysis. Then, we generalized the PNB-based framework along with a new key guessing strategy.

- **Refined PNB-based Differential Cryptanalysis.** The PNB-based differential cryptanalysis is composed of two parts: forward truncated differential and backward PNB-based approximation. For the two parts, we have proposed several new techniques and improvements, which are summarized as follows.

- **Backward PNB-based approximation:** In the PNB-based approximation, the essential problem is how to find numerous PNBs with a large correlation of backward approximation. Unfortunately, more PNBs and larger correlation are usually contradictory. To simplify the problem, we first show that the backward approximation $\Delta f = \Delta g$ connected with the forward differential corresponding to two initializations can be seen as applying twice an approximation $f = g$ in a single initialization, and thus its correlation is the square of the correlation of the latter (see also Proposition 1). This fact is simple and was not illuminated in the previous analysis, but it is surprising that it significantly simplifies calculations of the backward correlation. By this observation, we can treat the PNB-based approximation similar to the linear part in differential-linear cryptanalysis. However, the PNB-based approximation is

much more complicated. We then introduce the technique called *syncopation* that takes advantage of ARX structures to analyze the propagation characteristic of PNBs and amplify the correlation of the approximation. To further facilitate the analysis, the approximated part f is divided into two parts f_0 and f_1 , that is, $f = f_1 \circ f_0$. The PNB-based approximating function g is a restriction of f with PNBs to be a fixed value, and this relationship also applies to g_0 and f_0 . The *syncopation* technique is used in the g_0 (to say f_0) part for reducing its dependence on PNBs so that the correlation between g_0 and f_0 is high. We analyze the properties of syncopation in the basic operations of ARX designs, especially including modular subtraction and modular addition. We also present a tool to determine syncopations, which leads to a new and efficient method to find a good set of PNBs with a large correlation by restrictions on non-PNBs. By the new observations and techniques, a refined framework of the PNB-based differential attack is formalized as described in Section IV.

- **Forward differential:** By analyzing the equations that control differential propagation, we achieve two main improvements in the differential part. Firstly, we show several useful observations on these differential equations, and propose a concise and effective way to satisfy the given differential. This treatment leads to a save of time by 2^{-2} in the key recovery at the cost of 2^4 times of data, compared with the work of Beierle, Leander and Todo [BLT20]. Compared with Dey et al.'s technique [DGSS22], it does not require restrictions on exploitable keys (which shrink the space of all possible PNBs) and saves up to 2^{22} times of data. Secondly, a more comprehensive theoretical explanation is presented for the proportion of *strong keys* (about 30%) reported in [BLT20], which covers all the remaining six (out of eight) cases lacked in the previous theoretical analysis [DDSM22]. The details are shown in Section VI.
- **Generalized PNE-based Framework.** To find a better approximation with high correlation and fewer secret bits, we further generalized the PNB-based approximation method by a concept called the *probabilistic neutral expression* (PNE). In the PNE-based framework, probabilistic neutral expressions of the secret key are identified instead of directly finding non-significant secret bits in the traditional method. New techniques, such as the *carry-preserving* technique and a new key guessing strategy, have been developed in the PNE-based framework.
 - **Carry-Preserving Technique:** In the PNB-based approximation, only part of the secret information is guessed, that is exhausting the significant bits and leaving the non-significant ones (PNBs) unknown. For ARX ciphers, modular addition is the only non-linear operation in the approximation, which can be separated into XOR of three parts from a bit-wise perspective, i.e., two input vectors and a carry vector, where the carry vector is non-linearly dependent on the input vectors. For modular addition, we proposed the *carry-preserving* technique, which completely prevents the unknown information from entering the carry vector in the approximation. Thus, we can consider the carry vector as a fully determined known part, so the non-linear operation modular addition is simplified. As a result, this technique can not only amplify the correlation of PNB-based approximation, but also simplify the target function in the approximation which is utilized by the following new key guessing strategy.
 - **New Key Guessing Strategy:** Let $H : \mathbb{F}_2^n \rightarrow \mathbb{F}_2^n$ be an invertible function of the secret key K . We find that guessing the equivalent key $H(K)$ instead of the original key K can reduce the number of guessing key in backward approximation. The choice of key function H depends on the specific structure of target ciphers. Specifically, a common structure in ARX ciphers is analyzed, which consists of Rotation and XOR and leads to the linear equivalent key. To adapt the new key guessing strategy, we proposed the *probabilistic neutral expression* (PNE) concerning $H(K)$ in backward approximation, which is a generalization of probabilistic neutral bits. Then, with the carry-preserving technique, an efficient method is presented to find conditional PNEs in the PNE-based framework. The results show that more PNEs are found while maintaining a high correlation, especially for the extreme case where very few PNBs could be found by traditional PNB method.

b) **Application results:** As applications of the proposed techniques, we present several attacks on round-reduced ChaCha, targeting both 256-bit and 128-bit key variants.

- **Attacks on 256-bit ChaCha.** With regard to ChaCha7, several attacks are obtained with different complexities. Specifically, we can launch a key-recovery attack on ChaCha7 with a time complexity of $2^{210.3}$ using a data complexity of $2^{103.3}$. With a data complexity of $2^{68.9}$, our attack on ChaCha7 takes a time complexity of $2^{216.9}$. The data and time complexity of this attack are both less than the previous best known attack with a time complexity of $2^{221.95}$ [DGSS22]. With the help of the syncopation technique, we can further improve the efficiency of the recent new attacks [XXTQ24], and obtain key-recovery attacks on ChaCha7 and ChaCha7.25 with time complexities $2^{187.24}$ and $2^{220.04}$ respectively. Additionally, leveraging the hybrid use of the syncopation and carry-preserving techniques and a new 3.5-round distinguisher in the generalized PNE-based framework, we introduce an effective attack on 7.5 rounds of ChaCha with time complexity of $2^{254.6}$ and data complexity of $2^{125.3}$, which outperforms a brute force attack.

As far as we know, these attacks are the best known (partial) key-recovery attacks on ChaCha7, ChaCha7.25 and ChaCha7.5. In particular, this is the first time that a key recovery attack is achieved on 7.5 rounds of ChaCha which is faster than a brute force attack. As mentioned earlier, since the first 7-round attack was presented by Aumasson et al. [AFK⁺08] in 2008, nearly fifteen years have passed while no public cryptanalytic results superior to a brute force have been proposed for more than 7 rounds of ChaCha. In consequence, we take a small but difficult step forward towards the

security analysis of ChaCha.

Related-Key Attack on ChaCha8. The paper also evaluated the security of ChaCha8 in the related-key setting where one pair of keys with a fixed difference is used. In the generalized PNE-based framework, we show a key-recovery attack with time complexity of $2^{253.3}$ and data complexity of $2^{99.2}$ for ChaCha8 by utilizing the syncopation and carry-preserving techniques. Specially, to the best of our knowledge, we present the first related-key attack on 256-bit ChaCha8 which is one out of three original ciphers in the ChaCha family [Ber08a]. At Real-World Crypto 2020, Aumasson [Aum19] argues in favor of using ChaCha8 in practice. Our results show ChaCha8 is theoretically attacked by our techniques, that is ChaCha8 fails to achieve the 256-bit security if one pair of keys with a fixed difference is permitted in usage.

- **Attacks on 128-bit ChaCha.** As for 128-bit ChaCha, we present an improved key-recovery attack on ChaCha6.5 with a time complexity of $2^{114.3}$ and a data complexity of $2^{100.3}$. Aimed at higher rounds, ChaCha7 $\llcorner$ is defined as the variant of 7 rounds of 128-bit ChaCha without the last rotation. Our attack on ChaCha7 $\llcorner$ takes the time complexity of $2^{122.9}$ and data complexity of $2^{108.8}$.

TABLE I: Key-recovery attacks on ChaCha

Cipher	Rounds	Time	Data	Reference
256-bit ChaCha	7	2^{248}	2^{27}	[AFK ⁺ 08]
		$2^{246.5}$	2^{27}	[SZFW12]
		$2^{237.7}$	2^{96}	[CM16]
		$2^{230.86}$	$2^{48.83}$	[BLT20]
		$2^{221.95}$	$2^{75.89\dagger}$	[DGSS22]
		$2^{218.92}$	$2^{72.89\dagger}$	[DGSS23]
		$2^{213.8\dagger}$	$2^{117.81\dagger}$	[BGG ⁺ 23]
		$2^{192.89\perp}$	$2^{93.79}$	[Dey24]
		$2^{189.7}$	$2^{102.63}$	[XXTQ24]
		$2^{216.9}$	$2^{68.9}$	Sect. VII-A
		$2^{210.3}$	$2^{103.3}$	Sect. VII-A
		$2^{187.2*}$	$2^{108.2}$	Sect. VII-A
	7.25	$2^{244.85}$	$2^{74.75\dagger}$	[DGSS23]
		$2^{245.34\dagger}$	$2^{122.34}$	[BGG ⁺ 23]
		$2^{228.24\perp}$	$2^{100.90}$	[Dey24]
		$2^{223.9}$	$2^{100.8}$	[XXTQ24]
	7.5	2^{220*}	2^{109}	Sect. VII-B
		$2^{254.6*}$	$2^{125.3}$	Sect. VII-C
	8 (RK)	$2^{253.3*}$	$2^{99.2}$	Sect. VII-D
128-bit ChaCha	6.5	$2^{123.04}$	$2^{66.94}$	[DGSS22]
		$2^{121.40}$	$2^{66.29}$	[DGSS23]
		$2^{113.08}$	$2^{37.27}$	[Dey24]
		$2^{114.3*}$	$2^{100.3}$	Sect. VIII-B
	7 $\llcorner$	$2^{122.9*}$	$2^{108.8}$	Sect. VIII-A

* The attacks are introduced in the extended paper.

$\dagger$ The complexity is recalculated by our evaluation; see Appendix A for details.

$\perp$ The cost of memory access is not taken into account; see Appendix A for details.

Our key-recovery attacks on round-reduced ChaCha are summarized in Table I, with a comparison of the best known attacks that are better than brute force. Last but not least, it is important to emphasize that our attacks do not pose any threat on ChaCha with full 20 rounds in the actual deployment.

Enhancements in the Extended Paper. Compared to the previous version at CRYPTO 2023 [WLHL23], this extended version introduces the generalized PNE-based framework, which includes the *carry-preserving* technique and new key guessing strategy, see also Section V. Within the generalized framework, by employing the *syncopation* and *carry-preserving* techniques, we have achieved a series of significant analytical results on the ChaCha cipher. Specifically, for 256-bit ChaCha, the extended paper provides improved attacks on ChaCha7 and ChaCha7.25, the first 7.5-round attack in a single-key scenario, and the first 8-round attack in a related-key scenario, see also Sections VII-A, VII-B, VII-C and VII-D, respectively. Additionally, the paper provides the security analysis of 128-bit ChaCha in Section VIII. As a result, our techniques permit us to defeat 7 rounds without the last rotation for 128-bit ChaCha, see also Section VIII-A. The new attack results are indicated with an asterisk (*) in Table I. Furthermore, the extended paper includes the proofs of Lemma 1, Lemma 2, Lemma 5 and Theorem 1 which proof the remaining six (out of eight) cases lacked in the previous theoretical analysis [DDSM22]. Besides, the extended paper also includes the details of practical key-recovery attacks on ChaCha with a 64-bit secret key for demonstrating and verifying our techniques and attacks in Section IX.

3) *Organization of the paper:* In Sect. II, we recall the specification of ChaCha, and list the notations used throughout this paper. In Sect. III, we review the differential attacks based on PNB method and recent advances in the cryptanalysis of ChaCha. Then in Sect. IV, we introduce the *syncopation* technique for the PNB-based approximation in backward direction and refine the PNB-based differential attack taking the technique into account. After that, the generalized framework based on PNE is presented in Sect. V. Next in Sect. VI, the analysis of the differential part in forward direction is provided. As applications of our techniques, we present the attacks on 256-bit ChaCha and 128-bit ChaCha in Sects. VII and VIII respectively. The experiments of practical key-recovery attacks is given in Sect. IX. Finally, Sect. X concludes this paper.

II. PRELIMINARIES

In this section, we first take a look at the stream cipher ChaCha, then summarize the notations used throughout this paper.

In 2008, the stream cipher ChaCha [Ber08a] was introduced by Bernstein as a variant of Salsa20, aiming to bring better diffusion for similar performance. To start with, we give a brief description of ChaCha. The cipher operates on 32-bit words, takes as input a 256-bit secret key $k = (k_0, k_1, \dots, k_7)$, a 96-bit nonce $v = (v_0, v_1, v_2)$ and a 32-bit block counter t_0 , and produces a sequence of 512-bit keystream blocks. In the subsequent content, we will refer to the nonce and block counter words together as IV words. The operations consist of bit-wise XOR ($\oplus$), left rotation ($\lll$) and addition modulo 2^{32} ($\boxplus$). The state of ChaCha consists of sixteen 32-bit words, which can be represented as a 4×4 matrix

$$\mathbf{X} = \begin{pmatrix} x_0 & x_1 & x_2 & x_3 \\ x_4 & x_5 & x_6 & x_7 \\ x_8 & x_9 & x_{10} & x_{11} \\ x_{12} & x_{13} & x_{14} & x_{15} \end{pmatrix} = \begin{pmatrix} c_0 & c_1 & c_2 & c_3 \\ k_0 & k_1 & k_2 & k_3 \\ k_4 & k_5 & k_6 & k_7 \\ t_0 & v_0 & v_1 & v_2 \end{pmatrix}. \quad (1)$$

As for the initial state, it is built by placing the four predefined constants (c_0, c_1, c_2, c_3) , secret key $k = (k_0, k_1, \dots, k_7)$, block counter t_0 and nonce (v_0, v_1, v_2) as in the above matrix, where the four constants for the version with a 256-bit key are $c_0 = 0x61707865$, $c_1 = 0x3320646e$, $c_2 = 0x79622d32$, $c_3 = 0x6b206574$.

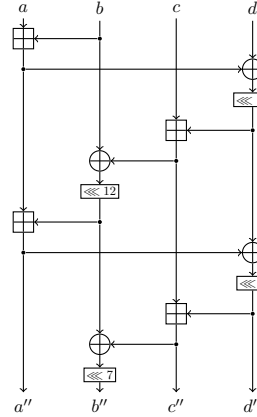


Fig. 1: The function *quarterround* of ChaCha.

The round function *Round* consists of 4 parallel applications of the function *quarterround*. The function *quarterround* uses 4 modular additions and 4 XOR's and 4 rotations to update four 32-bit state words. Specifically, the function *quarterround* transforms a 4-word vector (a, b, c, d) into (a'', b'', c'', d'') via an intermediate vector (a', b', c', d') :

$$\begin{aligned} a' &= a \boxplus b; & d' &= (d \oplus a') \lll 16; \\ c' &= c \boxplus d'; & b' &= (b \oplus c') \lll 12; \\ a'' &= a' \boxplus b'; & d'' &= (d' \oplus a'') \lll 8; \\ c'' &= c' \boxplus d''; & b'' &= (b' \oplus c'') \lll 7. \end{aligned}$$

As depicted in Fig. 1, this transformation is invertible and updates each word twice, so the round function of ChaCha is also invertible. Let $X^{(r)}$ denote the state after r rounds, i.e., $X^{(r)} = \text{Round}^r(X^{(0)})$. The inverse of round function is denoted as Round^{-1} , then $X^{(0)} = \text{Round}^{-r}(X^{(r)})$. In the odd rounds, called column rounds, the function *quarterround* is applied to the four columns (x_0, x_4, x_8, x_{12}) , (x_1, x_5, x_9, x_{13}) , $(x_2, x_6, x_{10}, x_{14})$ and $(x_3, x_7, x_{11}, x_{15})$. In the even rounds, called diagonal rounds, the *quarterround* function is applied to the four diagonals $(x_0, x_5, x_{10}, x_{15})$, $(x_1, x_6, x_{11}, x_{12})$, (x_2, x_7, x_8, x_{13}) and (x_3, x_4, x_9, x_{14}) . The keystream block Z of ChaChaR is obtained as the word-wise modular addition of the initial state and the state after R rounds, which means $Z = X^{(0)} \boxplus X^{(R)}$ and $X^{(R)} = \text{Round}^R(X^{(0)})$.

The notations used in this paper are listed in Table II.

TABLE II: Notations.

Notation	Description
x	An n -bit word, i.e., an n -bit vector $x = (x[n-1], x[n-2], \dots, x[0]) \in \mathbb{F}_2^n$.
$X^{(r)}$	State matrix after applications of r round functions.
x_i or $X[i]$	The i -th word of state matrix X .
$x_i[j_2 : j_1]$	The consecutive $(j_2 - j_1 + 1)$ -bit vector of the i -th word of state matrix X , starting from $x_i[j_1]$ ending to $x_i[j_2]$, $j_2 \geq j_1$.
$x \boxplus y$	Addition of x and y modulo 2^n .
$x \boxminus y$	Subtraction of x and y modulo 2^n .
$x \oplus y$	Bit-wise XOR of x and y .
$x \lll l$	Rotation of x by l bits to the left.
Δx	XOR difference of x and x' , i.e. $\Delta x = x \oplus x'$.

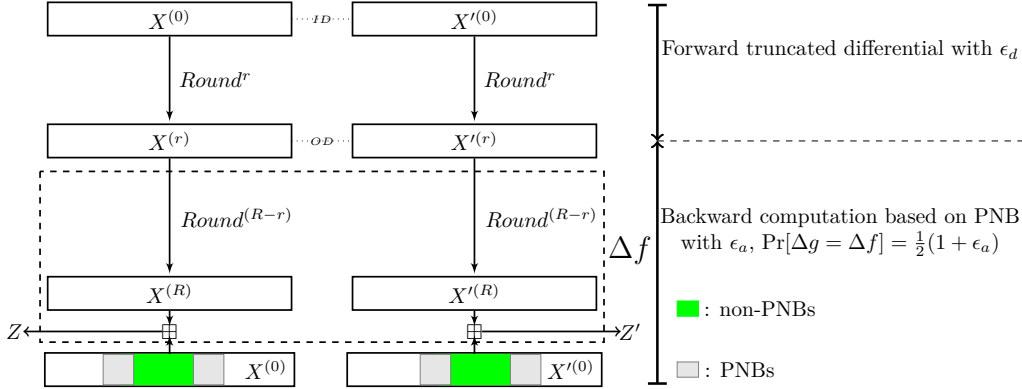


Fig. 2: Framework of PNB-based differential attack.

III. REVIEWING DIFFERENTIAL CRYPTANALYSIS OF CHACHA

In this section, we first review the differential attack proposed by Aumasson et al. [AFK⁺08], which is based on the technique called *probabilistic neutral bits* (shortcut PNBs). Then, we summarize the recent advances on cryptanalysis of ChaCha.

A. Differential Attack Based on PNB Method

In general, the differential attacks based on PNB method in [AFK⁺08] use a meet-in-the-middle idea. Usually, one selects a truncated differential with some (single-bit) input/output differences after certain rounds for forward direction. Then one identifies the set of PNBs with less influence over the output difference from backward direction. As for the significant key bits, we guess and recover them in the key recovery phase of the attacks. The framework of differential attack based on PNB method is illustrated in Fig. 2, where we use the shortcuts ID and OD for input and output difference.

Assume that we use a truncated differential with a single-bit input difference at $X^{(0)}[i][j]$ of IV and a single-bit output difference at $X^{(r)}[p][q]$ of middle state after r rounds, denoted by $(\Delta X^{(r)}[p][q], \Delta X^{(0)}[i][j])$, which is actually a differential-linear (DL) approximation, and its correlation ϵ_d is defined by

$$\Pr_{v,t} \left\{ \Delta X^{(r)}[p][q] = 0 \mid \Delta X^{(0)} = e_{ij} \right\} = \frac{1}{2} (1 + \epsilon_d), \quad (2)$$

where e_{ij} is a state whose j -th bit of i -th word is one and other bits are zero. Note that regarding key as a random variable, the median of correlation over all keys is used in the following analysis.

The corresponding output Z is observed for a nonce v , counter t and the given secret key k . Having k , v and t , one can invert the operations to access internal state from backward direction. Given the above forward differential, let

$$\Delta f(k, v, t, Z, Z') \triangleq (Round^{-(R-r)}(Z \boxminus X^{(0)}) \oplus Round^{-(R-r)}(Z' \boxminus X'^{(0)}))[p][q], \quad (3)$$

then $\Delta X^{(r)}[p][q] = \Delta f(k, v, t, Z, Z')$. To obtain a more efficient attack than exhaustive search over all possibilities of secret key, we need to find an approximation Δg of Δf which effectively depends on $m = 256 - n$ key bits. More formally, let k^1 correspond to the m -bit subkey of secret key $k = (k^1, k^0)$ and Δf be correlated to Δg with correlation ϵ_a ,

$$\Pr_{v,t} \left\{ \Delta f(k, v, t, Z, Z') = \Delta g(k^1, v, t, Z, Z') \right\} = \frac{1}{2} (1 + \epsilon_a). \quad (4)$$

Denote the correlation of Δg by ϵ , i.e. $\Pr_{v,t} \left\{ \Delta g(k^1, v, t, Z, Z') = 0 \right\} = \frac{1}{2}(1 + \epsilon)$. Under a reasonable independence assumption, the equality $\epsilon = \epsilon_d \cdot \epsilon_a$ holds.

a) *Probabilistic neutral bits*: Generally speaking, PNBs are these non-significant key bits with less influence (of size n), and on the opposite side non-PNBs are these significant key bits with great influence (of size m). To identify the set of PNBs (or non-PNBs), Aumasson et al. [AFK⁺08] gave a formal definition of a suitable measure for the amount of influence which every key bit has on output of Δf .

Definition 1 (Neutrality measure [AFK⁺08]). *The neutrality measure of $k[i]$ with respect to the function $\Delta f(k, W)$ is defined as γ_i , where $W = (v, t, Z, Z')$ and $\Pr \{ \Delta f(k, W) = \Delta f(k \oplus e_i, W) \} = \frac{1}{2}(1 + \gamma_i)$, i.e. the probability (over all k and W) that complementing the key bit $k[i]$ does not change the output of $\Delta f(k, W)$, where e_i is the unit vector of which i -th bit is one and other bits are all zero.*

In practice, a threshold γ is set, and $PNB = \{i \mid |\gamma_i| > \gamma\}$ and $Non-PNB = \{i \mid |\gamma_i| \leq \gamma\}$. Assuming that k^1 denotes the significant key bits, i.e. non-PNBs, define $\Delta g(k^1, W)$ as $\Delta f(k, W)$ by setting all PNBs into a fixed value (e.g. all zeros). Then the correlation ϵ_a is measured experimentally by using enough random IVs under many randomly chosen keys.

The Neyman-Pearson decision theory provides a solution to estimate the number of samples N required to get the bounds on probabilities of false alarm p_{fa} and non-detection p_{nd} . It can be shown that $N \approx \left(\frac{\sqrt{\alpha \log 4 + 3} \sqrt{1 - (\epsilon_a \epsilon_d)^2}}{\epsilon_a \epsilon_d} \right)^2$ samples suffices to achieve that $p_{nd} = 1.3 \times 10^{-3}$ and $p_{fa} = 2^{-\alpha}$. With using the median correlation ϵ^* in the above equation, we have a success probability of at least $\frac{1}{2}(1 - p_{nd}) \approx \frac{1}{2}$ for the attack.

Subsequently, a two-step key-recovery procedure can be launched, where the non-PNBs are first recovered according to the given single-bit differential and then PNBs are found by exhaustive search. The time complexity of this key-recovery attack is calculated as $T = 2^m N + (2^m p_{fa} + (1 - p_{nd}))2^n \approx 2^m N + 2^{256-\alpha} + 2^n$, where $m + n = 256$. For details about this procedure, please refer to [AFK⁺08]. As applications of the PNB-based differential cryptanalysis, Aumasson et al. [AFK⁺08] presented the analysis of Salsa, ChaCha and Rumba, especially including the first attacks on 6 and 7 rounds of ChaCha.

B. Recent Advances in Cryptanalysis of ChaCha

In the following, we recall recent advances in cryptanalysis of ChaCha in [BLT20], [DGSS22], [BGG⁺23] and [XXTQ24].

At CRYPTO 2020, Beierle et al. [BLT20] presented the first 3.5-round differential-linear (DL) approximation for ChaCha, leading to further improvements on attacks on ChaCha6 and ChaCha7. More precisely, the DL distinguisher is divided into two parts, the differential characteristic in E_1 with one round and the middle DL approximation in E_m with the following 2.5 rounds. Since four *quarterround* functions are independent in the first round, the differential characteristics of E_1 can be represented over one *quarterround* function as $\Delta_{in} = ([, [, [, [6]) \rightarrow \Delta_m = ([2, [5, 9, 17, 29], [10, 22, 30], [10, 30])$. Then with Δ_m as input difference of E_m , they found the following DL approximation with probability $\Pr[(\Delta X_j^{(1)}, \Delta X_{j+4}^{(1)}, \Delta X_{j+8}^{(1)}, \Delta X_{j+12}^{(1)}) = \Delta_m \rightarrow \Delta X_{(j+1) \bmod 4}^{(3.5)}[0]] = \frac{1}{2}(1 + \epsilon_d)$, for $j \in \{0, 1, 2, 3\}$ where the correlation is evaluated experimentally as $\epsilon_d = 2^{-8.3}$. The experimental distinguisher of E_m is combined with the differential characteristic of E_1 , and the resulting distinguisher covers 3.5 rounds with single-bit input difference and output linear mask, which is summarized in Table III for the case $j = 1$. As evaluated in [BLT20], the probability that pairs with the input difference Δ_{in} satisfy the given differential in the first round is $p = 2^{-5}$ on average for about 70% of the keys. Therefore, to find right pair in the first round, on average we need to repeat the procedure of attacks for $p^{-1} = 2^5$ times with varying random IVs in the input difference column (ID column) where input difference is imposed.

TABLE III: The 3.5-round DL approximation in [BLT20].

ID	OD	$ \epsilon_d $
$\Delta X_{13}^{(0)}[6]$ satisfying $\Delta_{in} \rightarrow \Delta_m$	$\Delta X_2^{(3.5)}[0]$	$2^{-8.3}$

To avoid the cost of iterations for getting a right pair, Dey et al. [DGSS22] proposed another alternative way to find the right pairs with the help of a pre-computed list. Actually, one should pre-compute all the needed right pairs offline and keep them in a list. To achieve this goal, they introduced the concept of *exploitable key*, which was restated as the following in the case of ChaCha.

Definition 2 (Exploitable key [DGSS22]). *Let $k_{ID} \in \mathbb{F}_2^{64}$ and a subspace $\mathbb{S}_{K_{nmem}} = \{(a_{63}, a_{62}, \dots, a_0) \mid a_i = 0 \text{ for } i \notin K_{nmem}\} \subseteq \mathbb{F}_2^{64}$, if there exists at least one IV $v \in \mathbb{F}_2^{32}$ such that for any key $k'_{ID} \in k_{ID} \oplus \mathbb{S}_{K_{nmem}}$, $((k'_{ID}, v), (k'_{ID}, v'))$ forms a right pair, k_{ID} is an exploitable key with respect to the subspace $\mathbb{S}_{K_{nmem}}$, and the corresponding v is a favorable IV for k_{ID} .*

At FSE 2024, Bellini et al. [BGG⁺23] present a five-round DL distinguisher of ChaCha which is made up of one-round differential and four-round DL approximation. The one-round differential in the first round can be represented over one *quarterround* function $\Delta'_{in} = ([, [, [, [9, 29]) \rightarrow \Delta'_m = ([5, 25], [12, 28], [21, 25], [13, 21])$ with probability 2^{-7} , where Δ'_{in} is an input difference with two active input bits and Δ'_m is an output difference with eight output bits. Then with Δ'_m as input

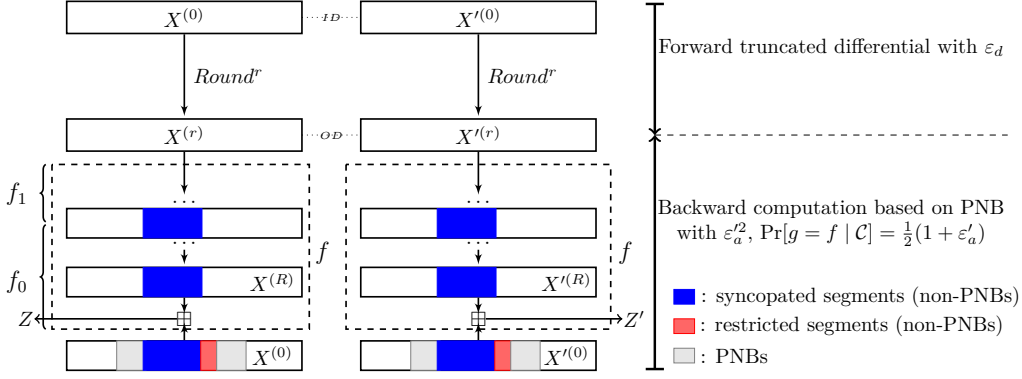


Fig. 3: Refined framework of PNB-based differential attack.

difference, they found a two-round DL approximation with correlation $2^{-30.15}$ followed by a two-round linear approximation with correlation 2^{-2} and obtained a four-round DL approximation with correlation $\varepsilon_d = 2^{-34.15} = 2^{-30.15} \cdot (2^{-2})^2$ by Piling-Up Lemma, that is $\Pr[(\Delta X_3^{(1)}, \Delta X_7^{(1)}, \Delta X_{11}^{(1)}, \Delta X_{15}^{(1)}) = \Delta'_m \rightarrow \Delta(X_2^{(5)}[0] \oplus X_6^{(5)}[7] \oplus X_6^{(5)}[19] \oplus X_{10}^{(5)}[12] \oplus X_{14}^{(5)}[0])] = \frac{1}{2}(1 + \varepsilon_d)$. Very recently, Xu et al. [XXTQ24] presented a more accurate correlation for the four-round differential-linear distinguisher by taking differential-linear hull into account, whose correlation is re-estimated as $2^{-32.2}$. The four-round DL approximation is combined with the one-round differential, resulting in a five-round DL distinguisher with two-bit input difference, which is summarized in Table IV.

TABLE IV: The 5-round DL approximation in [BGG⁺23].

ID	OD	$ \varepsilon_d $
$\Delta X_{15}^{(0)}[9, 29]$ satisfying $\Delta'_{in} \rightarrow \Delta'_m$	$\Delta(X_2^{(5)}[0] \oplus X_6^{(5)}[7] \oplus X_6^{(5)}[19] \oplus X_{10}^{(5)}[12] \oplus X_{14}^{(5)}[0])$	$2^{-32.2}$

IV. THE PNB-BASED ATTACK WITH SYNCOPATION

In this paper, we describe a refined framework of differential attack based on the PNB method for ARX ciphers. In this section, we focus on the PNB-based approximation in the backward direction. The analysis of differential part in the forward direction will be discussed in the next section.

In the following, we will discuss the calculation of approximation correlation and searching for PNBs using properties of the ARX structure in the PNB-based approximation, along with the attack framework.

First, we show a simple but useful observation about the correlation of backward approximation, which significantly cuts down the workload of estimating the backward correlation. More precisely, we observe that the backward approximation of the differential described in (4) is actually the sum of the same approximation with two different inputs in initialization, and thus its correlation is the square of the correlation of the latter, which is shown in Proposition 1. This fact is simple and was not illuminated in the previous analysis, but it is surprising that it is powerful in the calculation of backward correlation.

After that, a technique called *syncopation* is introduced for the PNB-based approximation of ARX ciphers, to amplify the correlation of the approximation. To simplify the analysis, the approximated part of an ARX cipher, denoted by f , is divided into two parts f_0 and f_1 , that is, $f = f_1 \circ f_0$. The PNB-based approximating function g is a function of f with assigning PNBs to be a fixed value, and the same relationship holds between g_0 and f_0 . The *syncopation* technique is applied to the part g_0 (to say f_0) for reducing its dependence on PNBs so that the correlation between g_0 and f_0 is high. The syncopations split the input of f_0 into different segments such that each segment of output of f_0 only depends on the same segment of the input under some restrictions. A syncopation consists of a few non-PNBs following by PNBs, and a split segment consists of several non-PNBs. In the other words, the output of f_0 excluding the syncopations does not depend on PNBs, and therefore the approximation correlation between g_0 and f_0 is high. In this section, we analyze the properties of syncopation in modular subtraction and modular addition, and present a tool for determining syncopations, which leads to a new and efficient method for finding a good set of PNBs with a large correlation.

The attack framework is depicted in Fig. 3.

In the backward computations with PNBs, we observe that computing the backward correlation of output difference at middle states can be simplified into the computation of the correlation of the middle state in one initialization. This treatment is always used in the linear part when one constructs a differential-linear approximation. More precisely, let

$$f(k, v, t, Z) \triangleq \text{Round}^{-(R-r)}(Z \boxminus X^{(0)})[p][q], \quad (5)$$

and then by (3) we have $\Delta f(k, v, t, Z, Z') = f(k, v, t, Z) \oplus f(k, v', t', Z')$. Let g be an approximation of f which effectively depends on $m = 256 - n$ key bits, more formally, let k^1 correspond to the m -bit subkey of secret key $k = (k^1, k^0)$. And in general, g is constructed by assigning a fixed value $\hat{k}^0$ to k^0 , i.e.,

$$g(k^1, v, t, Z) \triangleq f(k, v, t, Z) \big|_{k^0=\hat{k}^0}. \quad (6)$$

Then we can use g to construct an approximation for the function Δf , i.e. $g(k^1, v, t, Z) \oplus g(k^1, v', t', Z')$, which is always the same to the one constructed directly by Δg . Under the assumption of independence, the square relationship is established between the correlation of this treatment and the traditional one, which is summarized in the following.

Proposition 1. *Let f and g be the functions as defined in (5) and (6), where Round is the round function, $X^{(0)}$ is the state as defined in (1) and k^1 corresponds to the m -bit subkey of secret key k . Let ϵ_a be the correlation of f and g ,*

$$\Pr_{v,t}\{f(k, v, t, Z) = g(k^1, v, t, Z)\} = \frac{1}{2}(1 + \epsilon_a), \quad (7)$$

and ε_a the correlation of backward approximation in the differential using g ,

$$\Pr_{v,t}\{f(k, v, t, Z) \oplus f(k, v', t', Z') = g(k^1, v, t, Z) \oplus g(k^1, v', t', Z')\} = \frac{1}{2}(1 + \varepsilon_a).$$

If Z and Z' are statistically independent, then it holds that $\varepsilon_a = \epsilon_a^2$.

Proof. If Z and Z' are statistically independent, then $Z \boxplus X^{(0)}$ and $Z' \boxplus X'^{(0)}$ are statistically independent. Therefore, by the definitions of f and g , $f(k, v, t, Z) \oplus g(k^1, v, t, Z)$ and $f(k, v', t', Z') \oplus g(k^1, v', t', Z')$ are statistically independent, and both equal to zero with probability of $\frac{1}{2}(1 + \epsilon_a)$. By Piling-up Lemma, $f(k, v, t, Z) \oplus g(k^1, v, t, Z) \oplus (f(k, v', t', Z') \oplus g(k^1, v', t', Z')) = 0$ holds with probability of $\frac{1}{2}(1 + \epsilon_a^2)$. Thus, we have $\varepsilon_a = \epsilon_a^2$. $\square$

By Proposition 1, we can detect the correlation ϵ_a instead of ε_a in practice. It takes about the square root of the amount of computations to detect the correlation ϵ_a in experiments compared with detecting ε_a by using the conventional method. In the following, we usually refer to ϵ_a as the correlation of backward single approximation $f = g$ in one initialization, and correspondingly, to ε_a as the correlation of backward double approximation $\Delta f = \Delta g$ in the differential.

A. Syncopation Technique

In the PNB-based approximation, the essential problem is how to find numerous PNBs with a large correlation. Unfortunately, more PNBs and larger correlation are contradictory in most cases. In the literatures, as far as we know, the methods for handling this problem are limited to the naive threshold rule [AFK⁺08] and greedy-like method recently shown in [DS17], [DGSS22], though the seminal attack was proposed fifteen years ago. Both methods treat the cipher as a black box, and yet the ARX structure of the cipher has been unexploited. In the existing similar work, this structure has been utilized by the partitioning technique [BC14], [Leu16], [BLT20] in linear cryptanalysis and differential-linear cryptanalysis. The idea of partitioning technique is to divide the input or output space into different subsets each of which corresponds to a good differential or linear characteristic. The PNB-based approximation is much more complicated than the cases of differential and linear characteristic, and the technique is not applicable to the PNB-based attack. Even in linear and differential-linear cryptanalysis, some ARX designs might be too complicated to use the partitioning technique, as stated by Beierle et al. in [BLT20], “*unfortunately, 7-round ChaCha is too complicated to apply our (partitioning) technique for the linear part*”. However, we find that some similar properties of the ARX structure utilized by the partitioning technique can be used in the PNB-based approximation though with a different way.

Inspired by the partitioning technique, we introduce the *syncopation* technique for the PNB-based attack, which exploits the properties of ARX structure. As mentioned above, in linear or differential-linear attacks, to increase the correlation of linear approximation, one can use the partitioning technique to choose corresponding linear masks for different subsets of ciphertexts. In the PNB-based attack, a much more complicated function is used to approximate the targeted function. For simplifying the analysis, we divide the approximating function g into two parts g_0 and g_1 , that is, $g = g_1 \circ g_0$. The *syncopation* technique is applied to the first part g_0 for reducing its dependence on PNBs so that the correlation of the approximation g_0 is high with the corresponding part f_0 .

First the notations and definitions are provided for description of *syncopation* technique. Let $Y = H(X)$ be an ARX function. For $X = (X^o, X^\#)$ and $Y = (Y^o, Y^\#)$, if Y^o only depends on X^o under the condition that a system $Q^{\mathcal{R}}(X^o)$ of equations on X^o is satisfied, then $X^\#$ and $Y^\#$ are called the *syncopation bits*, X^o and Y^o the *syncopated bits*, and the bits of X^o appearing in $Q^{\mathcal{R}}$ the *restricted bits*, denoted by $X^{\mathcal{R}}$. The consecutive bits in each word operated by the ARX function is called a *segment*. For a state S , each segment of its syncopation bits $S^\#$ that are consecutive in the state S is called a *syncopation*, and each segment of its syncopated bits S^o that are consecutive in the state S is called a *syncopated segment*. The pattern of syncopations and syncopated segments is called the *syncopation property*. A simple example of the syncopation property is given in Fig. 4, and the terms and notations are illustrated in Fig. 4b.

Let the secret key k consist of two parts, the PNBs k^0 and non-PNBs k^1 , as defined previously. For the initial state, we set the PNBs to syncopation bits and the other bits to syncopated bits. Let u be a positive integer. We denote by $k^{\mathcal{R}}$ the set of the least u significant bits of a non-PNB segment that are adjacent to PNBs in a word. We restrict conditions on non-PNBs $k^{\mathcal{R}}$ so that the syncopated segment of the output of a PNB-based approximating function does not depend on PNBs, and therefore the approximation correlation will be amplified.

To analyze the propagation of syncopation property for ARX ciphers, like differential and linear characteristic, we provide the basic properties of modular addition and subtraction which are the nonlinear operations in ARX designs.

a) *Property of Modular Addition:* Let us consider the operation of modular addition $x = s \boxplus k$ where k is a word of the secret key, and s is a public word, as depicted in Fig. 4. Assuming that $k[t+w-1:t]$ is a non-PNB segment and known in backward computation, we wonder under what conditions $x[t+w-1:t]$ is independent of the PNBs and can be determinedly computed from s and non-PNBs of k . To this end, Lemma 1 is derived.

Lemma 1. *Let $x = s \boxplus k$ and $x, k, s \in \mathbb{F}_2^n$. If $k[t-1:t-u] \oplus s[t-1:t-u] \neq \vec{1}$ for $1 \leq u \leq t$, then we have*

$$\begin{aligned} x[t+w-1:t] &= s[t+w-1:t] + k[t+w-1:t] + \text{Carry}[t] - \text{Carry}[t+w] \cdot 2^w, \\ \text{Carry}[t+w] &= \begin{cases} 0 & \text{if } s[t+w-1:t] + k[t+w-1:t] + \text{Carry}[t] < 2^w, \\ 1 & \text{if } s[t+w-1:t] + k[t+w-1:t] + \text{Carry}[t] \geq 2^w, \end{cases} \\ \text{Carry}[t] &= s[t-\iota] \quad \text{if } k[t-1:t-\iota+1] \oplus s[t-1:t-\iota+1] = \vec{1} \text{ and } k[t-\iota] \oplus s[t-\iota] = 0 \end{aligned}$$

where the width of segment $w \geq 0$, $1 \leq \iota \leq u$ and $\text{Carry}[i]$ denotes the i -th bit of carry vector, i.e. $s \boxplus k = s \oplus k \oplus \text{Carry}$.

Proof. By considering modular addition $x = s \boxplus k$, the following holds

$$\begin{aligned} x[t+w-1:t] &= (s[t+w-1:t] + k[t+w-1:t] + \text{Carry}[t]) \bmod 2^w, \\ \text{Carry}[t+w] &= \lfloor (s[t+w-1:t] + k[t+w-1:t] + \text{Carry}[t]) / 2^w \rfloor, \end{aligned}$$

where $\text{Carry}[i]$ is i -th bit of carry vector, i.e. $\text{Carry}[i] \triangleq (s \boxplus k)[i] \oplus s[i] \oplus k[i]$. That is

$$x[t+w-1:t] = s[t+w-1:t] + k[t+w-1:t] + \text{Carry}[t] - \text{Carry}[t+w] \cdot 2^w,$$

and

$$\text{Carry}[t+w] = \begin{cases} 0 & \text{if } s[t+w-1:t] + k[t+w-1:t] + \text{Carry}[t] < 2^w, \\ 1 & \text{if } s[t+w-1:t] + k[t+w-1:t] + \text{Carry}[t] \geq 2^w. \end{cases}$$

With representing $x = s \boxplus k$ in a bit level, we have

$$x[t] = s[t] \oplus k[t] \oplus \text{Carry}[t],$$

where $\text{Carry}[t] = \text{MAJ}(\text{Carry}[t-1], s[t-1], k[t-1])$, and the majority function $\text{MAJ}(a, b, c) = ab \oplus ac \oplus bc$.

If $k[t-1] = s[t-1]$, then we have $\text{Carry}[t] = s[t-1]$. As for the case $k[t-1] \neq s[t-1]$, we have $\text{Carry}[t] = \text{Carry}[t-1]$. Moreover, if $k[t-2] = s[t-2]$, then we have $\text{Carry}[t] = \text{Carry}[t-1] = s[t-2]$. By analogy, we have that $\text{Carry}[t] = s[t-\iota]$ where ι is such that $k[t-1:t-\iota+1] \oplus s[t-1:t-\iota+1] = \vec{1}$ and $k[t-\iota] \oplus s[t-\iota] = 0$. $\square$

Note that the parameter u in Lemma 1 is the number of bits of restricted segments, and ι is an auxiliary parameter to identify the minimum of index such that the premise of lemma holds. For $w = 0$, only $\text{Carry}[t]$ is computed in the above lemma. In Fig. 4a, according to Lemma 1, the syncopated segment $x[26:16]$ is independent of the syncopation bits, specially including $k[14:6]$, under the condition $k[15] = s[15]$, where $t = 16$, $w = 11$ and $u = 1$.

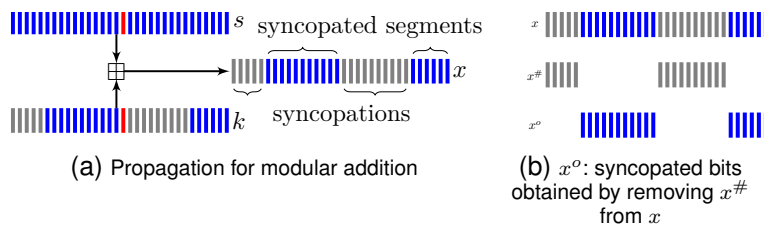


Fig. 4: An example for syncopation property, where the blue lines represent for syncopated bits, red lines for restricted bits and gray lines for syncopation bits.

b) *Property of Modular Subtraction:* Let us consider the operation $F_k : \mathbb{F}_2^n \rightarrow \mathbb{F}_2^n, x \mapsto s = x \boxplus k$ where k is a word of the secret key and s is a public word. For instance, F_k serves as the last operation of branches b and c of ChaCha. Assuming that $k[t+w-1:t]$ is a non-PNB segment and known, we are interested in the case that $x[t+w-1:t]$ can be determinedly computed from s and non-PNBs of k . Similarly, the property of modular subtraction is derived as Lemma 2.

Lemma 2. Let $x = s \boxplus k$ and $x, k, s \in \mathbb{F}_2^n$. If $k[t-1:t-u] \oplus s[t-1:t-u] \neq \vec{0}$ for $1 \leq u \leq t$, then we have

$$\begin{aligned} x[t+w-1:t] &= \text{Carry}[t+w] \cdot 2^w + s[t+w-1:t] - k[t+w-1:t] - \text{Carry}[t], \\ \text{Carry}[t+w] &= \begin{cases} 0 & \text{if } s[t+w-1:t] > k[t+w-1:t], \\ \text{Carry}[t] & \text{if } s[t+w-1:t] = k[t+w-1:t], \\ 1 & \text{if } s[t+w-1:t] < k[t+w-1:t], \end{cases} \\ \text{Carry}[t] &= s[t-\iota] \oplus 1 \quad \text{if } k[t-1:t-\iota+1] \oplus s[t-1:t-\iota+1] = \vec{0} \text{ and } k[t-\iota] \oplus s[t-\iota] = 1 \end{aligned}$$

where the width of segment $w \geq 0$, $1 \leq \iota \leq u$, and $\text{Carry}[i]$ denotes the i -th bit of carry vector, i.e. $x \boxplus k = x \oplus k \oplus \text{Carry}$.

Proof. By considering modular addition $s = x \boxplus k$, the following holds

$$\begin{aligned} s[t+w-1:t] &= (x[t+w-1:t] + k[t+w-1:t] + \text{Carry}[t]) \bmod 2^w, \\ \text{Carry}[t+w] &= \lfloor (x[t+w-1:t] + k[t+w-1:t] + \text{Carry}[t]) / 2^w \rfloor \end{aligned}$$

where $\text{Carry}[i]$ is i -th bit of carry vector, i.e. $\text{Carry}[i] \triangleq (x \boxplus k)[i] \oplus x[i] \oplus k[i]$. That is

$$x[t+w-1:t] + k[t+w-1:t] + \text{Carry}[t] = \text{Carry}[t+w] \cdot 2^w + s[t+w-1:t],$$

then we have

$$x[t+w-1:t] = \text{Carry}[t+w] \cdot 2^w + s[t+w-1:t] - k[t+w-1:t] - \text{Carry}[t].$$

Since $0 \leq x[t+w-1:t] < 2^w$, by contradiction, we can easily conclude that

$$\text{Carry}[t+w] = \begin{cases} 0 & \text{if } s[t+w-1:t] > k[t+w-1:t], \\ \text{Carry}[t] & \text{if } s[t+w-1:t] = k[t+w-1:t], \\ 1 & \text{if } s[t+w-1:t] < k[t+w-1:t]. \end{cases}$$

With representing $s = x \boxplus k$ in a bit level, we have

$$s[t] = x[t] \oplus k[t] \oplus \text{Carry}[t]$$

where $\text{Carry}[t] = \text{MAJ}(\text{Carry}[t-1], x[t-1], k[t-1])$, and the majority function $\text{MAJ}(a, b, c) = ab \oplus ac \oplus bc$.

Since $s[t-1] = x[t-1] \oplus k[t-1] \oplus \text{Carry}[t-1]$, if $k[t-1] \neq s[t-1]$, then we have $x[t-1] \neq \text{Carry}[t-1]$, and $\text{Carry}[t] = k[t-1] = s[t-1] \oplus 1$. As for the case $k[t-1] = s[t-1]$, we have $x[t-1] = \text{Carry}[t-1]$, and $\text{Carry}[t] = \text{Carry}[t-1]$. Moreover, if $k[t-2] \neq s[t-2]$, then we have $x[t-2] \neq \text{Carry}[t-2]$, and $\text{Carry}[t] = \text{Carry}[t-1] = k[t-2] = s[t-2] \oplus 1$. By analogy, we have $\text{Carry}[t] = s[t-\iota] \oplus 1$ where ι is such that $k[t-1:t-\iota+1] \oplus s[t-1:t-\iota+1] = \vec{0}$ and $k[t-\iota] \oplus s[t-\iota] = 1$. $\square$

According to Lemma 1 and 2, given the segments $k[t+w-1:t-u]$ and $s[t+w-1:t-u]$, the segment $x[t+w-1:t]$ is known under the condition that the equation $Q^{\mathcal{R}}(k[t-1:t-u], s[t-1:t-u])$ is satisfied, where $Q^{\mathcal{R}}(k[t-1:t-u], s[t-1:t-u])$ denotes $k[t-1:t-u] \oplus s[t-1:t-u] \neq \vec{1}$ and $k[t-1:t-u] \oplus s[t-1:t-u] \neq \vec{0}$ for modular addition and subtraction respectively. Based on the properties of modular addition and subtraction, we summarize the propagation of syncopation property for modular addition and subtraction in the following.

c) *Propagation for Modular Addition and Subtraction:* For the operation of modular addition or subtraction on k and s , if all the bits of s are syncopated bits, then, compared with k , the syncopation property of the output is unchanged except that it brings the conditions on restricted bits and converts the restricted bits to syncopation bits. As depicted in Fig. 4a, the sum of top augend and the bottom addend has a similar syncopation property with the syncopation property of the bottom addend, where all the bits of top augend are syncopated bits. Here a condition on restricted bits $k[15] = s[15]$ is introduced by modular addition and $x[15]$ is converted to a syncopation bit. This property is useful in our attacks.

Usually, a syncopated segment is obtained by imposing conditions $Q^{\mathcal{R}}$ on restricted bits, which is called as profitable syncopated segment. Hereinafter, assuming that there is a set of profitable syncopated segments of internal state with carries, denoted by $\mathcal{X} = \{(x[t+w-1:t], \text{Carry}[t+w])\}$, and let $\mathcal{C}$ be the set of conditions related to $\mathcal{X}$, that is, $\mathcal{C} = \{Q^{\mathcal{R}}(k[t-1:t-u], s[t-1:t-u])\}$.

d) *Propagation for Rotation and XOR*: For the rotation operation, the propagation of syncopation property is equivalent in terms of rotation, that is, the bit-wise pattern of syncopation property after rotation is equivalent to rotation of the one before rotation. The propagation of syncopation property on the XOR operation has the following property: the set of the syncopations is the union of input syncopations and the set of the syncopated segments is the intersection of input syncopated segments.

By analysis of the propagation of syncopation property on basic operations, we can get a general property for ARX structures in PNB-based approximation.

Proposition 2. *Let $f(k, s)$ be a function consisting of ARX operations, where the secret key k consists of two parts, the PNBs k^0 and non-PNBs k^1 . Let g be a restriction of f with k^0 to be a fixed value. Let the PNBs k^0 be syncopation bits, the other bits of (k, s) be syncopated bits, and $k^{\mathcal{R}}$ be restricted bits. If $k^{\mathcal{R}}$ satisfies the syncopation property propagating from (k, s) to $f(k, s)$, that is, the restricted equations introduced by modular operations are satisfied, then any syncopated segment in the output of f is independent with k^0 and thus equals the corresponding segment in the output of g .*

Instead of finding a linear mask in the partitioning technique, the goal of syncopation technique is to find a PNB set consisting of as many as possible PNBs (to say syncopation bits), as well as maximize the overall length of syncopated segments of internal state in the ARX function for a fixed size of PNB set.

Algorithm 1 Determining syncopated segments given a set of PNBs

Input: A set of PNBs.

Output: The syncopated segments with conditions on u -bit restricted segments.

```

1: for  $0 \leq i \leq 7$  do
2:   for  $0 \leq j < 31 - u$  do
3:     if  $k_i[j]$  is a PNB and  $k_i[j + u : j + 1]$  are all non-PNBs then
4:        $k^{\mathcal{R}} \leftarrow k^{\mathcal{R}} \cup \{k_i[j + u : j + 1]\}$ 
5:     end if
6:   end for
7:    $k^o \leftarrow k^o \cup k_i^o$ , where  $k_i^o = \{k_i[j_2 : j_1] \mid k_i[j_2 : j_1] \text{ are all non-PNBs with } k_i[j_1 - 1] \text{ (if } j_1 - 1 \geq 0) \text{ and } k_i[j_2 + 1] \text{ (if } j_2 + 1 \leq 31) \text{ being both PNBs.}\}$ 
8: end for
9: Update  $k^o$  by excluding  $k^{\mathcal{R}}$  from  $k^o$ .
10: According to  $k^o$  and  $k^{\mathcal{R}}$ , find the profitable syncopated segments  $\mathcal{X}$  and conditions  $\mathcal{C}$  on restricted segments with analyzing the specific structure of cipher.
11: Return  $\mathcal{X}$  and  $\mathcal{C}$ .
```

The first question is how to determine the syncopated segments in the PNB-based attack. Next, we present an algorithm to determine the profitable syncopated segments of internal state which are obtained by imposing conditions on the restricted segments.

e) *A Tool for determining syncopated segments and restricted segments*: First we find all the possible positions of secret key on which the syncopation technique can be applied, including the syncopated segments and restricted segments of secret key. Then, according to the specific structure of the target cipher, the profitable syncopated segments of the internal state and corresponding conditions are determined by the secret key's syncopated segments and restricted segments. Algorithm 1 summarizes the procedure and notes that the cases of $u = 1$ or $u = 2$ are usually used in our attacks. Note that, for a state S , $S^{\mathcal{R}}$ is part of S^o by their definitions, but for convenience by k^o hereinafter, we mean the syncopated bits of the secret key k that exclude the restricted bits $k^{\mathcal{R}}$.

Example 1. *Here is an example of Algorithm 1. As shown in Fig. 4a, the bits $\{k[31 : 27], k[14 : 6]\}$ are PNBs and the bits $\{k[26 : 15], k[5 : 0]\}$ are non-PNBs. After running Step 2–9 of Algorithm 1 with $u = 1$, we get the syncopated segments $k^o = \{k[26 : 16], k[5 : 0]\}$ and restricted segments $k^{\mathcal{R}} = \{k[15]\}$. The structure in Fig. 4a illustrates Step 10 of Algorithm 1 for $x = s \boxplus k$ where s is public and known. According to the propagation for modular addition, the set of profitable syncopated segments of internal state is $\mathcal{X} = \{(x[26 : 16], \text{Carry}[27])\}$ and its condition is $\mathcal{C} = \{k[15] = s[15]\}$.*

B. Refined PNB-based Attack with Syncopation

First, we present a new and efficient method to find good set of PNBs with a greedy algorithm. Then, a refined procedure of key-recovery attack is summarized, along with the analysis of complexities.

1) *New efficient method to construct PNB set*: In our method, we experimentally estimate the backward correlation of $f = g$ as described in Proposition 1, i.e. the correlation ϵ_a in (7) instead of ϵ_a in (4). This trick allows us to save lots of computations as explained before and makes our method very efficient. With the tool of determining syncopated segments, we take the syncopation technique into account, which results in a new criteria of identifying PNBs. The resulting key bits

Algorithm 2 A greedy algorithm for searching a conditional PNB set

Input: The size n of CPNB set and $CPNB^{shortlist}$.

Output: The CPNB set, its correlation, syncopated segments and conditions.

- 1: Initialize a set $CPNB_0 = \emptyset$.
 - 2: **for** $i \in \{0, 1, \dots, n-1\}$ **do**
 - 3: **for** $j \in CPNB^{shortlist}$ and $j \notin CPNB_i$ **do**
 - 4: $CPNB_{temp} \leftarrow CPNB_i \cup \{j\}$. $\triangleright$ The bit positions of k^0 in the key k .
 - 5: Update $\mathcal{X}_{i,j}$ and $\mathcal{C}_{i,j}$ by performing Algorithm 1 with $CPNB_{temp}$ as input.
 - 6: Under the conditions $\mathcal{C}_{i,j}$, estimate backward correlation $\epsilon_a'^{(i,j)}$ in (7) with assigning k^0 by setting key bits of $CPNB_i$ to zero and flipping key bit j .
 - 7: **end for**
 - 8: Choose index j_i with maximal correlation, and $CPNB_{i+1} \leftarrow CPNB_i \cup \{j_i\}$.
 - 9: **end for**
 - 10: Determine $\mathcal{X}_n$ and $\mathcal{C}_n$ of $CPNB_n$ by Algorithm 1.
 - 11: Under the conditions $\mathcal{C}_n$, estimate backward correlation ϵ_a' in (7) with assigning k^0 by setting key bits of $CPNB_n$ to zero.
 - 12: Return $CPNB_n$, ϵ_a' , $\mathcal{X}_n$ and $\mathcal{C}_n$.
-

by new criteria are called the conditional PNBs (CPNBs for short). Specifically, we first generalize the definition of neutrality measure to conditional neutrality measure, and present a greedy algorithm to find good set of CPNBs.

Definition 3 (Conditional neutrality measure). *The conditional neutrality measure of the key bit $k[i]$ with respect to the function $f(k, v, t, Z)$ and condition $\mathcal{C}_i(k, v, t, Z)$ is defined as γ'_i , and $\Pr\{f(k, v, t, Z) = f(k \oplus e_i, v, t, Z) \mid \mathcal{C}_i(k, v, t, Z)\} = \frac{1}{2}(1 + \gamma'_i)$ i.e. the probability (over all k and (v, t)) that complementing the key bit $k[i]$ does not change the output of $f(k, v, t, Z) \mid \mathcal{C}_i(k, v, t, Z)$. Here e_i is the unit vector where the i -th bit is one and other bits are all zeros.*

In practice, the condition $\mathcal{C}_i(k, v, t, Z)$ for CPNB $k[i]$ is obtained by Algorithm 1 with treating only $k[i]$ as a PNB and other key bits as non-PNBs. As in the traditional way, one could construct a set of CPNBs by setting a threshold γ_{cNeutr} for conditional neutrality measure, i.e. $CPNB = \{i \mid |\gamma'_i| \geq \gamma\}$. However, there may raise the incompatibility among CPNBs. Suppose that the key bits $k[i]$ and $k[j]$ are separately derived as CPNBs, but they should not be CPNBs at the same time if the condition $\mathcal{C}_i(k, v, t, Z)$ for $k[i]$ being a CPNB depends on $k[j]$. Therefore, we treat all the CPNBs as a whole when constructing a set of CPNBs.

In general, the procedure of finding a good set of CPNBs is divided into two steps. First, in the preprocessing step, those key bits are excluded whose conditional neutrality measure is less than a threshold γ_{cNeutr} , which will not become good CPNBs. As a result, we get a preliminary shortlisting of CPNBs, denoted by $CPNB^{shortlist}$ whose elements are the key bits with conditional neutrality measure greater than γ_{cNeutr} . Note that the value of γ_{cNeutr} is set to quite lower than the ones in the conventional method, e.g. $\gamma_{cNeutr} = 2^{-5.0}$. In the second step, a greedy algorithm is launched to find a good set of CPNBs from $CPNB^{shortlist}$, which is described in detail below.

a) *Greedy algorithm with new criteria:* Once the size of CPNB set is specified, we construct the set of CPNBs by selecting PNB one by one. In the i -th iteration, the index of key bit with the maximal conditional correlation in backward direction, i.e. minimizing the time complexity, is selected and added into the CPNB set. To be accurate, we declare a temporary CPNB set $CPNB_{temp}$ to include a new key bit j , and update the set of profitable syncopated segments $\mathcal{X}_{i,j}$ and the conditions $\mathcal{C}_{i,j}$ by performing Algorithm 1 with $CPNB_{temp}$ as input. Then under the conditions $\mathcal{C}_{i,j}$, the correlation $\epsilon_a'^{(i,j)}$ in (7) is estimated with assigning k^0 by flipping the new bit j and setting the other bits in $CPNB_{temp}$ to zeros. This iteration is repeated until enough CPNBs are identified. The detailed procedure is presented in Algorithm 2, where ϵ_a' is the conditional backward correlation in a single initialization, that is, $\Pr[f = g \mid \mathcal{C}_n] = \frac{1}{2}(1 + \epsilon_a')$.

When there are large numbers of equations in conditions $\mathcal{C}$ of syncopated segments $\mathcal{X}$, it takes a lot of time overhead to satisfy them, and we will not be able to further explore the impact of conditions on the backward approximation. However, according to the propagation of modular addition and subtraction (Lemma 1 and 2), we know that under the conditions $\mathcal{C}$ of $\mathcal{X}$, the values of syncopated segments $\mathcal{X}$ are independent on PNBs and can be determinedly computed. In practice, we then estimate theoretically the backward correlation at Step 6 and 11 of Algorithm 2 on the premise that the values of $\mathcal{X} = \{(x[t+w-1:t], Carry[t+w])\}$ are known, that is, estimating the correlation between g_1 and f_1 with fixing the values of $\mathcal{X}$ in g_0 to the ones in f_0 . Moreover, our experiments show that the theoretically estimated correlation is very close to the actual one, and we can always use the theoretical value instead of the actual one. It is worth noting that the use of theoretical evaluation of conditional backward correlation makes our method to find good set of CPNBs very efficient.

2) *New key-recovery attack with syncopation technique:* In a chosen-IV attack, an attacker can get the keystream blocks by xoring the plaintexts and ciphertexts. Therefore, the accessible in-and-out data consists of public values (e.g., constants and IV)

and keystream block Z , which are collectively denoted by M for simplicity. Assume that a set of n CPNBs is generated by our new method, along with the set of syncopated segments $\mathcal{X}$ and corresponding conditions $\mathcal{C}$. Once obtaining a pair of accessible in-and-out data (M, M') , according to its values of the restricted bits $S^{\mathcal{R}}$, the attacker classifies it into the corresponding subset. In the online phase, since the non-PNBs of secret key are guessed, the values of key bits in $k^{\mathcal{R}}$ are known due to the bits of $k^{\mathcal{R}}$ are syncopated bits (that is, non-PNBs). The pairs are then filtered under the conditions $\mathcal{C}$. Suppose that N pairs are given, about $N^* = qN$ pairs will be remaining for subsequent statistic testing, where $q = (\prod (1 - \frac{1}{2^u})^{\theta_u})^2$ according to the premises of Lemma 1 and 2 and θ_u is the number of syncopated segments with an u -bit restricted segment. The detailed procedure of key-recovery attack is summarized as Algorithm 3.

Algorithm 3 Recovery of the full secret key

Input: N pairs $\{(M, M')\}$ of accessible data with IV satisfying ID , set of n CPNBs denoted by k^0 and corresponding conditions $\mathcal{C}$.

Output: The full secret key.

- 1: Classify N pairs into subsets according to the values of $S^{\mathcal{R}}$. ▷ Pre-processing data where no key bits are involved.
 - 2: **for all** $\hat{k}^1 \in \{0, 1\}^m$ **do** ▷ Identify the key $k = (k^1, k^0)$, $|k^1| = m$ and $|k^0| = n$.
 - 3: Set $\hat{k} = (\hat{k}^1, 0^n)$, according to the values of $k^{\mathcal{R}}$, choose the subsets of $\{(M, M')\}$ whose values of $S^{\mathcal{R}}$ satisfying equations in conditions $\mathcal{C}$, with about $N^* = qN$ pairs.
 - 4: Compute the correlation at OD with the filtered N^* pairs.
 - 5: **if** the optimal decision rule legitimates $\hat{k}^1$ as a (possibly) correct one **then**
 - 6: Perform an additional exhaustive search over the remaining n -bit subkey k^0 . Once that the right key is found, output it.
 - 7: **end if**
 - 8: **end for**
-

Analysis of complexities. The total time complexity of Algorithm 3 is $T = T_0 + T_1$, where the complexity T_0 of Step 1 is N , and the complexity T_1 of Step 2–8 can be estimated as done in [AFK⁺08]. Therefore,

$$T = N + 2^m N^* + 2^{\kappa - \alpha} + 2^n,$$

where $N = \frac{1}{q}N^*$, $N^* \approx (\frac{\sqrt{\alpha \log 4 + 3\sqrt{1 - (\varepsilon_a \varepsilon_d)^2}}}{\varepsilon_a \varepsilon_d})^2$, $q = (\prod (1 - \frac{1}{2^u})^{\theta_u})^2$, θ_u is the number of syncopated segments with an u -bit restricted segment, ε_d is the forward correlation, $\varepsilon_a = \epsilon_a'^2$ is the conditional backward correlation utilizing the syncopation technique, and κ is the size of secret key and $\alpha = 256$ for ChaCha. The data complexity is of N . The memory complexity is mainly consumed in Step 1 of at most N , and if we only keep these subsets used later, it takes memory of $2^{|k^{\mathcal{R}}|}N^*$. For concrete attacks, we refer to Sect. VII for details.

V. THE GENERALIZED FRAMEWORK BASED ON PNE

In this section, we introduce the generalized framework for the backward computation in the differential attack. Before the formal starting, we first provide a high-level overview of the underlying ideas. Let $f(K, Z, IV)$ be the approximated function in the backward computation, where K is the n -bit secret key, Z and IV are keystream and initial vector respectively. In the traditional method, the probabilistic neutral bits (PNBs) of f concerning K are searched and identified, and an approximation $g(K', Z, IV)$ of f is given with correlation ε_a where K' (nonPNBs) is part of K . In the paper, we further generalize and extend the PNB-based approximation method to find a better approximation with higher correlation and fewer secret bits. Let $H : \mathbb{F}_2^n \rightarrow \mathbb{F}_2^n$ be an invertible function of K , and f' be function satisfying $f'(H(K), Z, IV) = f(K, Z, IV)$. Different from the previous method, we search and identify the non-significant ones of $H(K)$, called as *probabilistic neutral expression* (PNE for short), and give an approximation $g'(H'(K), Z, IV)$ of $f'(H(K), Z, IV)$ with correlation ϵ_a' where $H'(K)$ (nonPNEs) is part of $H(K)$. In Fig. 5, we compare the new idea with the traditional one. The core idea is to apply the function H on K to obtain a simplified function f' , which makes the target function easier to approximate in cryptanalysis. The exact representations of H and f' depend on the specific cipher and attack parameters.

The structure of this section is arranged as follows. In Section V-A, we introduce the *carry-preserving* technique for modular addition which simplifies the non-linear operation in ARX ciphers. Then, in Section V-B, a new key-guessing strategy is proposed for the generalized PNE-based framework, especially for a common structure in ARX ciphers. After that, for applications, we illustrate these techniques on the backward computation of ChaCha, and present a method of searching PNEs for ChaCha in Section V-C.

A. Carry-Preserving Technique

In this section, we will introduce the *carry-preserving* technique. First, we deduce the property of carry bit's preserving its value under certain conditions in the modular addition operation, and formalize it as Lemma 3.

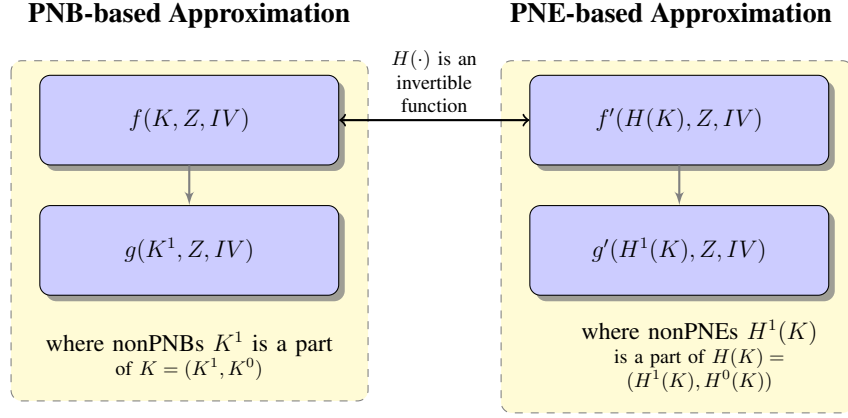


Fig. 5: Comparison of Traditional and New Frameworks

Lemma 3. Let $x, k, z \in \mathbb{F}_2^n$ and $z = x \boxplus k$, and the carry vector of modular addition is denoted by c , i.e. $c = z \oplus x \oplus k$. If $z[t+i] \oplus c[t] = 1$ for $i = 0, 1, \dots, w-1$, then $c[t+i+1] = c[t]$ for $i = 0, 1, \dots, w-1$.

Proof. Recall the bit-wise representation of modular addition. Let a modular addition $z = x \boxplus k$ where $x = (x_{n-1}, \dots, x_1, x_0)$, $k = (k_{n-1}, \dots, k_1, k_0)$, $z = (z_{n-1}, \dots, z_1, z_0) \in \mathbb{F}_2^n$, and the carry vector is $c = (c_{n-1}, \dots, c_1, c_0)$ i.e. $c = z \oplus x \oplus k$. There is a way to iteratively calculate the carry vector c from z and k as shown in the following formula,

$$c_{i+1} = (z_i \oplus 1)c_i \oplus (z_i \oplus 1)k_i \oplus c_i k_i, \quad i = 0, 1, \dots, n-2,$$

with $c_0 = 0$. In the following, the above iterative calculation is also denoted by $\text{Carry}'(z, k)$, that is $c = z \oplus x \oplus k = \text{Carry}'(z, k)$ where $z = x \boxplus k$.

From the above representation of c , we have that $c[t+1] = (z[t] \oplus 1 \oplus c[t])k[t] \oplus (z[t] \oplus 1)c[t]$. If $z[t] \oplus c[t] = 1$, then $c[t+1] = c[t]$. If $z[t+1] \oplus c[t+1] = 1$ i.e. $z[t+1] \oplus c[t] = 1$, then $c[t+2] = c[t+1] = c[t]$. Repeat the argument w times, and we get the conclusion of the lemma. $\square$

According to Lemma 3, if the bits of segment $z[t+w-1:t]$ are set into the value of $c[t] \oplus 1$, we have that all the bits of segment $c[t+w:t+1]$ are equal to $c[t]$. That is, the w -bit segment $c[t+w:t+1]$ of carry vector preserves the value of $c[t]$ and will not be affected by the segment of $k[t+w-1:t]$.

In the PNB-based approximation, non-significant bits of secret key are first identified in backward computation, and then only significant bits are guessed with ignoring the non-significant ones i.e. PNBs. Thus, the PNBs of secret key k are unknown and not accessible, which are denoted by an index set S . Next, we discuss what context we can compute without accessing the part of k indexed by S . Based on the above property of modular addition, we can draw the following conclusion summarized in Proposition 3.

Proposition 3 (Carry-Preserving Technique). Let $x = (x_{n-1}, x_{n-2}, \dots, x_0)$, $k = (k_{n-1}, k_{n-2}, \dots, k_0) \in \mathbb{F}_2^n$, and $S \subset \{0, 1, \dots, n-1\}$ be an index set of components in vector k , and $e_S \in \mathbb{F}_2^n$ is an n -bit vector with $e_i = 1$ if $i \in S$ and $e_i = 0$ if $i \in \bar{S}$ where $\bar{S}$ is the complement of S i.e. $S \cup \bar{S} = \{0, 1, \dots, n-1\}$. For modular addition $z = x \boxplus k$ with carry vector $c = (x \boxplus k) \oplus x \oplus k$, under the condition that the system of linear equations $\{z_i \neq c_i \mid i \in S\}$ are satisfied, the function on x and k , $\partial(z, k) = (z \boxplus k) \oplus (k \odot e_S)$ can be computed from z and $k \odot e_{\bar{S}}$ as follows, where the operation $\odot$ denotes bit-wise product i.e. $(a_{n-1}, a_{n-2}, \dots, a_0) \odot (b_{n-1}, b_{n-2}, \dots, b_0) = (a_{n-1}b_{n-1}, a_{n-2}b_{n-2}, \dots, a_0b_0)$.

For $i = 0, \dots, n-2$, with $c_0 = 0$,

$$c_{i+1} = \begin{cases} c_i, & \text{if } i \in S \\ (z_i \oplus 1)c_i \oplus (z_i \oplus 1)k_i \oplus c_i k_i, & \text{if } i \notin S \end{cases}$$

then $\partial(z, k) = z \oplus c \oplus (k \odot e_{\bar{S}})$.

Proof. Since $\partial(z, k) = (z \boxplus k) \oplus (k \odot e_S) = (z \oplus k \oplus c) \oplus (k \odot e_S) = z \oplus c \oplus (k \odot e_{\bar{S}})$, then we can only focus how to compute c from z and $k \odot e_{\bar{S}}$. According to Lemma 3, with $w = 1$, we have $i \in S$, $c_{i+1} = c_i$. Then, for $i = 0, \dots, n-1$, according to the index situation, compute c as follows:

- $i \in S$, $c_{i+1} = c_i$,
- $i \in \bar{S}$, $c_{i+1} = (z_i \oplus 1)c_i \oplus (z_i \oplus 1)k_i \oplus c_i k_i$,

where $c_0 = 0$. $\square$

From Proposition 3, we can see that, with the *carry-preserving* technique, modular addition can be inverted except for the part of k in S . That is, the new technique allow attackers to access more information of internal state from the guessed

subkey and public data. In Example 2, an example of 8-bit modular addition is given to demonstrate how the carry vector is determined in Proposition 3. To provide a clear visual representation, the change of carry vectors before and after using the *carry-preserving* technique is illustrated in Fig. 6, in which these unknown bits of carry vector are marked by gray and these known bits by white, blue, red or green colors.

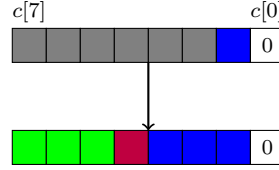


Fig. 6: The schematic diagram of carry vectors before and after using the *carry-preserving* technique, where the unknown bits of carry vector are marked by gray and known bits by white, blue, red or green colors.

Example 2. Let $x, k, z \in \mathbb{F}_2^8$ and $z = x \boxplus k$. In attacks, z is public data and k is secret key with non-PNBs guessed, assuming that $k[1, 2]$, $k[5, 6, 7]$ are PNBs, and others are non-PNBs. Let $S = \{1, 2, 5, 6, 7\}$, and $\bar{S} = \{0, 3, 4\}$. Note that $c[2] = (z[1] \oplus c[1] \oplus 1)k[1] \oplus (z[1] \oplus 1)c[1]$, $c[3] = (z[2] \oplus c[2] \oplus 1)k[2] \oplus (z[2] \oplus 1)c[2]$, $c[6] = (z[5] \oplus c[5] \oplus 1)k[5] \oplus (z[5] \oplus 1)c[5]$, $c[7] = (z[6] \oplus c[6] \oplus 1)k[6] \oplus (z[6] \oplus 1)c[6]$, $c[8] = (z[7] \oplus c[7] \oplus 1)k[7] \oplus (z[7] \oplus 1)c[7]$ (can be ignored due to no need to know $c[8]$), then the carry bits $c[2]$, $c[3]$, $c[6]$, $c[7]$ and $c[8]$ are uncertain due to involving unknown key bits i.e. PNBs $k[1, 2]$, $k[5, 6, 7]$ respectively. Since $c[3]$ is uncertain, then $c[4] = (z[3] \oplus c[3] \oplus 1)k[3] \oplus (z[3] \oplus 1)c[3]$ is uncertain, so $c[5] = (z[4] \oplus c[4] \oplus 1)k[4] \oplus (z[4] \oplus 1)c[4]$ is also uncertain.

According to Proposition 3, the linear equations $\{z[i] \neq c[i] \mid i \in S\}$ are imposed, and the carry vector $c = (x \boxplus k) \oplus x \oplus k$ can be computed from z and $k[0, 3, 4]$ as following,

$$\begin{cases} c[0] = 0, \\ c[1] = (z[0] \oplus 1)k[0], \\ c[2] = c[1], \\ c[3] = c[2], \\ c[4] = (z[3] \oplus 1)k[3] \oplus (z[3] \oplus 1)c[3] \oplus c[3]k[3], \\ c[5] = (z[4] \oplus 1)k[4] \oplus (z[4] \oplus 1)c[4] \oplus c[4]k[4], \\ c[6] = c[5], \\ c[7] = c[6]. \end{cases}$$

After applying Proposition 3, the bits of $c[7 : 3]$ will no longer depend on the unknown key bits, and consequently, they are determinedly computed from public data z and guessed key bits $\{k[0], k[3], k[4]\}$. So, with the *carry-preserving* technique, one can deterministically calculate the values of partial intermediate state which are uncertain before in attacks, which is useful to increase the correlation of approximation.

B. New Key Guessing Strategy

In this section, we will introduce the key-guessing strategy for the generalized PNE-based framework and focus on the results corresponding to a common structure in ARX ciphers. More precisely, the new key guessing strategy is about guessing the significant expressions of secret key and ignoring these non-significant ones called *probabilistic neutral (Boolean) expressions* (PNEs for short). More precisely, let $H : \mathbb{F}_2^n \rightarrow \mathbb{F}_2^n$ be an invertible function of secret key K , and $H(K)$ is the equivalent system of key expressions. Similar to PNB, the formal definition of PNE's neutrality measure is given as follow, which measures the influence amount which key expression has on output of f' .

Definition 4 (Neutrality measure of PNE). *The neutrality measure of the i -th key expression of $H(K)$ with respect to $f'(H(K), W)$ is defined as γ_i , where $W = (Z, IV)$ and $\Pr\{f'(H(K), W) = f'(H(K) \oplus e_i, W)\} = \frac{1}{2}(1 + \gamma_i)$, i.e. the probability (over all K and W) that complementing i -th key expression of $H(K)$ does not change the output of $f'(H(K), W)$, where e_i is the unit vector of which i -th bit is one and other bits are all zero.*

The conditional neutrality measure of PNE can similarly be defined. In fact, the concept of *probabilistic neutral (Boolean) expressions* is a generalization of *probabilistic neutral bits*, and we will collectively call them PNEs.

Linear Equivalent Key. In the following, we analyze a common structure in ARX ciphers consisting of XOR and left rotation operations, e.g. in ChaCha and BLAKE, and give the corresponding function H . Consider the special structure in Fig. 7 where

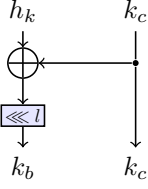


Fig. 7: Linear Relationship of Key

$k_b, k_c \in \mathbb{F}_2^n$ are the original key of size $2n$. As regards the linear equivalent relationship of key in Fig. 7, the function H can be expressed by a matrix

$$H = \begin{bmatrix} \rho^{n-l} & I \\ 0 & I \end{bmatrix}, \text{ where } \rho = \begin{bmatrix} 0 & 1 & 0 & \cdots & 0 \\ 0 & 0 & 1 & \cdots & 0 \\ 0 & 0 & 0 & \cdots & 1 \\ 1 & 0 & 0 & \cdots & 0 \end{bmatrix}.$$

That is the linear equivalent key h_k and k_c concerning k_b and k_c is computed as

$$\begin{bmatrix} h_k \\ k_c \end{bmatrix} = H \begin{bmatrix} k_b \\ k_c \end{bmatrix} = \begin{bmatrix} \rho^{n-l} & I \\ 0 & I \end{bmatrix} \begin{bmatrix} k_b \\ k_c \end{bmatrix}.$$

Since H is invertible, then the original key can be computed from the linear equivalent key as the following

$$\begin{bmatrix} k_b \\ k_c \end{bmatrix} = H^{-1} \begin{bmatrix} h_k \\ k_c \end{bmatrix} = \begin{bmatrix} \rho^l & \rho^l \\ 0 & I \end{bmatrix} \begin{bmatrix} h_k \\ k_c \end{bmatrix}. \quad (8)$$

In attacks, we partially guess the linear equivalent (h_k, k_c) instead of the original key (k_b, k_c) . Here to mention that the original key is used for computation in the carry-preserving technique. Next we show how many bits of the original key can we get given the part of linear equivalent key, and this is given in the following proposition.

Lemma 4. Let $S_1, S_2 \subseteq \{0, 1, \dots, n-1\}$ be the index sets of components of h_k and k_c , and the complements of S_1 and S_2 be denoted by $\bar{S}_1$ and $\bar{S}_2$, respectively. If only the parts $\bar{S}_1$ of h_k and $\bar{S}_2$ of k_c are given, then the part $\bar{S}_2$ of k_c is computed, and for k_b the part is $\{0, 1, \dots, n-1\} \setminus ((S_1 + l) \cup (S_2 + l))$ where $S + l = \{e + l \bmod n \mid e \in S\}$.

Proof. According to Eq.(8), $k_c = k_c$ and $k_b = (h_k \lll l) \oplus (k_c \lll l)$, and the result is directly derived. $\square$

C. Application to ChaCha

In this section, we present the application of the results in Section V-A and Section V-B on the stream cipher ChaCha, along with a method of searching PNEs for ChaCha.

1) *Backward computation of ChaCha:* Let us focus on the inversion of ChaCha's last round, which comprises the word-wise modular subtraction with initial state and four parallel applications of quarter-round's inversion on columns or diagonals as follows.

$$QF(z_a, z_b, z_c, z_d, k_b, k_c) = \text{InvQR}(z_a, z_b \boxminus k_b, z_c \boxminus k_c, z_d),$$

where the function $\text{InvQR}()$ is the inverse of quarter-round function, and the constants on branches a and d are ignored.

Separating bit-wise representation. For $x_b = z_b \boxminus k_b$, the bit-wise representation is $z_b \oplus k_b \oplus c_b$ where c_b is the carry vector of x_b and k_b i.e. $c_b = (x_b \boxplus k_b) \oplus x_b \oplus k_b = z_b \oplus x_b \oplus k_b$. The same to $x_c = z_c \boxminus k_c$ with the carry vector of x_c and k_c i.e. $c_c = (x_c \boxplus k_c) \oplus x_c \oplus k_c = z_c \oplus x_c \oplus k_c$. For the last rotation and XOR in quarter-round function at branch b and branch c , we have that

$$\begin{bmatrix} y_b \\ x_c \end{bmatrix} = H \begin{bmatrix} x_b \\ x_c \end{bmatrix},$$

where y_b is the word before the last rotation and XOR at branch b . With the bit-wise expression of $z_b \boxminus k_b, z_c \boxminus k_c$, we separate them into three parts as the following:

$$\begin{bmatrix} y_b \\ x_c \end{bmatrix} = H \begin{bmatrix} z_b \oplus k_b \oplus c_b \\ z_c \oplus k_c \oplus c_c \end{bmatrix} = H \begin{bmatrix} z_b \\ z_c \end{bmatrix} \oplus H \begin{bmatrix} k_b \\ k_c \end{bmatrix} \oplus H \begin{bmatrix} c_b \\ c_c \end{bmatrix}.$$

Let $\begin{bmatrix} h_k \\ k_c \end{bmatrix} = H \begin{bmatrix} k_b \\ k_c \end{bmatrix}$, and

$$QF'(z_a, z_b, z_c, z_d, h_k, k_c) = \text{InvQR}^*(z_a, y_b, x_c, z_d),$$

$$(z_a, y_b, x_c, z_d)^T = \begin{bmatrix} H \begin{bmatrix} z_b \\ z_c \end{bmatrix} \oplus \begin{bmatrix} z_a \\ h_k \\ k_c \\ z_d \end{bmatrix} \oplus H \begin{bmatrix} c_b \\ c_c \end{bmatrix} \end{bmatrix}$$

where the function $InvQR^*$ is a variant of $InvQR$ without the last rotation and XOR at branch b . It could be easily check that $QF'(z_a, z_b, z_c, z_d, h_k, k_c) = QF(z_a, z_b, z_c, z_d, k_b, k_c)$.

Since the new function, QF' 's input variables consist of the linear equivalent key (h_k, k_c) instead of the original key (k_b, k_c) , and then the two carry vectors (c_b, c_c) need to be computed from the linear equivalent key and (z_b, z_c) . For an approximation g' of the target f' , the part of linear equivalent key (h_k, k_c) is involved in g' with ignoring these non-significant ones. First, it should be determined which part of the original key (k_b, k_c) can be derived from the part of (h_k, k_c) as discussed in Lemma 4. Then the carry-preserving technique are applied twice on (c_b, c_c) independently with the parts of the original key (k_b, k_c) . According to the carry-preserving technique in Proposition 3, under the condition that the required system is satisfied, the two carry vectors (c_b, c_c) of g' have the same values of f' . Once the known parts of original key are determined, the following corollary gives the number of conditions the carry-preserving technique, which is derived by combining Lemma 4 and Proposition 3.

Corollary 1. *Let $S_1, S_2 \subset \{0, 1, \dots, n-1\}$ be the index sets of components of h_k and k_c respectively. If the part $\bar{S}_1$ of h_k and part $\bar{S}_2$ of k_c are given, then under the conditions that the system of $|(S_1 + l) \cup (S_2 + l)| + |S_2| \leq |S_1| + 2|S_2|$ linear equations are satisfied, that is*

$$\{z_b[i] \neq c_b[i] \mid i \in (S_1 + l) \cup (S_2 + l)\} \cup \{z_c[i] \neq c_c[i] \mid i \in S_2\}$$

the two functions $\partial(z_b, h_k) = (z_b \boxplus h_k) \oplus (h_k \odot e_{S_1})$ and $\partial(z_c, k_c) = (z_c \boxplus k_c) \oplus (k_c \odot e_{S_2})$ can be computed from z_b, z_c and the part $\bar{S}_1$ of h_k and part $\bar{S}_2$ of k_c .

2) *Method of Searching PNEs:* In this section, we present a method for searching Probabilistic Neutral Expressions (PNEs) for ChaCha. We begin with a generalized introduction to the method, followed by a specific algorithm tailored for searching PNEs in the context of ChaCha.

a) *Two-Phase Strategy:* In general, a two-phase strategy is proposed to get the final sets of PNEs and guessed expressions of secret key, consisting of *primary-PNBs-search* and *advanced-PNEs-search* phases as the following:

- First, in the *primary-PNBs-search* phase, we aim to find as many traditional PNBs as possible while utilizing the syncopation technique in backward approximation as shown in Algorithm 2.
- Then, in the *advanced-PNEs-search* phase, more new PNEs are searched for using the *carry-preserving* technique under new key guessing strategy. In this phase, we first identify the columns or diagonals in which no or few PNBs are found in the first phase. On these columns or diagonals, we use the procedure of searching PNEs to find more probabilistic neutral expressions as many as possible as described in Algorithm 4.

b) *Algorithm of Searching PNEs for ChaCha:* According to the discussion of Sect. V-C1, and considering the linear equivalent keys on some columns or diagonals, we obtain the corresponding target function

$$f'(H(K), Z, IV) = f'_r \circ (QF'(z_a, z_b, z_c, z_d, h_k, k_c), \dots),$$

where f'_r is the remaining part of f' and $f'(H(K), Z, IV) = f(K, Z, IV)$ holds. Then we search the PNEs of target function f' concerning $H(K)$. Assuming that an approximation g' of f' is found with PNEs index S_1 of h_k and PNEs index S_2 of k_c respectively. From Corollary 1, under the condition that at most $|S_1| + 2|S_2|$ linear equations are satisfied, f' becomes

$$f'_r \circ (InvQR^*(z_a, \left(H \begin{bmatrix} \partial(z_b, h_k) \\ \partial(z_c, k_c) \end{bmatrix} \right))^T \oplus (h_k \odot e_{S_1}, k_c \odot e_{S_2}), z_d, \dots). \quad (9)$$

Note that $\partial(z_b, h_k)$ is dependent on $h_k \odot e_{\bar{S}_1}$ and $\partial(z_c, k_c)$ is dependent on $k_c \odot e_{\bar{S}_2}$ in the above equation, and then they have the same value in g' and f' .

Procedure of greedy algorithm. To construct the PNE set of specific size, we first exclude those not good PNEs from $H(K)$ according to their neutrality measure. Then we select PNE bit by bit with a greedy criteria. Precisely, in the i -th iteration, the index of PNEs with the maximal correlation of Eq.(9), is selected and added into the PNE set. This iteration is repeated until enough PNEs are identified. The detailed procedure is presented in Algorithm 4.

Although the carry-preserving technique can discover more new PNEs, the cost of conditions it imposes is very high and is dependent on the lengths of PNB segments, as shown in Corollary 1 for each quarter-round. If the carry-preserving technique is applied too many times, the cost of its conditions will become enormous to collect sufficient data in attacks.

VI. THEORETICAL ANALYSIS OF DIFFERENTIAL EQUATIONS

In this section, our improvements on forward differential is described, which are obtained by an in-depth analysis of the equations controlling differential propagation in ARX structure. More precisely, a concise and effective way is found to satisfy the given differential, resulting in a reduced complexity in key-recovery attacks. Besides, a more comprehensive theoretical explanation of *strong keys* is presented, which covers all the remaining six (out of eight) cases lacked in previous analysis.

According to the experiments of [BLT20], for about 70% of the keys, called *weak keys*, there exists at least one IV satisfying the differential characteristic $\Delta_{in} \rightarrow \Delta_m$ in the first round, which is the differential characteristic of *quarterround* function

Algorithm 4 Greedy Algorithm of Searching PNEs

Input: The Boolean key expressions $H(K)$, and size n' of PNE set.

Output: The PNE set, its correlation and conditions.

- 1: Initialize a set $PNE_0 = \emptyset$.
 - 2: Get PNE_{cands} from $H(K)$. ▷ Exclude not good PNEs
 - 3: **for** $i \in \{0, 1, \dots, n' - 1\}$ **do** ▷ Find PNEs bit by bit with greedy criteria
 - 4: **for** $e \in PNE_{cands}$ and $e \notin PNE_i$ **do**
 - 5: Estimate conditional backward correlation $\epsilon_a^{(i,e)}$ of f' in Eq.(9) by setting PNE_i to random values and flipping the value of e .
 - 6: **end for**
 - 7: Choose e with maximal backward correlation, then $PNE_{i+1} \leftarrow PNE_i \cup \{e\}$.
 - 8: **end for**
 - 9: Estimate conditional backward correlation ϵ_a in Eq.(9) by setting $PNE_{n'}$ to random values.
 - 10: Return $PNE_{n'}$, corresponding correlation ϵ_a and conditions $\mathcal{C}_c$ of $PNE_{n'}$ by Corollary 1.
-

described in Section III-B. On the flip side, their experiments imply the existence of *strong keys* for which we can not find such IV. Recently, Dey et al. [DDSM22] provided a theoretical explanation about the differential characteristic mainly for two (out of eight) special cases, especially pointing out the case in which 12.5% of all keys becomes strong keys. But, for the other six cases, there is no theoretical explanation now. In the direction of finding a right pair, Dey et al. [DGSS22] proposed to achieve the right pairs by constructing a list consisting of at least one favorable IV for these *exploitable keys*. However, this treatment views the differential propagation as a black box, which results in a lot of data and storage overhead. In the following, it is shown that we fill in the missing parts of the previous work and overcome the shortcomings of existing methods to some extent based on the deeper analysis of differential equations of ARX structure.

For the differential characteristic $\Delta_{in} \rightarrow \Delta_m$ in the first round which is described in Section III-B, there are totally five differential equations to satisfy:

$$\begin{cases} c[22] = Carry_{(c,d')}[22], & (10) \\ d'[2] = Carry_{(a',b')}[2], & (11) \\ c'[10] = Carry_{(c',d'')}[10], & (12) \\ c'[30] = Carry_{(c',d'')}[30], & (13) \\ d''[22] = Carry_{(c',d'')}[22], & (14) \end{cases}$$

where $Carry_{(x,y)}[i]$ denotes the i -th bit of carry vector $Carry_{(x,y)}$ of modular addition between x and y , i.e. $x \boxplus y = x \oplus y \oplus Carry_{(x,y)}$. As shown in Fig. 1, Eqs. (10) and (11) guarantee that the difference is not propagated to next bits in the second and third addition modular of *quarterround* function respectively, Eqs. (12), (13) and (14) guarantee that the difference is not propagated to next bits in the fourth addition of *quarterround* function.

In the initialization of ChaCha, a fixed constant is loaded into branch a , an unknown secret key is loaded into branches b and c . Therefore, we can not control branches a , b and c . Only branch d can be varied to satisfy the differential equations given the secret key. First, it is observed that at least one bit of d is almost linearly involved in Eqs. (12), (13) and (14). Therefore, we can always flip the linear bits to satisfy corresponding equations. We summarized this observation in the following.

a) Observation of last three differential equations:

- Eq. (12): $d[26] = Carry_{(c',d'')}[10] \oplus Carry_{(c,d')}[10] \oplus c[10] \oplus a'[26]$.
- Eq. (13): $d[14] = Carry_{(c',d'')}[30] \oplus Carry_{(c,d')}[30] \oplus c[30] \oplus a'[14]$.
- Eq. (14): $d[30] \oplus d[18] = Carry_{(c',d'')}[22] \oplus Carry_{(a',b')}[14] \oplus Carry_{(c,d')}[2] \oplus a'[30] \oplus a'[14] \oplus b[2] \oplus c[2] \oplus a'[18]$.

Thus, based on the above observation, the last three equations are satisfied by flipping $d[26]$, $d[14]$ and $d[30]$, which is verified by experiments given later.

As for the first two differential equations, no obvious linear bits of b are involved in equations. Therefore, we could not simply flip some bits of d to satisfy the two equations. It is noted that the two equations can not always be satisfied, and specially, Dey et al. [DDSM22] discussed two special cases where Eq. (11) is always established and must not be established, i.e. $a'[2 : 0] \in \{000, 100\}$. For details about their results, please refer to Theorems 5 and 6 in [DDSM22]. Next, we give a more in-depth analysis of Eqs. (10) and (11) by considering them together, which results in a theoretical interpretation for other six complex cases. The analysis stems from an observation of interaction between the first two equations, which control the differential propagation of two consecutive modular additions.

b) Observation of the first two differential equations: Considering an internal variable $c'[21 : 20]$, we further analyze the interaction between constraints of Eqs. (10) and (11). Since the relation $b' = (b \oplus c') \lll 12$, we can obtain the range of $c'[21 : 20]$ from the constraints of $b'[1 : 0]$ derived by the assumption that Eq. (11) holds. Precisely, assume that Eq. (11)

holds, we have that $c'[21 : 20] \in C_{a'[2:0]}^{b[21:20]} \triangleq \{b'[1 : 0] \oplus b[21 : 20] \mid a'[2] = \lfloor (a'[1 : 0] + b'[1 : 0]) / 2^2 \rfloor\}$. Due to the fact that $Carry_{(c,d')}[22] = \begin{cases} 0, & \text{if } c'[21 : 20] > c[21 : 20], \\ 1, & \text{if } c'[21 : 20] < c[21 : 20], \\ Carry_{(c,d')}[20], & \text{if } c'[21 : 20] = c[21 : 20], \end{cases}$ Eq. (10) also restricts the range of $c'[21 : 20]$, i.e. if $c[22] = 1$ then $c'[21 : 20] \leq c[21 : 20]$; otherwise $c[22] = 0$ then $c'[21 : 20] \geq c[21 : 20]$. Therefore, the two constraints of Eq. (10) and (11) may contradict. Taking the two equations into account together, it is first identified that the cases Eqs. (10) and (11) can not hold at the same time. For this, a function $sign(\cdot, \cdot) : \{\alpha \mid \alpha \in \mathbb{Z}\} \times \mathbb{Z} \rightarrow \{1, -1, 0\}$ is defined as

$$sign(S, \beta) = \begin{cases} 1 & \text{if } \forall \alpha \in S, \alpha > \beta, \\ -1 & \text{if } \forall \alpha \in S, \alpha < \beta, \\ 0 & \text{otherwise,} \end{cases}$$

where $sign(\cdot) \neq 0$ implies a contradiction between Eqs. (10) and (11). Otherwise, a heuristically optimal setting of d is given such that as few bits of b and c as possible are involved in. Formally, we write in a form of lemma below.

Lemma 5. *Given two words $a'[2 : 0] \in \{0, 1\}^3$ and $b[21 : 20] \in \{0, 1\}^2$, let $C_{a'[2:0]}^{b[21:20]} \triangleq \{b'[1 : 0] \oplus b[21 : 20] \mid a'[2] = \lfloor (a'[1 : 0] + b'[1 : 0]) / 2^2 \rfloor\}$.*

- *For $a'[2 : 0] = 000$ and $\forall b[21 : 20] \in \{0, 1\}^2$, $a'[2 : 0] \in \{111, 001\}$ and $b[21 : 20] \in \{01, 10\}$, no matter what value of $c[22 : 20]$, Eqs. (10) and (11) can hold simultaneously, specially with the following setting.*
If $c[21 : 20] = \min(C_{a'[2:0]}^{b[21:20]})$ and $c[22] = 1$, or $c[21 : 20] = \max(C_{a'[2:0]}^{b[21:20]})$ and $c[22] = 0$,

$$\begin{cases} d[5 : 4] = a'[5 : 4] \oplus (3 \times c[22]) \\ d[3 - i] = a'[3 - i] \oplus c[19 - i] \oplus 1, i \in \{0, 1, \dots, t - 1\} \\ d[3 - t] = a'[3 - t] \oplus c[19 - t] \end{cases} \quad (15)$$

where $t \in \{0, 1, \dots, 19\}$ is the minimum integer such that $c[19 - t] = c[22]$. Otherwise,

$$\begin{cases} d[5 : 4] = a'[5 : 4] \oplus (2 \times c[22] + 1 - c[19]) \\ d[3] = a'[3] \oplus c[19] \end{cases} \quad (16)$$

- *For $a'[2 : 0] = 100$, Eq (11) must not be satisfied.*
- *For $a'[2 : 0] \in \{101, 110, 010, 011\}$ and $\forall b[21 : 20] \in \{0, 1\}^2$, $a'[2 : 0] \in \{111, 001\}$ and $b[21 : 20] \in \{00, 11\}$,*
– When $c[21 : 20] \notin C_{a'[2:0]}^{b[21:20]}$, if $c[22] = \frac{1}{2}(1 + sign(C_{a'[2:0]}^{b[21:20]}, c[21 : 20]))$ then Eqs. (10) and (11) can not hold simultaneously; otherwise, then Eqs. (10) and (11) can hold simultaneously, specially with the following setting,

$$\begin{cases} d[5 : 4] = a'[5 : 4] \oplus (\min(C_{a'[2:0]}^{b[21:20]}) - c[21 : 20] + c[22] \times 4 - c[19]) \\ d[3] = a'[3] \oplus c[19] \end{cases} \quad (17)$$

- *When $c[21 : 20] \in C_{a'[2:0]}^{b[21:20]}$, Eqs. (10) and (11) can hold simultaneously, specially with the following setting. If $c[21 : 20] = \min(C_{a'[2:0]}^{b[21:20]})$ and $c[22] = 1$, or $c[21 : 20] = \max(C_{a'[2:0]}^{b[21:20]})$ and $c[22] = 0$, the setting is same to Eqs. (15); otherwise, the setting is same to Eqs. (16).*

Proof. For modular addition of c and d' , the following holds

$$c[21 : 20] + d'[21 : 20] + Carry_{(c,d')}[20] = c'[21 : 20] + Carry_{(c,d')}[22] \times 4.$$

Taking Eq. (10) and (11) into the above formula, we have that

$$c[21 : 20] + d'[21 : 20] + Carry_{(c,d')}[20] = c'[21 : 20] + c[22] \times 4, \text{ for } c'[21 : 20] \in C_{a'[2:0]}^{b[21:20]} \quad (18)$$

where $C_{a'[2:0]}^{b[21:20]} \triangleq \{b'[1 : 0] \oplus b[21 : 20] \mid a'[2] = \lfloor (a'[1 : 0] + b'[1 : 0]) / 2^2 \rfloor\}$. Therefore, our goal is to find a value of $\sigma = d'[21 : 20] + Carry_{(c,d')}[20]$ satisfy Eq. (18), which can be controlled by varying branch d .

First, assume that the value of $sign(C_{a'[2:0]}^{b[21:20]}, c[21 : 20])$ is not equal to zero for some $c[21 : 20]$, if Eq. (11) is satisfied, then we have the assertion about the value of $Carry_{(c,d')}[22]$, i.e. $Carry_{(c,d')}[22] = \frac{1}{2}(1 - sign(C_{a'[2:0]}^{b[21:20]}, c[21 : 20]))$, where $sign(\cdot, \cdot) : \{\alpha \mid \alpha \in \mathbb{Z}\} \times \mathbb{Z} \rightarrow \{1, -1, 0\}$ is defined as

$$sign(S, \beta) = \begin{cases} 1 & \text{if } \forall \alpha \in S, \alpha > \beta, \\ -1 & \text{if } \forall \alpha \in S, \alpha < \beta, \\ 0 & \text{otherwise.} \end{cases}$$

However Eq. (11) requires that $Carry_{(c,d')}[22] = c[22]$. Therefore, when $c[22] = \frac{1}{2}(1 + sign(C_{a'[2:0]}^{b[21:20]}, c[21 : 20]))$, Eq. (10) and (11) can not hold simultaneously.

Next, it is discussed that for different cases, whether the above situation happens, and if not, how we set d to satisfy Eq. (18), i.e. satisfying Eqs. (10) and (11) simultaneously.

- For $a'[2:0] = 000$ and $\forall b[21:20] \in \{0,1\}^2$, $a'[2:0] \in \{111,001\}$ and $b[21:20] \in \{01,10\}$, the value of $\text{sign}(C_{a'[2:0]}^{rb[21:20]}, c[21:20])$ is equal to zero for any values of $c[21:20]$.

If $c[22] = 1$, then Eq. (10) requires $c'[21:20] \leq c[21:20]$, and if $c[21:20] = \min(C_{a'[2:0]}^{rb[21:20]})$, then with considering Eq. (11) together, we have that $c'[21:20]$ is only equal to $c[21:20]$. Putting $c'[21:20] = c[21:20]$ into Eq. (18), we have $d'[21:20] + \text{Carry}_{(c,d')}[20] = c[22] \times 4$, that is $d'[21:20] = c[22] \times 4 - c[22]$ and $\text{Carry}_{(c,d')}[20] = c[22]$. With expanding $\text{Carry}_{(c,d')}[20]$, we have $\text{Carry}_{(c,d')}[20] = c[19-t]$, if $d'[19-i] = c[19-i] \oplus 1$ for $0 \leq i \leq t-1$ and $d'[19-t] = c[19-t]$, where $0 \leq t \leq 19$. In order to involve as less bits of b and c as possible, we always choose the minimum integer t such that $c[19-t] = c[22]$ where $0 \leq t \leq 19$. Considering the relation $d' = (d \boxplus a') \lll 16$, Eqs. (10) and (11) are satisfied simultaneously with the setting of Eqs. (15). Similarly, the setting also applies in the case where $c[22] = 0$ and $c[21:20] = \max(C_{a'[2:0]}^{rb[21:20]})$.

Otherwise, if $c[22] = 1$, then we choose $c'[21:20] = c[21:20] - 1 < c[21:20]$; if $c[22] = 0$, then we choose $c'[21:20] = c[21:20] + 1 > c[21:20]$. Putting this setting of $c'[21:20]$ into Eq. (18), we have $d'[21:20] + \text{Carry}_{(c,d')}[20] = 2 \times c[22] + 1$. Note that there is no restriction for the value of $\text{Carry}_{(c,d')}[20]$ any more with this setting of $c'[21:20]$. Then with setting $d'[19] = c[19]$, and we have $\text{Carry}_{(c,d')}[20] = c[19]$. As a result, Eqs. (10) and (11) are satisfied simultaneously with the setting of Eqs. (16).

- For $a'[2:0] = 100$, it is easily derived that Eq (11) must not be satisfied.
- For $a'[2:0] \in \{101, 110, 010, 011\}$ and $\forall b[21:20] \in \{0,1\}^2$, $a'[2:0] \in \{111,001\}$ and $b[21:20] \in \{00,11\}$,
 - When $c[21:20] \notin C_{a'[2:0]}^{rb[21:20]}$, we have that $\text{sign}(C_{a'[2:0]}^{rb[21:20]}, c[21:20])$ is not equal to zero, and if $c[22] = \frac{1}{2}(1 + \text{sign}(C_{a'[2:0]}^{rb[21:20]}, c[21:20]))$, then Eqs. (10) and (11) can not hold simultaneously; otherwise, we can choose $c'[21:20]$ into any value of $C_{a'[2:0]}^{rb[21:20]}$, especially $\min(C_{a'[2:0]}^{rb[21:20]})$, and Eqs. (10) and (11) can hold simultaneously with the setting of Eqs. (17).
 - When $c[21:20] \in C_{a'[2:0]}^{rb[21:20]}$, we can discuss like done in the first item. Eqs. (10) and (11) can hold simultaneously, specially with the following setting. If $c[21:20] = \min(C_{a'[2:0]}^{rb[21:20]})$ and $c[22] = 1$, or $c[21:20] = \max(C_{a'[2:0]}^{rb[21:20]})$ and $c[22] = 0$, setting is same to Eqs. (15); otherwise, setting is same to Eqs. (16).

□

Example 3. Here, we show two specific examples of strong keys derived by Lemma 5.

- For the case of key such that $a'[2:0] = 110$, $b[21:20] = 00$, when Eq. (11) holds, we have that $c'[21:20] \in \{10,11\}$. If $c[21:20] \in \{00,01\}$, then $\text{Carry}_{(c,d')}[22] = 0$. If $c[22] = 1$, Eq. (10) must not be satisfied, and the corresponding key becomes a strong key.
- For the case of key such that $a'[2:0] = 010$, $b[21:20] = 00$, when Eq. (11) holds, we have that $c'[21:20] \in \{00,01\}$. If $c[21:20] \in \{10,11\}$, then $\text{Carry}_{(c,d')}[22] = 1$. If $c[22] = 0$, Eq. (10) must not be satisfied, and the corresponding key becomes a strong key.

In attacks, it is necessary to know how many unknown bits of secret key need to be guessed and how many public bits are consumed to satisfy the first two equations. For this purpose, the following corollary is directly derived by the above lemma.

Corollary 2. For $a'[2:0] \in \mathbb{F}_2^3$ and $a'[2:0] \neq 100$, with the setting of Lemma 5,

- In Eq. (15), there are at most 22 bits of d i.e. $d[5:0]$ and $d[31:16]$, and at most 32 bits of b and 23 bits of c i.e. $c[22:0]$ are needed to guess. Note that this setting is used only when $c[21:20] = \min(C_{a'[2:0]}^{rb[21:20]})$ and $c[22] = 1$, or $c[21:20] = \max(C_{a'[2:0]}^{rb[21:20]})$ and $c[22] = 0$, which are 25% of all cases. If $t \leq 3$, with probability $\frac{15}{16}$, at most 6 bits of d i.e. $d[5:0]$ are restricted, and at most 8 bits of b i.e. $a'[5:0]$ and $b[21:20]$ and 7 bits of c i.e. $c[22:16]$ are needed to guess.
- In Eqs. (16) and (17), there are 3 bits of d i.e. $d[5:3]$, and at most 8 bits of b i.e. $a'[5:0]$ and $b[21:20]$ and 4 bits of c i.e. $c[22:19]$ needed to guess.

A. Reducing Complexities

Based on our observations of the five differential equations, we propose to use a concise and efficient way to generate a right pair for the given differential $\Delta_{in} \rightarrow \Delta_m$ in the first round. Given a weak key, from Corollary 2, we know that with probability greater than $\frac{15}{16}$ one can use at most six IV bits $\{0:5\}$ to satisfy the first two equations. And we can flip three IV bits $\{14, 26, 30\}$ to satisfy the last three equations based on the aforementioned observation. Inspired by the analysis of five differential equations, a proposition is presented as follows.

Proposition 4. Let $k_{ID} \in \mathbb{F}_2^{64}$, if k_{ID} is a weak key that there exists at least one $v \in \mathbb{F}_2^{32}$ satisfying the given differential, then with probability greater than $\frac{15}{16} = 93.75\%$, one can find v in a subspace $\mathbb{S}_{V_{free}} \subseteq \mathbb{F}_2^{32}$ with 2^9 IVs such that

$((k_{ID}, v), (k_{ID}, v'))$ forms a right pair, where the subspace $\mathbb{S}_{V_{nfree}} = \{(a_{31}, a_{30}, \dots, a_0) \mid a_i = 0 \text{ for } i \notin V_{nfree}\}$ and the index subset $V_{nfree} = \{0 : 5, 14, 26, 30\}$.

We performed experiments to verify the proposition. As a result, for 66.4% keys among all the keys we find one v in the subspace in Proposition 4 that satisfies the given differential, which confirms Proposition 4, i.e., $\geq 70\% \times \frac{15}{16} = 65.6\%$.

B. Theoretical Interpretation of Strong Key

According to Lemma 5, we derive a lower bound for the percentage of strong keys $k_{ID} \in \mathbb{F}_2^{64}$ that there is no IV $v \in \mathbb{F}_2^{32}$ satisfying the given differential $\Delta_{in} \rightarrow \Delta_m$ in the first round for each case of $a'[2 : 0]$.

Theorem 1. Among all the keys, there are at least 29.6875% strong keys $k_{ID} \in \mathbb{F}_2^{64}$ that there is no IV $v \in \mathbb{F}_2^{32}$ satisfying the given differential, for different $a'[2 : 0]$ we have

$$\tau = \begin{cases} 0 & \text{if } a'[2 : 0] = 000, \\ 1 & \text{if } a'[2 : 0] = 100, \\ 0.0625 & \text{if } a'[2 : 0] = 001, \\ 0.375 & \text{if } a'[2 : 0] = 101, \\ 0.25 & \text{if } a'[2 : 0] = 010, \\ 0.25 & \text{if } a'[2 : 0] = 110, \\ 0.375 & \text{if } a'[2 : 0] = 011, \\ 0.0625 & \text{if } a'[2 : 0] = 111. \end{cases}$$

where τ denotes the percentage of strong keys derived by Lemma 5.

Proof. According to Lemma 5, for $c'[2 : 0] \in \{101, 110, 010, 011\}$ and $\forall b[21 : 20] \in \{0, 1\}^2$, $c'[2 : 0] \in \{111, 001\}$ and $b[21 : 20] \in \{00, 11\}$, if $c[21 : 20] \in \{0, 1\}^2 \setminus C_{a'[2:0]}'^{b[21:20]}$ where $C_{a'[2:0]}'^{b[21:20]} \triangleq \{b'[1 : 0] \oplus b[21 : 20] \mid a'[2] = \lfloor (a'[1 : 0] + b'[1 : 0]) / 2^2 \rfloor\}$,

then there always exists a value of $c[22]$ such that a key become the strong key. Therefore, we have $\tau = \frac{1}{2} \times \frac{4 - |C_{a'[2:0]}'^{b[21:20]}|}{4}$, where τ denote the percentage of strong keys derived by Lemma 5. More precisely, we have

$$\tau = \begin{cases} 0.0625 & \text{if } a'[2 : 0] = 001, \\ 0.375 & \text{if } a'[2 : 0] = 101, \\ 0.25 & \text{if } a'[2 : 0] = 010, \\ 0.25 & \text{if } a'[2 : 0] = 110, \\ 0.375 & \text{if } a'[2 : 0] = 011, \\ 0.0625 & \text{if } a'[2 : 0] = 111. \end{cases}$$

Besides, Eq. (11) must not be satisfied when $a'[2 : 0] = 100$. Totally, there are at least 29.6875% strong keys for all possibilities of $a'[2 : 0]$. $\square$

Analyzing six out of eight more complex cases, our theoretical derivation of strong keys is basically consistent with the experimental result 30%.

VII. APPLICATIONS: ATTACKS ON 256-BIT CHACHA

In this section, we present the key-recovery attacks on ChaCha7, $7.5^\oplus$, ChaCha7.5 and ChaCha6 with the new techniques, where unless otherwise specified, the 3.5-round DL distinguisher in Table III will be used. Finally, a key-recovery attack on ChaCha8 is presented in the related-key setting.

A. Attacks against ChaCha7

First the inverse structure of ChaCha7 is analyzed, which is used to determine all the syncopated segments of internal state in Algorithm 1. The last round of ChaCha7 is illustrated in Fig. 8. For the inversion of last *quarterround* function, since the following relationships,

$$\begin{cases} x_7^{(7)} = z_7^{(7)} \boxplus k_3 \\ x_{11}^{(7)} = z_{11}^{(7)} \boxplus k_7 \\ y_{11}^{(6)} = (z_{11}^{(7)} \boxplus x_{15}^{(7)}) \boxplus k_7 \\ x_{11}^{(6)} = (z_{11}^{(7)} \boxplus x_{15}^{(7)} \boxplus y_{15}^{(6)}) \boxplus k_7 \end{cases}, \begin{cases} x_{3..}^{(7)} = z_3^{(7)} \boxplus c_3 \\ x_{15..}^{(7)} = z_{15}^{(7)} \boxplus v_2 \\ y_{15..}^{(6)} = (x_{15}^{(7)} \ggg 8) \oplus x_{3..}^{(7)} \end{cases}$$

the syncopation technique can be applied on words $x_7^{(7)}$, $x_{11}^{(7)}$, $y_{11}^{(6)}$, and $x_{11}^{(6)}$ with using the property of modular subtraction (Lemma 2), where the words underlined by x are known since they can be derived from constants, IV and keystream blocks.

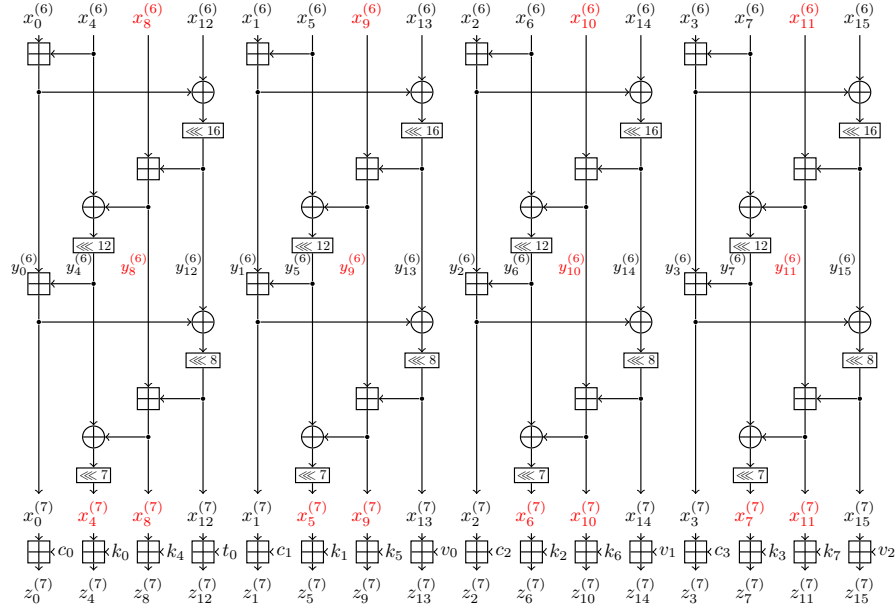


Fig. 8: Last round of ChaCha7.

The analysis is similar for the other three *quarterround* functions, and we mark all the words by red in Fig. 8 which the syncopation technique can be applied on. Given a (C)PNB set, Algorithm 1 is performed, and for the words marked by red in Fig. 8, the syncopated segments of internal state and corresponding conditions on restricted bits are determined according to the syncopated segments and restricted segments of secret key. The attacks with better parameters are obtained for ChaCha7 by setting $u = 1$, i.e., only using the syncopated segments with a one-bit restricted segment.

The set of CPNBs. In the first step, we obtain a preliminary shortlisting of 151 possible CPNBs by setting a threshold 2^{-5} for the conditional neutrality measure, which is $\{2, 3, 4, 5, 6, 7, 10, 11, 14, 15, 16, 19, 22, 26, 27, 31, 33, 35, 39, 40, 41, 43, 47, 48, 49, 51, 52, 53, 59, 60, 64, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 90, 91, 92, 93, 95, 96, 97, 99, 102, 103, 104, 105, 106, 107, 108, 109, 110, 111, 112, 115, 116, 117, 119, 120, 123, 124, 125, 126, 127, 128, 131, 132, 135, 136, 137, 140, 143, 147, 148, 152, 155, 156, 157, 158, 159, 160, 161, 162, 164, 165, 168, 169, 170, 174, 180, 181, 186, 188, 191, 192, 193, 194, 195, 196, 199, 200, 201, 202, 204, 205, 207, 208, 211, 212, 213, 214, 216, 219, 220, 221, 222, 223, 224, 225, 226, 227, 228, 232, 233, 237, 240, 244, 245, 246, 247, 249, 250, 252, 255\}$. Then Algorithm 2 is executed with the input of the preliminary shortlisting, and the size of the CPNB set is exhaustively searched. As a result, two sets with 74 CPNBs and 89 CPNBs are selected by minimizing time complexity under the requirement that data complexity does not exceed the amount of available IV. The set of 74 CPNBs is $\{2, 3, 4, 5, 47, 48, 49, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 90, 91, 102, 103, 104, 105, 106, 107, 108, 109, 110, 123, 124, 125, 126, 127, 155, 156, 157, 158, 159, 168, 169, 191, 192, 193, 194, 199, 200, 207, 208, 211, 212, 219, 220, 221, 222, 223, 224, 225, 226, 244, 245, 246, 247, 255\}$. In Fig. 9, the 74 gray lines represent 74 CPNBs, i.e. syncopation bits according to their indexes of secret key, and the blue and red lines represent the non-CPNBs where the syncopated segments k^o and restricted segments k^R of secret key are denoted by areas of blue and red lines respectively. The profitable syncopated segments for the 74 CPNBs are summarized in Table V along with 27 conditions, where we ignore the carry bits after syncopated segments. The set of 89 CPNBs is $\{2, 3, 4, 5, 6, 14, 15, 26, 47, 48, 49, 51, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 90, 91, 95, 102, 103, 104, 105, 106, 107, 108, 109, 110, 111, 115, 123, 124, 125, 126, 127, 135, 136, 147, 155, 156, 157, 158, 159, 168, 169, 170, 191, 192, 193, 194, 195, 199, 200, 207, 208, 211, 212, 219, 220, 221, 222, 223, 224, 225, 226, 227, 232, 244, 245, 246, 247, 255\}$. The profitable syncopated segments of the 89 CPNBs are given in Table VI along with 40 conditions.

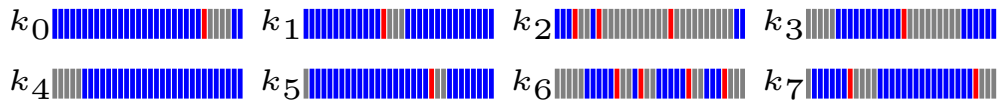


Fig. 9: The classification of all key bits for 74 CPNBs, where gray line represents CPNBs, and blue and red lines represent non-CPNBs, from left to right for $k_i[31 : 0]$.

From Table V, we remove 13 inessential conditions whose coordinates in table are $\{(T0, 8), (T0, 11), (T1, 8), (T1, 9), (T1, 10), (T1, 11), (T1, 12), (T2, 6), (T2, 8), (T2, 9), (T2, 10), (T2, 11), (T2, 12)\}$, where (Ti, j) locates a condition with type Ti and top-

TABLE V: Syncopated segments of 74 CPNBs, where “-” denotes $w = 0$.

syncopated segment $\leftarrow$ condition	(i, j_1, j_2)
T0: $x_4^{(7)}[j_2 : j_1] \leftarrow k_0[i] \neq z_4^{(7)}[i]$	(6, 7, 31)
T0: $x_5^{(7)}[j_2 : j_1] \leftarrow k_1[i] \neq z_5^{(7)}[i]$	(18, 19, 31)
T0: $x_6^{(7)}[j_2 : j_1] \leftarrow k_2[i] \neq z_6^{(7)}[i]$	$\{(12, -, -), (24, 25, 25), (28, 29, 31)\}$
T0: $x_7^{(7)}[j_2 : j_1] \leftarrow k_3[i] \neq z_7^{(7)}[i]$	(15, 16, 26)
T0: $x_9^{(7)}[j_2 : j_1] \leftarrow k_5[i] \neq z_9^{(7)}[i]$	(10, 11, 30)
T1: $y_9^{(6)}[j_2 : j_1] \leftarrow k_5[i] \neq (z_9^{(7)} \boxplus x_{13}^{(7)})[i]$	
T2: $x_9^{(6)}[j_2 : j_1] \leftarrow k_5[i] \neq (z_9^{(7)} \boxplus x_{13}^{(7)} \boxplus y_{13}^{(6)})[i]$	
T0: $x_{10}^{(7)}[j_2 : j_1] \leftarrow k_6[i] \neq z_{10}^{(7)}[i]$	$\{(3, 4, 6), (9, 10, 14), (17, 18, 18), (21, 22, 26)\}$
T1: $y_{10}^{(6)}[j_2 : j_1] \leftarrow k_6[i] \neq (z_{10}^{(7)} \boxplus x_{14}^{(7)})[i]$	
T2: $x_{10}^{(6)}[j_2 : j_1] \leftarrow k_6[i] \neq (z_{10}^{(7)} \boxplus x_{14}^{(7)} \boxplus y_{14}^{(6)})[i]$	
T0: $x_{11}^{(7)}[j_2 : j_1] \leftarrow k_7[i] \neq z_{11}^{(7)}[i]$	$\{(3, 4, 19), (24, 25, 30)\}$
T1: $y_{11}^{(6)}[j_2 : j_1] \leftarrow k_7[i] \neq (z_{11}^{(7)} \boxplus x_{15}^{(7)})[i]$	
T2: $x_{11}^{(6)}[j_2 : j_1] \leftarrow k_7[i] \neq (z_{11}^{(7)} \boxplus x_{15}^{(7)} \boxplus y_{15}^{(6)})[i]$	

 TABLE VI: Syncopated segments of 89 CPNBs, where “-” denotes $w = 0$.

syncopated segment $\leftarrow$ condition	(i, j_1, j_2)
T0: $x_4^{(7)}[j_2 : j_1] \leftarrow k_0[i] \neq z_4^{(7)}[i]$	$\{(7, 8, 13), (16, 17, 25), (27, 28, 31)\}$
T0: $x_5^{(7)}[j_2 : j_1] \leftarrow k_1[i] \neq z_5^{(7)}[i]$	$\{(18, -, -), (20, 21, 31)\}$
T0: $x_6^{(7)}[j_2 : j_1] \leftarrow k_2[i] \neq z_6^{(7)}[i]$	$\{(12, -, -), (24, 25, 25), (28, 29, 30)\}$
T0: $x_7^{(7)}[j_2 : j_1] \leftarrow k_3[i] \neq z_7^{(7)}[i]$	$\{(16, 17, 18), (20, 21, 26)\}$
T0: $x_8^{(7)}[j_2 : j_1] \leftarrow k_4[9] \neq z_8^{(7)}[9]$	$\{(9, 10, 18), (20, 21, 26)\}$
T1: $y_8^{(6)}[j_2 : j_1] \leftarrow k_4[9] \neq (z_8^{(7)} \boxplus x_{12}^{(7)})[9]$	
T2: $x_8^{(6)}[j_2 : j_1] \leftarrow k_4[9] \neq (z_8^{(7)} \boxplus x_{12}^{(7)} \boxplus y_{12}^{(6)})[9]$	
T0: $x_9^{(7)}[j_2 : j_1] \leftarrow k_5[i] \neq z_9^{(7)}[i]$	(11, 12, 30)
T1: $y_9^{(6)}[j_2 : j_1] \leftarrow k_5[i] \neq (z_9^{(7)} \boxplus x_{13}^{(7)})[i]$	
T2: $x_9^{(6)}[j_2 : j_1] \leftarrow k_5[i] \neq (z_9^{(7)} \boxplus x_{13}^{(7)} \boxplus y_{13}^{(6)})[i]$	
T0: $x_{10}^{(7)}[j_2 : j_1] \leftarrow k_6[i] \neq z_{10}^{(7)}[i]$	$\{(4, 5, 6), (10, 11, 14), (17, 18, 18), (21, 22, 26)\}$
T1: $y_{10}^{(6)}[j_2 : j_1] \leftarrow k_6[i] \neq (z_{10}^{(7)} \boxplus x_{14}^{(7)})[i]$	
T2: $x_{10}^{(6)}[j_2 : j_1] \leftarrow k_6[i] \neq (z_{10}^{(7)} \boxplus x_{14}^{(7)} \boxplus y_{14}^{(6)})[i]$	
T0: $x_{11}^{(7)}[j_2 : j_1] \leftarrow k_7[i] \neq z_{11}^{(7)}[i]$	$\{(4, 5, 7), (9, 10, 19), (24, 25, 30)\}$
T1: $y_{11}^{(6)}[j_2 : j_1] \leftarrow k_7[i] \neq (z_{11}^{(7)} \boxplus x_{15}^{(7)})[i]$	
T2: $x_{11}^{(6)}[j_2 : j_1] \leftarrow k_7[i] \neq (z_{11}^{(7)} \boxplus x_{15}^{(7)} \boxplus y_{15}^{(6)})[i]$	

to-bottom order j in right column of table, $0 \leq i \leq 2$ and $0 \leq j \leq 12$. For example, coordinate (T0, 8) locates the syncopated segment and its condition $x_{10}^{(7)}[14 : 10] \leftarrow k_6[9] \neq z_{10}^{(7)}[9]$. In the experiment of 2^{10} random keys and $2^{16.3}$ IVs for each key, the conditional correlation is theoretically estimated as $\epsilon'_a = 0.2365$. We have conducted a verification experiment for this theoretical calculation, in which 2^6 random keys are used with $2^{32.6}$ IVs for each key. The experiment results show that the conditional correlation is $\epsilon'_a = 0.2369$, which is very close to the theoretical one and justifies the theoretical calculation of conditional correlation. Therefore, the conditional backward correlation is $\epsilon_a = 2^{-4.2}$, and with $n = 74$, $\theta_1 = 14$ and $\alpha = 48$, the time and data complexities are $T = 2^{213.9}$ and $N = 2^{59.9}$ respectively.

Analysis of complexities. The analysis in Section VI is used to satisfy the differential in the first round for the forward distinguisher in Table III. According to Corollary 2, we should know fifteen key bits $\{32 : 37, 52, 53, 176 : 182\}$ to satisfy the first two differential equations, and they are not involved in the set of 74 CPNBs. Therefore, we can freely satisfy the first two differential equations by setting at most six IV bits $\{0 : 5\}$ as done in Lemma 5. The overall attack process is as follows: in Step 1 of Algorithm 3 one prepares 2^6 structures of data with N pairs in each structure by traversing six IV bits $\{0 : 5\}$, and in Step 3 of Algorithm 3 one first selects the structure which satisfies the first two differential equations according to the value of guessed key, and then as specified in Algorithm 3 chooses the subsets satisfying 28 equations in conditions and then performs the statistic testing. To satisfy the last three differential equations, this process needs to be performed 2^3 times. The N pairs should be generated by varying the other three branches at which no input difference is imposed, i.e. $N \leq 2^{96}$, without affecting differential propagation in the first round. As a result, the total time complexity is $2^{213.9+3} = 2^{216.9}$, and data complexity is $2^{59.9+9} = 2^{68.9}$. As for the memory complexity is of $2^{6+|k^R|N^*} = 2^{51.9}$ since we only store the subsets used later in each structure with ignoring the subsets that must not satisfy 28 equations in conditions.

Similarly, we remove 13 inessential conditions from the 40 conditions of 89 CPNBs, and their coordinates in Table VI

are $\{(T0, 14), (T0, 17), (T1, 14), (T1, 15), (T1, 16), (T1, 18), (T1, 19), (T2, 11), (T2, 12), (T2, 15), (T2, 16), (T2, 18), (T2, 19)\}$, where (Ti, j) locates a condition with type Ti and top-to-bottom order j in right column of table, $0 \leq i \leq 2$ and $0 \leq j \leq 19$. By the experiment with 2^{10} random keys and $2^{20.3}$ IVs for each key, the conditional correlation ϵ'_a is theoretically estimated as $2^{-4.14}$. Therefore, the conditional backward correlation is $\epsilon_a = 2^{-8.28}$, and with $n = 89$, $\theta_1 = 27$ and $\alpha = 54$, we obtain another attack with less time complexity of $2^{207.3+3} = 2^{210.3}$ and data complexity of $2^{94.3+9} = 2^{103.3}$. The corresponding memory complexity is of $2^{6+|k^R|} N^* = 2^{73.3}$.

Improved attack. Below, we present an improved key-recovery attack on ChaCha7 by leveraging our syncopation technique. This new attack benefits from our finding of the forward differential of the first round in Table IV and the amplified correlation of backward approximation by the syncopation technique.

First, we give the details of our observation on the differential propagation in the first round of five-round DL distinguisher in Table IV. That is, we find that there exists some IV bits which have less influence on the differential $\Delta'_{in} = ([,], [9, 29]) \rightarrow \Delta'_m = ([5, 25], [12, 28], [21, 25], [13, 21])$. More precisely, let E_1 be *quarterround* function of the first round and X be input of E_1 . One can find a right pair $(X, X \oplus \Delta'_{in})$ with probability $\Pr_X[E_1(X) \oplus E_1(X \oplus \Delta'_{in}) = \Delta'_m] = 2^{-7}$. First, we identify those bits of IV such that when we flip the IV bit a right pair stays a right pair with high probability. And six such IV bits are found $\{22, 10, 23, 11, 30, 24\}$, which form a subspace of 2^6 elements $\mathbb{V} = \{v = a_1 e_{10} \oplus a_2 e_{11} \oplus a_3 e_{22} \oplus a_4 e_{23} \oplus a_5 e_{24} \oplus a_6 e_{30} \mid (a_1, a_2, a_3, a_4, a_5, a_6) \in \mathbb{F}_2^6\}$ where $e_i \in \mathbb{F}_2^{128}$ is an unit vector with the $(i + 96)$ -th bit is 1 and other bits are zeros. With 2^{30} random samples, we experimentally evaluate the probability that one find X such that all the 2^6 elements of the coset $X \oplus \mathbb{V}$ are right pairs, that is $p_1 = \Pr_X[\forall v \in \mathbb{V}, E_1(X \oplus v) \oplus E_1(X \oplus v \oplus \Delta'_{in}) = \Delta'_m]$ is estimated to $2^{-7.8}$. Moreover, there are extra 96 bits of IV which are full independently of the differential in the first round, that is three 32-bit IV words in the other three columns on which no input difference is imposed. Then with probability $2^{-7.8}$, one can find an input such that all the 2^{102} elements of its coset are right pairs of the differential in the first round.

Then, we give the procedure of key-recovery attack on ChaCha7. In the forward direction, we use the five-round DL distinguisher in Table IV and find the right pair of the first round according to the above discussion. For the backward direction, with the syncopation technique, we amplify the backward correlation of the 169 PNBs in [XXTQ24] by imposing the four conditions in Table VII. In the experiment with 2^{10} random keys and $2^{20.3}$ IVs for each key, the correlation ϵ'_a is theoretically estimated as $2^{-5.1}$. Therefore, the conditional backward correlation is $\epsilon_a = 2^{-10.2}$. With $n = 169$, $\theta_1 = 4$ and $\alpha = 83$, we obtain an attack with time complexity of $2^{179.4+7.8} = 2^{187.2}$ and data complexity of $2^{92.4+8+7.8} = 2^{108.2}$. The corresponding memory complexity is $2^{96.4}$.

TABLE VII: Syncopated segments of another improved attack on ChaCha7, where “-” denotes $w = 0$.

syncopated segment $\leftarrow$ condition	(i, j_1, j_2)
$T0: x_6^{(7)}[j_2 : j_1] \leftarrow k_2[i] \neq z_6^{(7)}[i]$	$(18, -, -)$
$T0: x_{10}^{(7)}[j_2 : j_1] \leftarrow k_6[i] \neq z_{10}^{(7)}[i]$	$(16, 17, 17)$
$T0: x_6^{(7)}[j_2 : j_1] \leftarrow k_2[i] \neq z_6^{(7)}[i]$	$(11, 12, 12)$
$T0: x_6^{(7)}[j_2 : j_1] \leftarrow k_2[i] \neq z_6^{(7)}[i]$	$(5, 6, 6)$

B. Attacks against ChaCha7.25

Similar to ChaCha7, we show an enhanced key-recovery attack on ChaCha7.25. In the forward direction, we use the five-round DL distinguisher in Table IV is used in the forward direction and find the right pair of the first round as done in ChaCha7. For the backward direction, with the syncopation technique, we amplify the backward correlation of the 133 PNBs in [XXTQ24] by imposing the six conditions in Table VIII. In the experiment with 2^{10} random keys and $2^{21.3}$ IVs for each key, the correlation ϵ'_a is theoretically estimated as $2^{-4.45}$. Therefore, the conditional backward correlation is $\epsilon_a = 2^{-8.9}$, and with $n = 133$, $\theta_1 = 6$ and $\alpha = 49$, we obtain an attack with time complexity of $2^{212.2+7.8} = 2^{220}$ and data complexity of $2^{89.2+12+7.8} = 2^{109}$. The corresponding memory complexity is $2^{95.2}$. The attack also works on the variants by adding modular addition at round function of ChaCha7.25 as shown in [XXTQ24].

TABLE VIII: Syncopated segments of improved attack on ChaCha7.25, where “-” denotes $w = 0$.

syncopated segment $\leftarrow$ condition	(i, j_1, j_2)
$x_6^{(7.5\oplus)}[j_2 : j_1] \leftarrow k_2[i] \neq z_6^{(7.5\oplus)}[i]$	$\{(5, 6, 6), (11, 12, 12), (18, -, -), (23, 24, 24)\}$
$x_1^{(7)}[j_2 : j_1] \leftarrow k_2[i] = (x_1^{(7.5\oplus)} \boxminus z_6^{(7.5\oplus)})[i]$	$(14, 15, 15)$
$x_{10}^{(7)}[j_2 : j_1] \leftarrow k_6[i] \neq (z_{10}^{(7.5\oplus)} \boxminus x_{15}^{(7.5\oplus)})[i]$	$(16, 17, 17)$

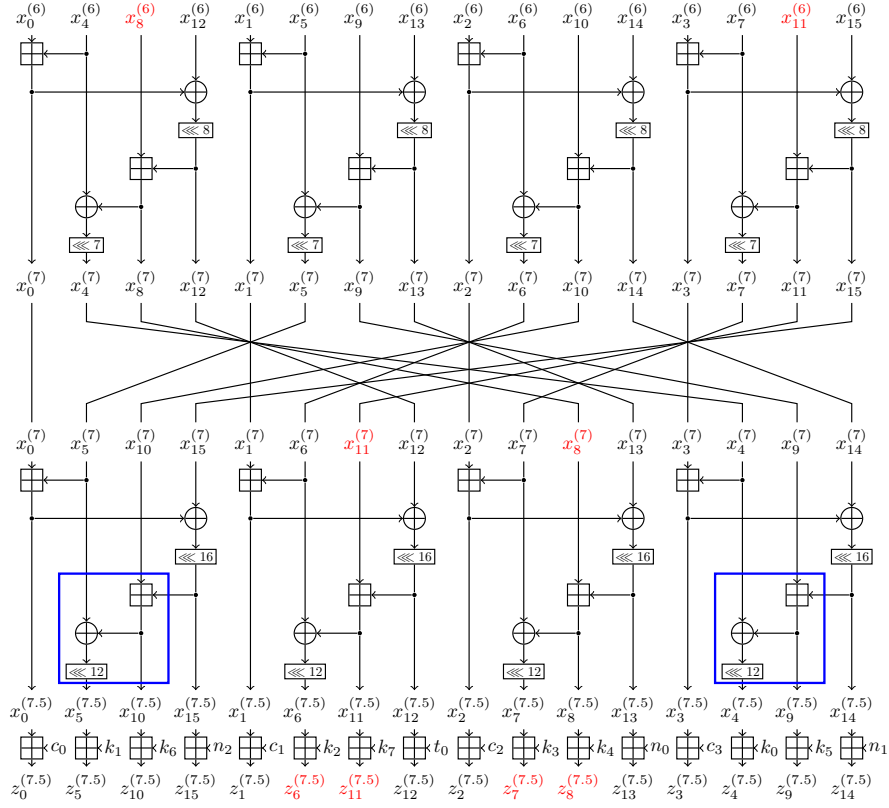


Fig. 10: Last One Round of ChaCha7.5

C. Attacks against ChaCha7.5

In this section, we propose an attack on ChaCha7.5, where "0.5 round" refers to the first one out of two updates applied to the sixteen words within the round function. And the last one round of ChaCha7.5 is illustrated in Fig. 10.

In the attack on ChaCha7.5, a new DL distinguisher of 3.5-round ChaCha is used, which consists of 1-round differential and 2.5-round DL approximation. The 1-round differential in the first round can be represented over one *quarterround* function $\Delta''_{in} = ([], [], [], [22]) \rightarrow \Delta''_m = ([16], [1, 13, 21, 25], [6, 14, 26], [14, 26])$, where Δ''_{in} is an input difference with one active input bits and Δ''_m is an output difference with ten output bits. For example, when the input difference is imposed on the first column and no difference is on other three columns in the first round, we have $(\Delta X_0^{(0)}, \Delta X_4^{(0)}, \Delta X_8^{(0)}, \Delta X_{12}^{(0)}) = \Delta''_{in} \rightarrow (\Delta X_0^{(1)}, \Delta X_4^{(1)}, \Delta X_8^{(1)}, \Delta X_{12}^{(1)}) = \Delta''_m$ with probability of 2^{-5} . Then with Δ''_m as input difference, 2.5-round DL approximation is found at the 3.5-round, that is $\Pr[(\Delta X_0^{(1)}, \Delta X_4^{(1)}, \Delta X_8^{(1)}, \Delta X_{12}^{(1)}) = \Delta'_m \rightarrow \Delta X_2^{(3.5)}[0]] = \frac{1}{2}(1 + \varepsilon_d)$. By the experiment with 2^{30} random samples, the correlation ε_d is evaluated to $2^{-9.4}$. Combining the 2.5-round DL approximation with the 1-round differential, a 3.5-round DL distinguisher is obtained, which is summarized in Table IX. Compared with the one in Table III, the new 3.5-round DL distinguisher has a lower correlation. However, the distinguisher involves fewer PNBS of key at ID column in the first round, allowing for a reduction in the computational cost of obtaining a right pair of the differential in the first round which will be explained later.

TABLE IX: The 3.5-round DL approximation used in ChaCha7.5.

ID	OD	$ \varepsilon_d $
$\Delta X_{12}^{(0)}[22]$ satisfying $\Delta''_{in} \rightarrow \Delta''_m$	$\Delta X_2^{(3.5)}[0]$	$2^{-9.4}$

In the *primary-PNBs-search* phase, the syncopation technique is applied in the last round of ChaCha7.5, which is these words labeled by red in Fig. 10. The CPNB sets with different sizes are searched by using Algorithm 2. By minimizing the time complexity, a set of 18 CPNBs is selected, that is $\{103, 127, 87, 86, 104, 120, 105, 156, 155, 106, 70, 157, 95, 121, 71, 122, 159, 158\}$. For the set of 18 CPNBs, the corresponding 4 profitable syncopated segments and conditions are summarized in Table X, where we ignore the carry bits after syncopated segments. In the experiment of 2^{10} random keys and $2^{17.3}$ IVs for each key, the conditional correlation is theoretically estimated as $\epsilon'_a = 2^{-1.2}$. Therefore, the conditional backward correlation is $\varepsilon_a = 2^{-2.4}$, and with $n = 18, \theta_1 = 4$ and $\alpha = 0.1$, the time and data complexities are $T = 2^{265.2}$ and $N = 2^{35.2}$ respectively. The attack is

invalid since its time complexity is beyond brute attack 2^{256} . However, as shown in the following, in virtue of new techniques of PNE-based framework in Sect. V, we can further refine the attack to make it effective.

TABLE X: Syncopated segments of 18 CPNBs.

syncopated segment $\leftarrow$ condition	(i, j_1, j_2)
$x_6^{(7.5)}[j_2 : j_1] \leftarrow k_2[i] \neq z_6^{(7.5)}[i]$	$\{(8, 9, 21), (24, 25, 30)\}$
$x_7^{(7.5)}[j_2 : j_1] \leftarrow k_3[i] \neq z_7^{(7.5)}[i]$	$\{(11, 12, 23), (\textcolor{red}{27}, \textcolor{red}{28}, \textcolor{red}{30})\}$

In the *advanced-PNEs-search* step, we apply the carry-preserving technique and new key guessing strategy at the 0-th and 3-th columns (no PNB is on the two diagonals in the *primary-PNBs-search* phase), which are identified by blue boxes in Fig. 10. By running Algorithm 4, we find 14 new PNEs utilizing the carry-preserving technique. The 14 PNEs are $\{191, 192, 193, 199, 200, 207, 208, 211, 212, 219, 220, 221, 222, 223\}$ from key words k_5 and k_6 , that is the guessed 114 = 128 – 14 expression of secret key are $\{(k_1 \oplus (k_6 \lll 12))[i] \mid i \in \{0, 1, \dots, 31\}\}$, $\{k_6[i] \mid i \in \{0, 1, \dots, 31\} \setminus \{0, 1, 7, 8, 15, 16, 19, 20, 27, 28, 29, 30, 31\}\}$, and $\{(k_0 \oplus (k_5 \lll 12))[i] \mid i \in \{0, 1, \dots, 31\}\}$, $\{k_5[i] \mid i \in \{0, 1, \dots, 31\} \setminus \{31\}\}$. Therefore, the 14 key bits $\{11, 32, 39, 40, 41, 42, 43, 44, 45, 51, 52, 59, 60, 63\}$ from key words k_0 and k_1 , are also unknown. Correspondingly, there are 25 conditions, as summarized in Table XI. In addition, we found that there is no PNB on key word k_7 . So, to obtain more PNB/PNE at k_7 , we implement a hybrid use of the syncopation and carry-preserving techniques at the 1-th column. Precisely, two extra PNBs $\{224, 255\}$ from k_7 are obtained, i.e. $k_7[0]$ and $k_7[31]$, instead, $k_2[11] \oplus k_7[31]$ and $k_2[12] \oplus k_7[0]$ are guessed with imposing the three conditions in Table XII. The two key bits $\{75, 76\}$ from k_2 are also unknown, i.e. $k_2[11]$ and $k_2[12]$. Note that the other key bits of k_2 and k_7 are guessed as done in the syncopation technique.

TABLE XI: Conditions of 14 PNEs for the carry-preserving technique.

conditions	preserving carry bits
$z_{10}^{(7.5)}[i] \neq \text{Carry}'(z_{10}^{(7.5)}, k_6)[i], i \in \{0, 1, 7, 8, 15, 16, 19, 20, 27, 28, 29, 30\}$	$\text{Carry}'(z_{10}^{(7.5)}, k_6)[i + 1]$
$z_5^{(7.4)}[i + 12] \neq \text{Carry}'(z_5^{(7.5)}, k_1)[i + 12], i \in \{0, 1, 7, 8, 15, 16, 20, 27, 28, 29, 30, 31\}$	$\text{Carry}'(z_5^{(7.5)}, k_1)[i + 13]$
$z_4^{(7.5)}[i + 12] \neq \text{Carry}'(z_4^{(7.5)}, k_0)[i + 12], i \in \{31\}$	$\text{Carry}'(z_4^{(7.5)}, k_0)[i + 13]$

TABLE XII: Three conditions for the carry-preserving technique.

conditions	preserving carry bits
$z_{11}^{(7.5)}[i] \neq \text{Carry}'(z_{11}^{(7.5)}, k_7)[i], i \in \{0\}$	$\text{Carry}'(z_{11}^{(7.5)}, k_7)[i + 1]$
$z_6^{(7.5)}[i + 12] \neq \text{Carry}'(z_6^{(7.5)}, k_2)[i + 12], i \in \{0, 31\}$	$\text{Carry}'(z_6^{(7.5)}, k_2)[i + 13]$

Before the online key-recovery attack, we need to pre-compute and store the right pairs of the differential $\Delta''_{in} \rightarrow \Delta''_m$ in the first round for all the possibilities of guessed key. Note that only one column is involved in the propagation of the differential, that is $(\Delta X_0^{(0)}, \Delta X_4^{(0)}, \Delta X_8^{(0)}, \Delta X_{12}^{(0)}) = \Delta''_{in} \rightarrow (\Delta X_0^{(1)}, \Delta X_4^{(1)}, \Delta X_8^{(1)}, \Delta X_{12}^{(1)}) = \Delta''_m$ where $(X_0^{(0)}, X_4^{(0)}, X_8^{(0)}, X_{12}^{(0)}) = (c_0, k_0, k_4, iv_0)$. There are totally six non-guessed and unknown bits of k_0 and k_4 in ID column, i.e., $\{11, 155, 156, 157, 158, 159\}$. With the technique of Definition 2 introduced in Section III-B, we need to find a favorable IV and keep it in the pre-computed list for every exploitable key with respect to the subspace $K_{mem} = \{11, 155, 156, 157, 158, 159\}$. In the experiments with 2^{12} random keys, the probability of exploitable keys is evaluated to $p_{exp} = 76\%$ among all keys. Since the dimension of $\mathbb{S}_{K_{mem}}$ is $64 - 6 = 58$, we should construct a pre-computed list of approximately $p_{exp} \times 2^{58} \approx 2^{57.6}$ possible exploitable keys and their favorable IVs.

Analysis of complexities. We remove one inessential condition from four conditions in the syncopation technique, which is labeled by red in Table X. By the experiment with 2^{10} random keys and $2^{18.3}$ IVs for each key, the correlation ϵ'_a is theoretically estimated as $2^{-2.3}$. Therefore, the conditional backward correlation is $2^{-4.6}$, and with $n = 34$, and $\alpha = 1.3$, an key-recovery attack is obtained with time complexity of $T = 2^{254.6}$ and data complexity of at most $2^{31} \times N = 2^{31} \times 2^{32.2+6+56} = 2^{125.2}$. The corresponding memory complexity is $2^{122.2}$.

D. Related-key Attack on ChaCha8

In this section, we evaluated the security of ChaCha8 in the related-key scenario, which is one out of three original ciphers in the ChaCha family. Our results show that ChaCha8 fails to achieve the 256-bit (length of key) security if one pair of keys with a fixed difference is permitted in usage.

TABLE XIII: Differential characteristic of first round which has the following difference in one column and zeros in other three columns.

i	$\Delta x_a^{(i)}$	$\Delta x_b^{(i)}$	$\Delta x_c^{(i)}$	$\Delta x_d^{(i)}$
0	00000000	02000000	02002020	02200000
0.5	02000000	02000000	02002000	00000020
1	00000000	00000000	02000000	00002000

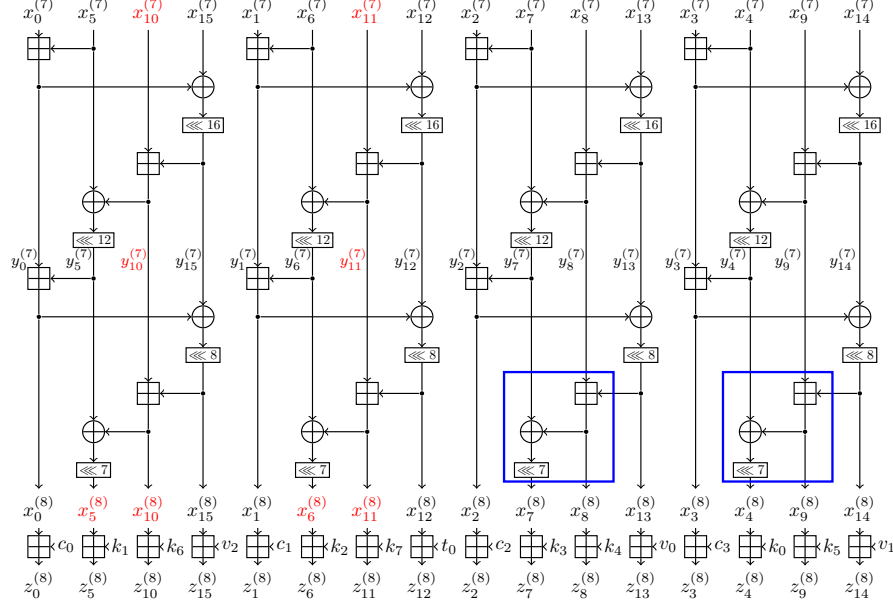


Fig. 11: Last One Round of ChaCha8

1) *Searching Forward differential*: First, to launch an attack, a differential-linear (DL for short) approximation should be found in forward direction. For this, a semi-automatic tool has been developed to search differential-linear distinguishers for ChaCha. There are two steps in the tool, consisting of the top-differential MILP searching and bottom-DL sampling experiment. Note that the input difference can be both imposed on key and IV in related-key scenario. Generally, to construct DL distinguishers with four rounds, in the first round, the differential characteristic is searched by MILP model, and then combined with a feasible DL distinguisher in the following three rounds. For the differential in the first round, we minimize the Hamming weight of output difference to have a DL distinguisher with better correlation in following rounds. The minimum of Hamming weight one is achieved, but with very small probability of 2^{-47} . So the Hamming weight of the output differential is restricted into two in the first round. In [MIM22], Miyashita et al. comprehensively analyzed the PNBs for all possible single-bit truncated output differences. Their results show that more PNBs are identified at the least significant bits of words in the internal state. So, to ensure that the DL distinguishers have enough PNBs in backward approximation, we considered the output mask $\Delta X_i^{(4)}[0]$ for $i = 0, 1, \dots, 15$ in the search tool.

As a result, we found a new differential-linear (DL) distinguisher of four rounds, consisting of differential in the first round and DL approximation with three rounds. The probability of differential in the first round is 2^{-7} , which is given in Table XIII. Beginning from the output difference of Table XIII in the first round, the DL approximation is at $\Delta X_1^{(4)}[0]$ of the forth round and its correlation of $0.0128 \approx 2^{-6.3}$ is observed with 2^{25} random sampling in experiments. We have conducted a experiment to valid the differential characteristic in Table XIII, where 2^{20} IVs and 2^{10} random keys are used. For about 25% keys, called as *weak keys*, there exists an IV satisfying the given differential by using about random 2^5 IVs. Since the DL distinguisher is invariant on four columns, and on average there is one columns satisfy the requirement of *weak keys*. Therefore, the attack should first be repeated by varying 2^5 random IVs, and then change the input difference on four columns, totally 2^7 iterations.

2) *Backward Approximation*: Given the forward differential with four rounds in the above section, it is needed to go back four rounds to attack ChaCha8. The last round of ChaCha8 is illustrated in Fig. 11. In the *primary-PNBs-search* phase, the syncopation technique is applied in the last round of ChaCha8, which is these words labeled by red in Fig. 11. With setting a threshold of 2^{-5} for conditional neutrality, we obtained a preliminary shortlisting of 77 possible CPNBs. Among them, the CPNB sets with different sizes are searched by using Algorithm 2. By minimizing the time complexity, a set of 30 CPNBs is selected, that is $\{223, 59, 63, 38, 60, 213, 62, 61, 212, 214, 95, 215, 39, 192, 40, 193, 86, 41, 64, 87, 51, 65, 255, 70, 194, 42, 83, 236, 251, 66\}$. For the set of 30 CPNBs, the corresponding 18 profitable syncopated segments and conditions are summarized in Table XIV, where

we ignore the carry bits after syncopated segments. In Table XIV, we remove five inessential conditions whose coordinates in table are $\{(T0, 1), (T2, 6), (T1, 8), (T1, 9), (T2, 9)\}$. In the experiment of 2^{10} random keys and $2^{17.3}$ IVs for each key, the conditional correlation is theoretically estimated as $\epsilon'_a = 2^{-2.55}$. Therefore, the conditional backward correlation is $\epsilon_a = 2^{-5.1}$, and with $n = 30, \theta_1 = 13$ and $\alpha = 5$, the time and data complexities are $2^7 \times T = 2^{260.9}$ and $2^7 \times N = 2^{60.8}$ respectively. The attack is invalid since its time complexity is beyond brute attack 2^{256} . However, leveraging new techniques of PNE-based framework in Sect. V, we can further refine the attack to make it effective.

TABLE XIV: Syncopated segments of 30 CPNBs, where “-” denotes $w = 0$.

syncopated segment $\leftarrow$ condition	(i, j_1, j_2)
T0: $x_5^{(8)}[j_2 : j_1] \leftarrow k_1[i] \neq z_5^{(8)}[i]$	$\{(11, 12, 18), (20, 21, 26)\}$
T0: $x_6^{(8)}[j_2 : j_1] \leftarrow k_2[i] \neq z_6^{(8)}[i]$	$\{(3, 4, 5), (7, 8, 18), (20, 21, 21), (24, 25, 30)\}$
T0: $x_{10}^{(8)}[j_2 : j_1] \leftarrow k_6[i] \neq z_{10}^{(8)}[i]$ T1: $x_{10}^{(7.5)}[j_2 : j_1] \leftarrow k_6[i] \neq (z_{10}^{(8)} \boxplus x_{15}^{(8)})[i]$ T2: $x_{10}^{(7)}[j_2 : j_1] \leftarrow k_6[i] \neq (z_{10}^{(8)} \boxplus x_{15}^{(8)} \boxplus x_{15}^{(7.5)})[i]$	$\{(3, 4, 19), (24, 25, 30)\}$
T0: $x_{11}^{(8)}[j_2 : j_1] \leftarrow k_7[i] \neq z_{11}^{(8)}[i]$ T1: $x_{11}^{(7.5)}[j_2 : j_1] \leftarrow k_7[i] \neq (z_{11}^{(8)} \boxplus x_{12}^{(8)})[i]$ T2: $x_{11}^{(7)}[j_2 : j_1] \leftarrow k_7[i] \neq (z_{11}^{(8)} \boxplus x_{12}^{(8)} \boxplus x_{12}^{(7.5)})[i]$	$\{(13, 14, 26), (28, 29, 30)\}$

In the *advanced-PNEs-search* step, we apply the carry-preserving technique and new key guessing strategy at the 2-th and 3-th columns, which are identified by blue boxes in Fig. 11. By running Algorithm 4, we find 10 new PNEs utilizing the carry-preserving technique. The 10 PNEs are $\{148, 159, 172, 173, 186, 187, 188, 189, 190, 191\}$, from key words k_4 and k_5 , that is the guessed $118 = 128 - 10$ expression of secret key are $\{(k_3 \oplus (k_4 \lll 7))[i] \mid i \in \{0, 1, \dots, 31\}\}$, $\{k_4[i] \mid i \in \{0, 1, \dots, 31\} \setminus \{20, 31\}\}$, and $\{(k_0 \oplus (k_5 \lll 7))[i] \mid i \in \{0, 1, \dots, 31\}\}$, $\{k_5[i] \mid i \in \{0, 1, \dots, 31\} \setminus \{12, 13, 26, 27, 28, 29, 30, 31\}\}$. Therefore, the 10 PNEs $\{1, 2, 3, 4, 5, 6, 19, 20, 102, 123\}$, from key words k_0 and k_3 , are also unknown. Correspondingly, there are $36 = (10 \times 2 - 2) \times 2$ conditions, as summarized in Table XV.

TABLE XV: Conditions of 10 PNEs for the carry-preserving technique.

conditions	preserving carry bits
$z_8^{(8)}[i] \neq \text{Carry}'(z_8^{(8)}, k_4)[i], i \in \{20\}$	$\text{Carry}'(z_8^{(8)}, k_4)[i + 1]$
$z_7^{(8)}[i + 7] \neq \text{Carry}'(z_7^{(8)}, k_3)[i + 7], i \in \{20, 31\}$	$\text{Carry}'(z_7^{(8)}, k_3)[i + 8]$
$z_9^{(8)}[i] \neq \text{Carry}'(z_9^{(8)}, k_5)[i],$ $i \in \{12, 13, 26, 27, 28, 29, 30\}$	$\text{Carry}'(z_9^{(8)}, k_5)[i + 1]$
$z_4^{(8)}[i + 7] \neq \text{Carry}'(z_4^{(8)}, k_0)[i + 7],$ $i \in \{12, 13, 26, 27, 28, 29, 30, 31\}$	$\text{Carry}'(z_4^{(8)}, k_0)[i + 8]$

Analysis of complexities. In the experiment of 2^{10} random keys and $2^{17.3}$ IVs for each key, the conditional correlation is theoretically estimated as $\epsilon'_a = 2^{-2.95}$. The conditional backward correlation is $\epsilon_a = 2^{-5.9}$. Therefore, we obtain a key-recovery attack on ChaCha8 with time complexity $2^7 \times T = 2^{253.3}$ and data complexity $2^7 \times N = 2^{99.2}$ in the related-key setting where one pair of keys with a fixed difference is used that is (k_i, k_{i+4}) satisfying $(\Delta x_b^{(0)}, \Delta x_c^{(0)})$ in Table XIII for $i \in \{0, 1, 2, 3\}$. The corresponding memory complexity is $2^{79.2}$.

VIII. APPLICATIONS: ATTACKS ON 128-BIT CHACHA

In this section, we present key-recovery attacks on 128-bit ChaCha, specifically for ChaCha7 $\lll$ and ChaCha6.5. The attacks on 128-bit ChaCha are similar to these on 256-bit ChaCha, and the differential characteristic in Table III is used in the attacks.

A. Attack on ChaCha7 $\lll$

For higher rounds of 128-bit ChaCha, ChaCha7 $\lll$ is defined as the variant of 7 rounds of 128-bit ChaCha without the last rotation. The attacks on 128-bit ChaCha are similar to these on 256-bit ChaCha. Next, we list the main parameters in the attacks on ChaCha7 $\lll$. With setting a threshold of 2^{-5} for conditional neutrality, we obtained a preliminary shortlisting of 85 possible CPNBs. Among them, the CPNB sets with different sizes are searched by using Algorithm 2. By minimizing the time complexity under the condition that the data complexity does not exceed the amount of available IVs, a set of 38 CPNBs is selected, that is $\{31, 63, 91, 92, 93, 94, 95, 127, 79, 80, 116, 117, 118, 27, 62, 28, 29, 119, 96, 64, 65, 40, 97, 83, 107, 30, 71, 84, 41, 66, 72, 98, 18, 67, 42, 85, 99, 32\}$. For the set of 38 CPNBs, the corresponding 40 profitable syncopated segments, and conditions are summarized in Table XVI, where we ignore the carry bits after syncopated segments. In Table XVI, we remove 13 inessential conditions whose coordinates in table are $\{(T3, 0), (T2, 1), (T3, 1), (T3, 2), (T0, 4), (T1, 4), (T2, 4), (T2, 5), (T3, 5), (T3, 6), (T2, 9), (T3, 8),$

(T3, 9)}. In the experiment of 2^{10} random keys and $2^{17.3}$ IVs for each key, the conditional correlation is theoretically estimated as $\epsilon'_a = 2^{-2.1}$. Therefore, the conditional backward correlation is $\epsilon_a = 2^{-4.3}$, and with $n = 38, \theta_1 = 27$ and $\alpha = 11$, the time and data complexities are $2^5 \times T = 2^{125.9}$ and $2^5 \times N = 2^{89.8}$ respectively. The corresponding memory complexity is $2^{57.8}$.

TABLE XVI: For ChaCha7 $\lll$, the syncopated segments of 38 CPNBs.

syncopated segment $\leftarrow$ condition	(i, j_1, j_2)
T0: $x_4^{(7\lll)}[j_2 : j_1] \leftarrow k_0[i] \neq z_4^{(7\lll)}[i]$ T1: $x_8^{(7\lll)}[j_2 : j_1] \leftarrow k_0[i] \neq z_8^{(7\lll)}[i]$ T2: $x_8^{(6.5)}[j_2 : j_1] \leftarrow k_0[i] \neq (z_8^{(7\lll)} \boxplus x_{12}^{(7\lll)})[i]$ T3: $x_8^{(6)}[j_2 : j_1] \leftarrow k_0[i] \neq (z_8^{(7\lll)} \boxplus x_{12}^{(7\lll)} \boxplus x_{12}^{(6.5)})[i]$	$\{(19, 20, 26)\}$
T0: $x_5^{(7\lll)}[j_2 : j_1] \leftarrow k_1[i] \neq z_5^{(7\lll)}[i]$ T1: $x_9^{(7\lll)}[j_2 : j_1] \leftarrow k_1[i] \neq z_9^{(7\lll)}[i]$ T2: $x_9^{(6.5)}[j_2 : j_1] \leftarrow k_1[i] \neq (z_9^{(7\lll)} \boxplus x_{13}^{(7\lll)})[i]$ T3: $x_9^{(6)}[j_2 : j_1] \leftarrow k_1[i] \neq (z_9^{(7\lll)} \boxplus x_{13}^{(7\lll)} \boxplus x_{13}^{(6.5)})[i]$	$\{(1, 2, 7), (11, 12, 29)\}$
T0: $x_6^{(7\lll)}[j_2 : j_1] \leftarrow k_2[i] \neq z_6^{(7\lll)}[i]$ T1: $x_{10}^{(7\lll)}[j_2 : j_1] \leftarrow k_2[i] \neq z_{10}^{(7\lll)}[i]$ T2: $x_{10}^{(6.5)}[j_2 : j_1] \leftarrow k_2[i] \neq (z_{10}^{(7\lll)} \boxplus x_{14}^{(7\lll)})[i]$ T3: $x_{10}^{(6)}[j_2 : j_1] \leftarrow k_2[i] \neq (z_{10}^{(7\lll)} \boxplus x_{14}^{(7\lll)} \boxplus x_{14}^{(6.5)})[i]$	$\{(4, 5, 6), (9, 10, 14), (17, 18, 18), (22, 23, 26)\}$
T0: $x_7^{(7\lll)}[j_2 : j_1] \leftarrow k_3[i] \neq z_7^{(7\lll)}[i]$ T1: $x_{11}^{(7\lll)}[j_2 : j_1] \leftarrow k_3[i] \neq z_{11}^{(7\lll)}[i]$ T2: $x_{11}^{(6.5)}[j_2 : j_1] \leftarrow k_3[i] \neq (z_{11}^{(7\lll)} \boxplus x_{15}^{(7\lll)})[i]$ T3: $x_{11}^{(6)}[j_2 : j_1] \leftarrow k_3[i] \neq (z_{11}^{(7\lll)} \boxplus x_{15}^{(7\lll)} \boxplus x_{15}^{(6.5)})[i]$	$\{(4, 5, 10), (12, 13, 19), (24, 25, 30)\}$

Another attack on ChaCha7 $\lll$. Another attack is obtained by combining the list technique of [DGSS22] to find right pairs in the first round. There are six CPNBs in the ID column, that is $\{32, 40, 41, 42, 62, 63\}$. To obtain a high fraction of exploitable keys, only 63, 40 and 41 can be included in the K_{nmem} subset. So we exclude three bits $\{32, 42, 62\}$ in 38 CPNBs and re-evaluated the conditional correlation, that is $\epsilon'_a = 2^{-1.95}$ with 2^{10} random keys and $2^{17.3}$ IVs for each key. Therefore, the conditional backward correlation is $\epsilon_a = 2^{-3.9}$, and with $n = 35, \theta_1 = 25$ and $\alpha = 9$, the time and data complexities are $T = 2^{122.9}$ and $2^{29} \times N = 2^{108.8}$ respectively. The corresponding memory complexity is $2^{83.8}$.

B. Attack on ChaCha6.5

Here we present the details of attack on ChaCha6.5, which improves the previous results in [DGSS22], [DGSS23].

The main parameters in the attack on ChaCha6.5 are as follows. With setting a threshold of 2^{-5} for conditional neutrality, we obtained a preliminary shortlisting of 79 possible CPNBs. Among them, the CPNB sets with different sizes are searched by using Algorithm 2. By minimizing the time complexity under the condition that the data complexity does not exceed the amount of available IVs, a set of 50 CPNBs is selected, that is $\{79, 80, 91, 92, 93, 94, 95, 124, 125, 127, 24, 115, 116, 104, 31, 96, 84, 117, 27, 105, 26, 106, 118, 83, 64, 28, 85, 97, 98, 63, 86, 12, 119, 65, 71, 40, 30, 29, 13, 66, 41, 14, 87, 99, 72, 52, 32, 67, 42, 15\}$. For the set of 50 CPNBs, the corresponding 52 profitable syncopated segments and conditions are summarized in Table XVII, where we ignore the carry bits after syncopated segments. In Table XVII, we remove 20 inessential conditions whose coordinates in table are $\{(T0, 0), (T0, 1), (T1, 0), (T1, 1), (T2, 0), (T2, 1), (T0, 2), (T0, 4), (T1, 2), (T2, 2), (T2, 3), (T2, 4), (T3, 2), (T0, 6), (T1, 5), (T1, 7), (T3, 8), (T1, 11), (T2, 11), (T3, 12)\}$. In the experiment of 2^{10} random keys and $2^{18.3}$ IVs for each key, the conditional correlation is theoretically estimated as $\epsilon'_a = 2^{-2.1}$. Therefore, the conditional backward correlation is $\epsilon_a = 2^{-4.2}$, and with $n = 50, \theta_1 = 32$ and $\alpha = 24$, the time and data complexities are $2^5 \times T = 2^{114.3}$ and $2^5 \times N = 2^{100.3}$ respectively. The corresponding memory complexity is $2^{63.3}$.

IX. EXPERIMENTS OF PRACTICAL ATTACKS

To demonstrate the proposed techniques and verify our attacks, we have implemented experiments on two practical key-recovery attacks against 5 rounds and 6 rounds of ChaCha with 64-bit unknown key bits. Specifically, to make the process of key recovery practical, it is assumed that under the condition that 192 out of 256 key bits have been obtained and known, the attack is performed to recover some of the remaining 64 bits of the secret key. The details of experiments can be found in the following, and the corresponding codes can be found at https://github.com/desert-oasis/chacha_syncopation.git.

Environment of experiments. We have implemented the procedure of new (partial) key-recovery attack with the syncopation technique through C++ programming, that is, Step 1 to 5 of Algorithm 3. All the experiments are conducted on Linux version 6.2.9-arch1-1 with Intel Core i9-13900KF and RAM of 128 GB.

TABLE XVII: For 128-bit ChaCha6.5, the syncopated segments of 50 CPNBs.

syncopated segment $\leftarrow$ condition	(i, j_1, j_2)
T0: $x_4^{(6.5)}[j_2 : j_1] \leftarrow k_0[i] \neq z_4^{(6.5)}[i]$ T1: $x_8^{(6.5)}[j_2 : j_1] \leftarrow k_0[i] \neq z_8^{(6.5)}[i]$ T2: $x_8^{(6)}[j_2 : j_1] \leftarrow k_0[i] \neq (z_8^{(6.5)} \boxplus x_{12}^{(6.5)})[i]$ T3: $x_8^{(5.5)}[j_2 : j_1] \leftarrow k_0[i] \neq (z_8^{(6.5)} \boxplus x_{12}^{(6.5)} \boxplus x_{13}^{(6)})[i]$	$\{(16, 17, 23), (25, -, -)\}$
T0: $x_5^{(6.5)}[j_2 : j_1] \leftarrow k_1[i] \neq z_5^{(6.5)}[i]$ T1: $x_9^{(6.5)}[j_2 : j_1] \leftarrow k_1[i] \neq z_9^{(6.5)}[i]$ T2: $x_9^{(6)}[j_2 : j_1] \leftarrow k_1[i] \neq (z_9^{(6.5)} \boxplus x_{13}^{(6.5)})[i]$ T3: $x_9^{(5.5)}[j_2 : j_1] \leftarrow k_1[i] \neq (z_9^{(6.5)} \boxplus x_{13}^{(6.5)} \boxplus x_{14}^{(6)})[i]$	$\{(1, 2, 7), (11, 12, 19), (21, 22, 30)\}$
T0: $x_6^{(6.5)}[j_2 : j_1] \leftarrow k_2[i] \neq z_6^{(6.5)}[i]$ T1: $x_{10}^{(6.5)}[j_2 : j_1] \leftarrow k_2[i] \neq z_{10}^{(6.5)}[i]$ T2: $x_{10}^{(6)}[j_2 : j_1] \leftarrow k_2[i] \neq (z_{10}^{(6.5)} \boxplus x_{14}^{(6.5)})[i]$ T3: $x_{10}^{(5.5)}[j_2 : j_1] \leftarrow k_2[i] \neq (z_{10}^{(6.5)} \boxplus x_{14}^{(6.5)} \boxplus x_{15}^{(6)})[i]$	$\{(4, 5, 6), (9, 10, 14), (17, 18, 18), (24, 25, 26)\}$
T0: $x_7^{(6.5)}[j_2 : j_1] \leftarrow k_3[i] \neq z_7^{(6.5)}[i]$ T1: $x_{11}^{(6.5)}[j_2 : j_1] \leftarrow k_3[i] \neq z_{11}^{(6.5)}[i]$ T2: $x_{11}^{(6)}[j_2 : j_1] \leftarrow k_3[i] \neq (z_{11}^{(6.5)} \boxplus x_{15}^{(6.5)})[i]$ T3: $x_{11}^{(5.5)}[j_2 : j_1] \leftarrow k_3[i] \neq (z_{11}^{(6.5)} \boxplus x_{15}^{(6.5)} \boxplus x_{12}^{(6)})[i]$	$\{(4, 5, 7), (11, 12, 18), (24, 25, 27), (30, -, -)\}$

a) *Attack on 5-round ChaCha*: For 5-round ChaCha, the attack uses a forward differential with 2.5 rounds, and approximates 2.5 rounds in backward direction with PNBs of 64 unknown key bits. More precisely, the median correlation of forward differential is $|\varepsilon_d| = 0.838$ with input difference at $\Delta x_{13}^{(0)}[6]$ and output difference at $\Delta x_2^{(2.5)}[0]$. In the experiments, k_i ($i \geq 2$) are assumed to be known and only k_0 and k_1 are uncertain. For backward approximation, we find 56 PNBs of k_0 and k_1 . Accordingly, the 8 non-PNBs are $\{k_0[i], i \in \{6, 9, 10, 11, 12, 13, 18\}\}$ and $k_1[6]$. With the syncopation (sync. for short) technique, the conditional backward correlation is experimentally estimated as $|\varepsilon_a| = 0.752$ under the condition that the $\theta_1 = 4$ constraints $k_0[i] \neq z_4^{(5)}[i]$, $i \in \{6, 9, 18\}$ and $k_1[6] \neq z_5^{(5)}[6]$ are satisfied. However, without the syncopation technique, the median correlation of backward approximation is experimentally estimated as $|\varepsilon_a| = 2^{-7.6}$ by fixing 56 PNBs into zeros. Therefore, the backward correlation is significantly amplified by utilizing the syncopation technique. To recover 8 non-PNBs of 64-bit secret key with the syncopation technique, we only need to implement Step 1 to 5 of Algorithm 3. According to the complexity analysis, the time complexity of key-recovery attack is $T = N + 2^8 \times N^*$, data complexity is $N = 2^{2 \times 4} \times N^*$ and memory complexity is $M = 2^4 \times N^*$. The Neyman-Pearson decision theory gives the results about estimating the number of samples N^* required to get the bounds on probabilities of false alarm p_{fa} and non-detection p_{nd} .

It can be shown that $N^* \approx \left(\frac{\sqrt{\alpha \log 4 + 3} \sqrt{1 - (\varepsilon_a \varepsilon_d)^2}}{\varepsilon_a \varepsilon_d} \right)^2$ samples suffices to achieve $p_{nd} = 1.3 \times 10^{-3}$ and $p_{fa} = 2^{-\alpha}$. With $\alpha = 8$, the time complexity is $T = 2^{15.4}$, data complexity is $N = 2^{14.4}$ and memory complexity is $M = 2^{10.4}$. As pointed out in [AFK⁺08], with using median correlation in the above equation, we have a success probability of at least $\frac{1}{2}(1 - p_{nd}) \approx \frac{1}{2}$ for the attack.

Result of experiments of 5-round ChaCha. It takes about 22 seconds to run the C++ program 10000 times. As a result, for 10000 randomly generated key, 7071 of them are successfully recovered 8 guessed key bits with the syncopation techniques, and thus the success probability is about 70.7%. The attacks on 5-round ChaCha are summarized in Table XVIII.

TABLE XVIII: Summary of attacks on 5-round ChaCha with 64 unknown key bits.

Attack method	Analysis	$ \varepsilon_a $	T	N	#Random keys	Success probability
With sync.	Theoretical	-	$2^{15.4}$	$2^{14.4}$	-	≥ 0.5
	Experimental	0.752	2.2×2^{-3} s	$2^{14.4}$	10000	$\frac{7071}{10000} \approx 70.7\%$
Without sync.	Theoretical	-	$2^{29.1}$	$2^{21.1}$	-	≥ 0.5
	Experimental	$2^{-7.6}$	22.5 s	$2^{21.1}$	100	$\frac{16}{100} \approx 16\%$

b) *Attack on 6-round ChaCha*: Concerning 6-round ChaCha, the attack is similar to the one of 5-round ChaCha, except that a forward differential with 3.5 rounds is used, and thus backward approximation consists of 2.5 rounds from the 6-th to 3.5-th round. Specifically, the forward differential takes $\Delta x_{13}^{(0)}[6]$ as the input difference and observes the output difference at $\Delta x_2^{(3.5)}[0]$. As shown in [BLT20], the correlation of the forward characteristic is evaluated as $2^{-8.3}$ under that condition that the output difference after the first round has the minimum Hamming weight 10, which takes about 2^5 iteration to achieve a right pair for the differential.

The 8 non-PNBs are $k_0[6]$ and $\{k_1[i], i \in \{6, 9, 10, 11, 12, 13, 18\}\}$. The conditional backward correlation is experimentally estimated as $|\varepsilon_a| = 0.759$ under the condition that the $\theta_1 = 4$ constraints $k_0[6] \neq z_4^{(6)}[6]$ and $k_1[i] \neq z_5^{(6)}[i]$, $i \in \{6, 9, 18\}$.

According to the complexity analysis, the time complexity of key-recovery attack is $T = 2^5 \times (N + 2^8 \times N^*)$, data complexity is $N = 2^{5+2 \times 4} \times N^*$ and memory complexity is $M = 2^4 \times N^*$, and with $\alpha = 13$, $T = 2^{37.1}$, $N = 2^{36.1}$ and $M = 2^{27.1}$.

Result of experiments of 6-round ChaCha. It takes about 1.6 hours to run once the C++ program with RAM of about 24 GB for the key-recovery attack of 6-round ChaCha. As a result, for 16 randomly generated key, 10 of them are successfully recovered 8 guessed key bits with the syncopation techniques, and thus the success probability among all keys is about 62.5%. In another experiment with 32 randomly generated keys, there are 21 weak keys, among which 15 keys are successfully recovered 8 guessed key bits with the syncopation techniques, and thus the success probability of key-recovery attack in Algorithm 3 is about $\frac{15}{21} \approx 71.4\%$.

X. CONCLUSION

This paper presents PNB-based differential cryptanalysis of ChaCha with *syncopation*. Specifically, we propose the *syncopation* technique and refine the key-recovery attack based on the PNB method with this new technique. In addition, we generalized the PNB-based framework by a new concept called *probabilistic neutral expression* where the *carry-preserving* technique and new key guessing strategy were proposed. The attacks on 6 and 7 rounds of 256-bit ChaCha have been improved using the new techniques. Furthermore, ChaCha7.5⁺ and ChaCha7.5 is defined for getting closer to ChaCha8, and several attacks are presented on ChaCha7.5⁺ and ChaCha7.5. As far as we know, this is the first attack on ChaCha7.5⁺ and ChaCha7.5, which is much more efficient than brute force. In a nutshell, we move a small but important step forward towards the analysis of ChaCha8. This paper also evaluated the security of ChaCha8 in the related-key setting where one pair of keys with a fixed difference is used. Besides, we applied our techniques to analyze the 128-bit ChaCha, leading to an attack on 7 rounds without the last rotation. It is interesting and worth considering how to apply the new techniques to the other ARX ciphers, such as Salsa and Chaskey.

REFERENCES

- [AFK⁺08] Jean-Philippe Aumasson, Simon Fischer, Shahram Khazaei, Willi Meier, and Christian Rechberger. New features of latin dances: Analysis of salsa, chacha, and rumba. In Kaisa Nyberg, editor, *Fast Software Encryption, 15th International Workshop, FSE 2008, Lausanne, Switzerland, February 10-13, 2008, Revised Selected Papers*, volume 5086 of *Lecture Notes in Computer Science*, pages 470–488. Springer, 2008.
- [Aum19] Jean-Philippe Aumasson. Too much crypto. *IACR Cryptol. ePrint Arch.*, page 1492, 2019.
- [BC14] Eli Biham and Yaniv Carmeli. An improvement of linear cryptanalysis with addition operations with applications to FEAL-8X. In Antoine Joux and Amr M. Youssef, editors, *Selected Areas in Cryptography - SAC 2014 - 21st International Conference, Montreal, QC, Canada, August 14-15, 2014, Revised Selected Papers*, volume 8781 of *Lecture Notes in Computer Science*, pages 59–76. Springer, 2014.
- [Ber05] Daniel J. Bernstein. The poly1305-aes message-authentication code. In Henri Gilbert and Helena Handschuh, editors, *Fast Software Encryption: 12th International Workshop, FSE 2005, Paris, France, February 21-23, 2005, Revised Selected Papers*, volume 3557 of *Lecture Notes in Computer Science*, pages 32–49. Springer, 2005.
- [Ber08a] Daniel J. Bernstein. Chacha, a variant of salsa20, 2008. <https://cr.yp.to/chacha/chacha-20080128.pdf>.
- [Ber08b] Daniel J. Bernstein. The salsa20 family of stream ciphers. In Matthew J. B. Robshaw and Olivier Billet, editors, *New Stream Cipher Designs - The eSTREAM Finalists*, volume 4986 of *Lecture Notes in Computer Science*, pages 84–97. Springer, 2008.
- [BG⁺23] Emanuele Bellini, David G rault, Juan Grados, Rusydi H. Makarim, and Thomas Peyrin. Boosting differential-linear cryptanalysis of chacha7 with MILP. *IACR Trans. Symmetric Cryptol.*, 2023(2):189–223, 2023.
- [BLT20] Christof Beierle, Gregor Leander, and Yosuke Todo. Improved differential-linear attacks with applications to ARX ciphers. In Daniele Micciancio and Thomas Ristenpart, editors, *Advances in Cryptology - CRYPTO 2020 - 40th Annual International Cryptology Conference, CRYPTO 2020, Santa Barbara, CA, USA, August 17-21, 2020, Proceedings, Part III*, volume 12172 of *Lecture Notes in Computer Science*, pages 329–358. Springer, 2020.
- [BS90] Eli Biham and Adi Shamir. Differential cryptanalysis of des-like cryptosystems. In Alfred Menezes and Scott A. Vanstone, editors, *Advances in Cryptology - CRYPTO '90, 10th Annual International Cryptology Conference, Santa Barbara, California, USA, August 11-15, 1990, Proceedings*, volume 537 of *Lecture Notes in Computer Science*, pages 2–21. Springer, 1990.
- [CM16] Arka Rai Choudhuri and Subhamoy Maitra. Significantly improved multi-bit differentials for reduced round salsa and chacha. *IACR Trans. Symmetric Cryptol.*, 2016(2):261–287, 2016.
- [CN21a] Murilo Coutinho and T. C. Souza Neto. Improved linear approximations to arx ciphers and attacks against chacha. *Cryptology ePrint Archive*, Paper 2021/224, 2021. <https://eprint.iacr.org/2021/224>.
- [CN21b] Murilo Coutinho and Tertuliano C. Souza Neto. Improved linear approximations to ARX ciphers and attacks against chacha. In Anne Canteaut and Fran ois-Xavier Standaert, editors, *Advances in Cryptology - EUROCRYPT 2021 - 40th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zagreb, Croatia, October 17-21, 2021, Proceedings, Part I*, volume 12696 of *Lecture Notes in Computer Science*, pages 711–740. Springer, 2021.
- [CPV⁺22] Murilo Coutinho, Iago Passos, Juan C. Grados V squez, F bio L cio Lopes de Mendon a, Rafael Timteo de Sousa, and F bio Borges. Latin dances reloaded: Improved cryptanalysis against salsa and chacha, and the proposal of forr . In Shweta Agrawal and Dongdai Lin, editors, *Advances in Cryptology - ASIACRYPT 2022 - 28th International Conference on the Theory and Application of Cryptology and Information Security, Taipei, Taiwan, December 5-9, 2022, Proceedings, Part I*, volume 13791 of *Lecture Notes in Computer Science*, pages 256–286. Springer, 2022.
- [DDSM22] Sabyasachi Dey, Chandan Dey, Santanu Sarkar, and Willi Meier. Revisiting cryptanalysis on chacha from crypto 2020 and eurocrypt 2021. *IEEE Trans. Inf. Theory*, 68(9):6114–6133, 2022.
- [Dey24] Sabyasachi Dey. Advancing the idea of probabilistic neutral bits: First key recovery attack on 7.5 round chacha. *IEEE Trans. Inf. Theory*, 70(8):6091–6106, 2024.
- [DGM23] Sabyasachi Dey, Hirendra Kumar Garai, and Subhamoy Maitra. Cryptanalysis of reduced round chacha- new attack and deeper analysis. *Cryptology ePrint Archive*, Paper 2023/134, 2023. <https://eprint.iacr.org/2023/134>.
- [DGSS22] Sabyasachi Dey, Hirendra Kumar Garai, Santanu Sarkar, and Nitin Kumar Sharma. Revamped differential-linear cryptanalysis on reduced round chacha. In Orr Dunkelman and Stefan Dziembowski, editors, *Advances in Cryptology - EUROCRYPT 2022 - 41st Annual International Conference on the Theory and Applications of Cryptographic Techniques, Trondheim, Norway, May 30 - June 3, 2022, Proceedings, Part III*, volume 13277 of *Lecture Notes in Computer Science*, pages 86–114. Springer, 2022.

- [DGSS23] Sabyasachi Dey, Hirendra Kumar Garai, Santanu Sarkar, and Nitin Kumar Sharma. Enhanced differential-linear attacks on reduced round chacha. *IEEE Trans. Inf. Theory*, 69(8):5318–5336, 2023.
- [DS17] Sabyasachi Dey and Santanu Sarkar. Improved analysis for reduced round salsa and chacha. *Discret. Appl. Math.*, 227:58–69, 2017.
- [DS20] Sabyasachi Dey and Santanu Sarkar. Proving the biases of salsa and chacha in differential attack. *Des. Codes Cryptogr.*, 88(9):1827–1856, 2020.
- [eST] eSTREAM. The ecrypt stream cipher project. <https://www.ecrypt.eu.org/stream/>.
- [IAN] IANIX. Chacha usage & deployment. <https://ianix.com/pub/chacha-deployment.html>.
- [LCM⁺16] Adam Langley, Wan-Teh Chang, Nikos Mavrogiannopoulos, Joachim Strömbergson, and Simon Josefsson. Chacha20-poly1305 cipher suites for transport layer security (TLS). *RFC*, 7905:1–8, 2016.
- [Leu16] Gaëtan Leurent. Improved differential-linear cryptanalysis of 7-round chaskey with partitioning. In Marc Fischlin and Jean-Sébastien Coron, editors, *Advances in Cryptology - EUROCRYPT 2016 - 35th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Vienna, Austria, May 8-12, 2016, Proceedings, Part I*, volume 9665 of *Lecture Notes in Computer Science*, pages 344–371. Springer, 2016.
- [LH94] Susan K. Langford and Martin E. Hellman. Differential-linear cryptanalysis. In Yvo Desmedt, editor, *Advances in Cryptology - CRYPTO '94, 14th Annual International Cryptology Conference, Santa Barbara, California, USA, August 21-25, 1994, Proceedings*, volume 839 of *Lecture Notes in Computer Science*, pages 17–25. Springer, 1994.
- [Mai16] Subhamoy Maitra. Chosen IV cryptanalysis on reduced round chacha and salsa. *Discret. Appl. Math.*, 208:88–97, 2016.
- [Mat93] Mitsuru Matsui. Linear cryptanalysis method for DES cipher. In Tor Helleseth, editor, *Advances in Cryptology - EUROCRYPT '93, Workshop on the Theory and Application of of Cryptographic Techniques, Lofthus, Norway, May 23-27, 1993, Proceedings*, volume 765 of *Lecture Notes in Computer Science*, pages 386–397. Springer, 1993.
- [MIM22] Shotaro Miyashita, Ryoma Ito, and Atsuko Miyaji. Pnb-focused differential cryptanalysis of chacha stream cipher. In Khoa Nguyen, Guomin Yang, Fuchun Guo, and Willy Susilo, editors, *Information Security and Privacy - 27th Australasian Conference, ACISP 2022, Wollongong, NSW, Australia, November 28-30, 2022, Proceedings*, volume 13494 of *Lecture Notes in Computer Science*, pages 46–66. Springer, 2022.
- [Nir15] Yoav Nir. Chacha20, poly1305, and their use in the internet key exchange protocol (IKE) and ipsec. *RFC*, 7634:1–13, 2015.
- [SZFW12] Zhenqing Shi, Bin Zhang, Dengguo Feng, and Wenling Wu. Improved key recovery attacks on reduced-round salsa20 and chacha. In Taekyoung Kwon, Mun-Kyu Lee, and Daesung Kwon, editors, *Information Security and Cryptology - ICISC 2012 - 15th International Conference, Seoul, Korea, November 28-30, 2012, Revised Selected Papers*, volume 7839 of *Lecture Notes in Computer Science*, pages 337–351. Springer, 2012.
- [WLHL23] Shichang Wang, Meicheng Liu, Shiqi Hou, and Dongdai Lin. Moving a step of chacha in syncopated rhythm. In Helena Handschuh and Anna Lysyanskaya, editors, *Advances in Cryptology - CRYPTO 2023 - 43rd Annual International Cryptology Conference, CRYPTO 2023, Santa Barbara, CA, USA, August 20-24, 2023, Proceedings, Part III*, volume 14083 of *Lecture Notes in Computer Science*, pages 273–304. Springer, 2023.
- [XXTQ24] Zhichao Xu, Hong Xu, Lin Tan, and Wenfeng Qi. Differential-linear cryptanalysis of reduced round chacha. *IACR Trans. Symmetric Cryptol.*, 2024(2):166–189, 2024.

APPENDIX A
DISCUSSION OF [DGSS22], [DGSS23], [BGG⁺23] AND [DEY24]

In this section, we discuss the complexity of recent attacks in [DGSS22], [DGSS23], [BGG⁺23] and [Dey24]. We re-estimated their attack's complexities and, for comparison, summarized the results in Table XIX.

TABLE XIX: Revaluations of Complexities of Attacks on ChaCha [DGSS22], [DGSS23], [BGG⁺23] and [Dey24]

Cipher	Time [†]	Memory [†]	Data	Reference
256-bit ChaCha7	$2^{221.95}$	$2^{47.31}$	$2^{90.2}$	[DGSS22]
	$2^{221.95}$	$2^{47.31}$	$2^{75.89}$	Revaluation of [DGSS22]
	$2^{218.92}$	$2^{47.31}$	$2^{87.18}$	[DGSS23]
	$2^{218.92}$	$2^{47.31}$	$2^{72.89}$	Revaluation of [DGSS23]
	$2^{206.8}$	-	$2^{110.81}$	[BGG ⁺ 23]
	$2^{213.8}$	-	$2^{117.81}$	Revaluation of [BGG ⁺ 23]
256-bit ChaCha7.25	$2^{192.89}$ E	2^{68} Elements	$2^{93.79}$	[Dey24]
	$2^{192.89}$ E + $2^{107} \times 2^{93.79}$ MA	$2^{68} \times 2^{93.79}$ Bits	$2^{100.79\dagger}$	Revaluation of [Dey24]
	$2^{244.85}$	$2^{49.49}$	$2^{93.24}$	[DGSS23]
	$2^{244.85}$	$2^{49.49}$	$2^{74.75}$	Revaluation of [DGSS23]
	$2^{238.34}$	-	$2^{122.34}$	[BGG ⁺ 23]
	$2^{245.34}$	-	$2^{122.34}$	Revaluation of [BGG ⁺ 23]
	$2^{228.24}$ E	2^{91} Elements	$2^{100.90}$	[Dey24]
	$2^{228.24}$ E + $2^{144*} \times 2^{93.90}$ MA	$2^{91} \times 2^{93.90}$ Bits	$2^{100.90}$	Revaluation of [Dey24]

[†] E and MA is short for encryption and memory access respectively. The time complexity 2^t E denotes that the attack takes 2^t times of encryption/decryption operations. The time complexity 2^t E + $2^a \times 2^b$ MA denotes that the attack takes 2^t times of encryption/decryption operations and 2^a times of memory accesses for 2^b -bit elements from the table of size $2^x \times 2^b$ bits which consists of 2^x elements and each element has 2^b bits. That is the memory complexity is $2^x \times 2^b$ bits. The notation - denotes that no memory information is provided in the paper.

[‡] The data complexity should also be multiplied by $\frac{1}{p} = 2^7$, as the attack needs to be repeated $\frac{1}{p} = 2^7$ times, which was overlooked in [Dey24].

* The number 2^a is not provided in [Dey24], and it is estimated as around 2^{t+10-b} in this paper since it is stated in [Dey24] that 2^{a+b} bit operations is the dominant complexity and converted into 2^{a+b-10} encryptions.

A. Discussion of [DGSS22], [DGSS23]

For ChaCha7, with the technique of Definition 2 in Section III-B, Dey [DGSS22] constructed a subset K_{nmem} with dimension 18, and evaluated that there are approximately 62% exploitable keys among all the keys, leading to a pre-computed list of about $0.62 \times 2^{64-18} = 2^{45.31}$ possible exploitable keys along with their favorable IVs. With a set of 79 PNBs of backward correlation $\varepsilon_a = 0.00057$, when $\alpha = 38.8$, this gives $N = 2^{44.89}$, and the time complexity is $2^{221.95}$. Since there are $2^{45.31}$ exploitable keys in the ID column, they estimated the overall data complexity as $2^{45.31} \times N = 2^{90.2}$. However, there are at most 2^{32} IVs in the ID column, the number N_{ID} of different IV pairs stored in the pre-computed list is less than 2^{31} , and therefore the overall data complexity can be cut down to $2^{31} \times N = 2^{75.89}$. The attacks in [DGSS23] can also be analyzed in a similar manner. We summarized the results in Table XIX for comparison.

B. Discussion of [BGG⁺23]

In [BGG⁺23], Bellini et al. presented a differential-linear distinguisher of five-round ChaCha with MILP, and presented vast improved differential-linear cryptanalysis of ChaCha. However, since the data complexity of attacks exceeds the limitation 2^{96} for using the technique of Sect 3.1 of [BLT20], the key-recovery attacks on ChaCha7 and ChaCha7.25 [BGG⁺23] can no longer use the technique in [BLT20] to achieve the speedup by a factor of 2^7 .¹

Next, we re-estimate the complexity of the attacks on ChaCha7 and ChaCha7.25 [BGG⁺23]. According to Sect. 2.1 in [BLT20], the correlation of the forward 5-round differential-linear distinguisher (derived by DL Distinguisher 2) in [BGG⁺23] is estimated as $\varepsilon_d = prq^2 = 2^{-7} \times 2^{-30.15} \times 2^{-22} = 2^{-41.15}$ which is lower than the one used in [BGG⁺23] by a factor of 2^{-7} . Then, with the same backward approximation in [BGG⁺23], the attack on ChaCha7 is re-estimated with time complexity of $2^{213.8} = 2^{206.8+7}$ and data complexity of $2^{117.81} = 2^{110.81+7}$, and the attack on ChaCha7.25 is re-estimated with time complexity of $2^{245.34} = 2^{238.34+7}$ and data complexity of $2^{122.34} = 2^{115.34+7}$. The results summarized in Table XIX.

C. Discussion of [Dey24]

In [Dey24], Dey presented a way to improve the key-recovery attacks on ChaCha in which they pre-compute and store the values for split target parts in several huge tables, and online combine the corresponding entries of these huge tables by XOR operations. In the following, we discuss the attack complexity of [Dey24].

¹Regarding the issue, we contacted the authors of [BGG⁺23], and they have confirmed it as shown in pages 31 and 33 of their presentation at FSE 2024. The authors recalculated the complexity in their presentation but without providing the details of calculations for their correction.

a) *The Attack on ChaCha7.5 [Dey24]*: The attack on ChaCha7.5 in [Dey24] is worse than a brute-force attack. Following the framework of PNB-based differential cryptanalysis in [AFK⁺08], Dey [Dey24] presented an attack on 7.5 rounds of ChaCha with time complexity of $2^{255.24}$. Since input pairs are used in the differential framework [AFK⁺08], the complexity should be multiplied by a factor of two. Besides, as analyzed in [AFK⁺08], the success probability of attack is about 0.5 since the median of correlation is used. Therefore, the attack on ChaCha7.5 in [Dey24] is less effective than a brute-force attack, similarly as mentioned in [MIM22].

b) *Other Attacks and Memory Access in [Dey24]*: The cost of accessing huge tables is neglected in the attack complexity in [Dey24]; however accessing huge tables is costly from the implementation aspect. Specifically, the time complexity of querying $\hat{\Gamma}_{g,OD_i}$'s from list L_{OD_i} in Line 3 and Line 4 of Algorithm 3 was neglected in the complexity calculation Equation (6) on page 12 in [Dey24]. Taking the 7-round attack as example, the precomputed list L_{OD_i} has size of $2^{161.79}$ bits and contains 2^{68} entries $\hat{\Gamma}_{g,OD_i}$'s, each of $2^{93.79}$ bits, and there are 2^{107} queries on the list L_{OD_i} and each query accesses $2^{93.79}$ bits. We denote its complexity by $2^{107} \times 2^{93.79}$ MA. With taking the cost of memory access into account, we re-estimated the complexity of attacks on ChaCha in [Dey24], which are summarized in Table XIX. The time complexity is re-estimated as $2^t E + 2^a \times 2^b$ MA that is 2^t times of encryption/decryption operations and 2^a times of memory accesses for 2^b -bit elements from table of size $2^x \times 2^b$ bits which consists of 2^x elements and each element has 2^b bits. That is the memory complexity is $2^x \times 2^b$ bits. Among them, the term $2^t E$ is the one claimed in [Dey24] which is calculated by $2^t = \frac{1}{p} \times (\sum_i^k 2^{m_i} \cdot N + 2^m \cdot N \cdot \frac{k-1}{2^{10 \cdot (R-r)}} + 2^{256} \cdot \text{Pr}_{fa} + 2^{256-m})$ where the part in parentheses is Equation (6) on page 12 in [Dey24]. Another term $2^a \times 2^b$ MA is the neglected one in [Dey24] and it is calculated by $2^a \times 2^b = \frac{1}{p} \times 2^m \times 2^b$, that is the cost of 2^m times of memory accesses for 2^b -bit elements from the table of size $2^x \times 2^b$ bits in Line 3 and Line 4 of Algorithm 3 where m is the number of guessed bits in the online phase of key-recovery attack and $2^b = N$, and the whole process need to be repeated $\frac{1}{p}$ times. The notation p is the probability of differential in the first round, and $p = 2^{-7}$ is used for 256-bit ChaCha7 and ChaCha7.25, and $p = 2^{-5}$ is used for 128-bit ChaCha6 and ChaCha6.5. For non-trivial attack parameters, the inequality $m > x$ always holds.